

Bioinformática para Biologia de Sistemas

Ney Lemke

Departamento de Biofísica e Farmacologia

Instituto de Biociências de Botucatu

Universidade Estadual Paulista — UNESP

Botucatu, 2026

Sobre este livro

Este livro reúne o conteúdo das notas de aula do curso *Bioinformática para Biologia de Sistemas*, ministrado no Departamento de Biofísica e Farmacologia da UNESP. O material destina-se a estudantes de graduação e pós-graduação com interesse em modelagem matemática, redes biológicas e análise integrada de dados ômicos.

Versão: 2 de julho de 2026

Licença: Material didático de uso acadêmico.

Prefácio

A Biologia de Sistemas nasceu de uma insatisfação produtiva. Décadas de biologia molecular produziram um catálogo extraordinário de peças — genes, proteínas, metabólitos, vias de sinalização — mas a compreensão de como essas peças se articulam para gerar vida continuava esquiva. Listar os componentes de uma célula não é o mesmo que entender a célula, assim como conhecer cada nota de uma partitura não é suficiente para compreender uma sinfonia.

Esta disciplina propõe uma virada de perspectiva: em vez de isolar elementos, integrá-los. Em vez de descrever, modelar e prever. Em vez de observar, perturbar e medir. O instrumento central dessa nova biologia é a matemática — não como adorno formal, mas como linguagem capaz de capturar a dinâmica, a robustez e a emergência de sistemas complexos.

Este livro reúne o conteúdo do curso *Bioinformática para Biologia de Sistemas*, ministrado no Departamento de Biofísica e Farmacologia da UNESP em Botucatu. Ele não pressupõe formação em matemática avançada, mas tampouco foge das equações quando elas são o caminho mais direto para a clareza. O leitor encontrará aqui um percurso que vai da estrutura molecular do DNA até a medicina personalizada, passando por redes gênicas, cascatas de sinalização, modelos populacionais e biologia sintética.

Como este livro está organizado. A obra divide-se em quatro partes. A **Parte I** estabelece os fundamentos: o que são sistemas biológicos, como se constroem modelos matemáticos e o que as redes estáticas revelam sobre a organização da vida. A **Parte II** aprofunda as ferramentas matemáticas indispensáveis — equações diferenciais, análise de estabilidade, bifurcações, estimação de parâmetros — que permitem transformar observações qualitativas em afirmações quantitativas rigorosas. A **Parte III** aplica esse instrumental aos grandes sistemas biológicos: regulação gênica, proteínas e dobramento, metabolismo, sinalização celular e dinâmica populacional. Por fim, a **Parte IV** explora as fronteiras do campo: análise integrada multi-ômica, fisiologia cardíaca como sistema de controle, medicina de sistemas, engenharia genética e os horizontes abertos pela inteligência artificial aplicada à biologia.

Cada capítulo foi escrito para ser lido de forma contínua, como um texto — não como uma sequência de tópicos desconexos. As equações aparecem quando acrescentam precisão que as palavras não conseguem dar. Os exemplos de código ilustram como os conceitos se traduzem em análise computacional reproduzível. Os exercícios convidam o leitor a sair da posição passiva e colocar a mão na massa.

A missão deste curso, e deste livro, pode ser resumida em uma frase: conectar dados a mecanismos, e números a narrativas biológicas.

Ney Lemke
Botucatu, 2026

Sumário

Parte I

Fundamentos

Capítulo 1

Sistemas Biológicos

A biologia moderna herdou do século XX uma visão predominantemente reducionista: para entender um sistema vivo, decompunha-se em partes cada vez menores, isolavam-se os componentes e estudavam-se em detalhe. Essa estratégia produziu conquistas extraordinárias — a estrutura do DNA, o código genético, as cascatas de sinalização celular. Mas ao mesmo tempo foi ficando claro que a soma das partes não reconstitui o todo. É desse reconhecimento que nasce a Biologia de Sistemas: a disciplina que estuda os organismos como sistemas integrados, onde as propriedades emergem das interações, e não dos elementos isolados.

Este capítulo apresenta os componentes fundamentais que constituem os sistemas biológicos — ácidos nucleicos, proteínas, lipídios, carboidratos — e as principais estruturas celulares que os organizam. Mais do que um catálogo de moléculas, o objetivo é mostrar como esses componentes se articulam em redes funcionais que serão modeladas matematicamente nos capítulos seguintes.

1.1 O que são Sistemas Biológicos

Definição 1.1 (Sistema Biológico). Um sistema biológico é uma rede complexa de componentes biologicamente relevantes que interagem entre si para realizar funções específicas. A compreensão de um sistema biológico exige não apenas o conhecimento dos componentes individuais, mas sobretudo a análise das interações que emergem entre eles.

Os sistemas biológicos se distinguem de outros sistemas físico-químicos por quatro propriedades fundamentais. A primeira é a **complexidade hierárquica**: a vida se organiza em múltiplos níveis que se encaixam uns nos outros — moléculas formam organelas, organelas compõem células, células constituem tecidos, tecidos integram órgãos, e assim por diante até o organismo completo e as populações. Cada nível

possui propriedades próprias que não se reduzem ao nível inferior.

A segunda propriedade é a **emergência**: o comportamento coletivo do sistema vai além do que qualquer componente isolado poderia exibir. A consciência emerge de neurônios individualmente incapazes de pensar; a homeostase de glicose emerge de células que individualmente secretam ou consomem açúcar; o batimento cardíaco emerge de cardiomiócitos que oscilam de forma coordenada. Compreender a emergência é uma das missões centrais da Biologia de Sistemas.

A terceira propriedade é a **autorregulação**: os sistemas biológicos monitoram continuamente seu estado interno e ajustam seus parâmetros em resposta a perturbações externas. Essa capacidade de *feedback* negativo — que estabiliza o sistema em torno de um ponto de equilíbrio — é o fundamento da homeostase e será formalizada matematicamente no Capítulo 4.

A quarta propriedade é a **adaptabilidade**: ao contrário de máquinas projetadas para operar em condições fixas, os organismos evoluem e se adaptam a ambientes em mudança, tanto em escala de vida individual (plasticidade fenotípica) quanto em escala evolutiva.

1.2 Componentes Biológicos Fundamentais

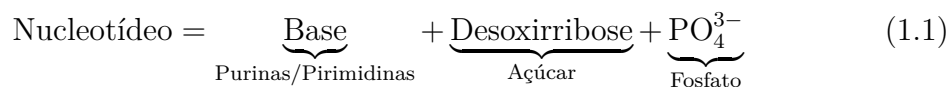
A matéria viva é construída a partir de quatro classes de macromoléculas: ácidos nucleicos, proteínas, lipídios e carboidratos. Cada classe possui um papel estrutural e funcional distinto, mas todas interagem de modo integrado nos processos celulares.

1.2.1 Ácidos Nucleicos: DNA e RNA

O DNA (*ácido desoxirribonucleico*) é o repositório da informação genética. Sua estrutura em dupla hélice, elucidada por Watson e Crick em 1953 com base nos dados de difração de raios-X de Rosalind Franklin, consiste em duas cadeias polinucleotídicas antiparalelas mantidas unidas por ligações de hidrogênio entre pares de bases complementares: adenina (A) pareia com timina (T) por duas ligações, e guanina (G) pareia com citosina (C) por três ligações. Essa assimetria energética — G-C é mais estável que A-T — tem consequências práticas para a desnaturação e a replicação do DNA.

Definição 1.2 (Nucleotídeo). A unidade básica do DNA é o nucleotídeo, composto por três elementos: uma base nitrogenada (purina ou pirimidina), o açúcar desoxirribose e um grupo fosfato. As purinas — adenina e guanina — possuem estrutura de anel duplo, enquanto as pirimidinas — timina e citosina — possuem anel simples. A

composição química do nucleotídeo pode ser expressa como:



A dupla hélice do DNA possui parâmetros estruturais precisos: diâmetro de aproximadamente 2 nm, comprimento de 3,4 nm por volta completa, 10 pares de bases por volta e ângulo de rotação de 36° entre pares consecutivos. Essa geometria define o sulco maior e o sulco menor — regiões de acesso diferencial para proteínas que leem a informação genética. A forma fisiológica predominante é a B-DNA; em condições de baixa hidratação, adota a forma A-DNA; e em sequências alternadas de purinas e pirimidinas pode ocorrer a rara forma Z-DNA, com hélice levogira.

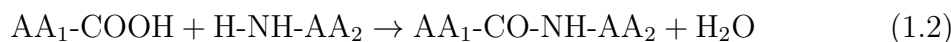
O RNA (*ácido ribonucleico*) difere do DNA em três aspectos: utiliza ribose no lugar de desoxirribose, substitui a timina pela uracila (U) e é tipicamente de fita simples. Embora seja fita simples, o RNA pode formar estruturas secundárias e terciárias complexas por pareamento intramolecular — hairpin loops, bulges, alças internas e pseudonós — que são essenciais para suas funções catalíticas e regulatórias.

As principais classes de RNA são o mRNA (mensageiro, que transporta a informação do DNA ao ribossomo), o tRNA (transportador, que carrega os aminoácidos durante a tradução) e o rRNA (ribossomal, componente estrutural e catalítico dos ribossomos). Além dessas classes clássicas, os RNAs regulatórios — microRNAs, siRNAs e lncRNAs — emergiram como componentes centrais dos sistemas de controle pós-transcricional, formando redes de regulação que serão tratadas no Capítulo 6.

1.2.2 Proteínas: Máquinas Moleculares

As proteínas são os principais executores das funções celulares. Construídas a partir de combinações dos 20 aminoácidos padrão codificados geneticamente, cada proteína dobra em uma estrutura tridimensional única que determina sua função. A organização estrutural das proteínas é descrita em quatro níveis hierárquicos.

A **estrutura primária** é a sequência linear de aminoácidos ligados por ligações peptídicas. A formação de cada ligação peptídica libera uma molécula de água:



A **estrutura secundária** emerge do dobramento local da cadeia polipeptídica estabilizado por ligações de hidrogênio entre grupos C=O e N-H do esqueleto peptídico. Os dois elementos mais comuns são a α -hélice — hélice destrogira com 3,6 resíduos por volta, onde o grupo carbonila do resíduo i faz ligação de hidrogênio com o grupo amino do resíduo $i + 4$, com ângulos de Ramachandran $\phi \approx -60^\circ$ e $\psi \approx -45^\circ$ — e a

folha- β , estrutura mais estendida onde as ligações de hidrogênio ocorrem entre fitas paralelas ou antiparalelas.

A **estrutura terciária** é o arranjo tridimensional completo de uma cadeia polipeptídica, estabilizado por interações de longo alcance: ligações de hidrogênio, interações iônicas, forças hidrofóbicas e pontes dissulfeto entre resíduos de cisteína. A maioria das proteínas globulares esconde resíduos apolares no interior e expõe resíduos polares e carregados na superfície voltada ao solvente aquoso.

A **estrutura quaternária** descreve a associação de múltiplas cadeias polipeptídicas (subunidades) em complexos funcionais. A hemoglobina, por exemplo, é um complexo $\alpha_2\beta_2$ no qual a cooperatividade entre as quatro subunidades permite a ligação alostérica de oxigênio — um dos exemplos mais estudados de regulação de sistemas proteicos.

1.2.3 Metabolismo e Enzimas

Definição 1.3 (Metabolismo). O metabolismo é o conjunto de reações químicas que ocorrem nas células para manter a vida. Divide-se em catabolismo — reações que degradam moléculas e liberam energia — e anabolismo — reações que sintetizam moléculas complexas a partir de precursores simples, consumindo energia.

As vias metabólicas centrais incluem a glicólise, o ciclo de Krebs e a fosforilação oxidativa. A glicólise converte uma molécula de glicose em dois piruvatos, com balanço líquido de dois ATP e dois NADH:



As enzimas são proteínas que catalisam reações bioquímicas específicas sem serem consumidas. A cinética de Michaelis-Menten descreve a velocidade de reação em função da concentração de substrato:

$$v = \frac{V_{\max}[S]}{K_M + [S]} \quad (1.4)$$

onde $V_{\max}$ é a velocidade máxima, $[S]$ é a concentração do substrato e K_M é a constante de Michaelis — a concentração de substrato para a qual $v = V_{\max}/2$. Quando $[S] \gg K_M$, a enzima opera em saturação; quando $[S] \ll K_M$, a velocidade é aproximadamente linear em $[S]$. Essa equação, derivada por Leonor Michaelis e Maud Menten em 1913, é a base quantitativa da enzimologia e será revisitada nos modelos de redes metabólicas do Capítulo 8.

O código a seguir implementa e plota a equação de Michaelis-Menten em Python:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def michaelis_menten(S, Vmax, Km):
5     """Calcula velocidade pela equação de Michaelis-
6         Menten."""
7
8     return (Vmax * S) / (Km + S)
9
10 Vmax, Km = 100, 5 # umol/min e mM
11 S = np.linspace(0, 50, 200)
12 v = michaelis_menten(S, Vmax, Km)
13
14 plt.plot(S, v, 'b-', linewidth=2, label='v(S)')
15 plt.axhline(Vmax, color='r', linestyle='--', label=f'Vmax = {Vmax}')
16 plt.axvline(Km, color='g', linestyle='--', label=f'Km = {Km} mM')
17 plt.xlabel('[S] (mM)'); plt.ylabel('v (mol /min)')
18 plt.legend(); plt.tight_layout(); plt.show()

```

Listing 1.1: Simulação da cinética de Michaelis-Menten

1.3 Organização Celular

A célula é a unidade fundamental da vida. Dois domínios principais de organização celular coexistem na biosfera: os **procariontes** — bactérias e arqueas — que não possuem núcleo delimitado por membrana, têm DNA circular e ribossomos 70S; e os **eucariontes** — animais, plantas, fungos e protistas — que possuem núcleo definido, DNA linear associado a histonas, organelas membranosas e ribossomos 80S.

A compartimentalização da célula eucarionte em organelas é uma solução evolutiva para um problema de incompatibilidade bioquímica: a síntese e a degradação de ácidos graxos utilizam os mesmos intermediários, mas devem ocorrer em condições opostas — síntese no citoplasma, degradação (β -oxidação) na mitocôndria. A separação física permite que ambos os processos coexistam sem interferência mútua.

A mitocôndria merece atenção especial: é a principal responsável pela produção de ATP por fosforilação oxidativa, gerando aproximadamente 34 ATP a partir da cadeia de transporte de elétrons, somados aos 2 ATP da glicólise e 2 ATP do ciclo de Krebs, totalizando cerca de 38 ATP por molécula de glicose em condições ideais.

O potencial eletroquímico iônico através da membrana mitocondrial interna obedece à equação de Nernst:

$$E_{ion} = \frac{RT}{zF} \ln \frac{[ion]_{ext}}{[ion]_{int}} = \frac{58 \text{ mV}}{z} \log_{10} \frac{[ion]_{ext}}{[ion]_{int}} \quad (1.5)$$

onde R é a constante dos gases, T a temperatura absoluta, z a valência iônica e F a constante de Faraday. Esse potencial transmembranar é a força motriz da síntese de ATP pela ATP sintase.

1.4 Integração: da Molécula ao Sistema

Definição 1.4 (Biologia de Sistemas). A Biologia de Sistemas é o estudo de sistemas biológicos por meio da análise integrada de múltiplos níveis de organização — genoma, transcriptoma, proteoma, metaboloma — com o objetivo de compreender como as propriedades emergentes de organismos vivos surgem das interações entre seus componentes moleculares.

O fluxo de informação biológica segue a lógica do Dogma Central: DNA é transcrito em RNA, que é traduzido em proteína, que catalisa reações que produzem metabólitos, que por sua vez influenciam a expressão gênica por mecanismos de feedback. Esse circuito não é unidirecional: proteínas reguladoras atuam sobre a transcrição do DNA; metabólitos ativam ou inibem enzimas por mecanismos alostéricos; RNAs não-codificantes modulam a estabilidade de outros RNAs. A célula é, portanto, um sistema de controle com múltiplas alças de retroalimentação — exatamente o tipo de objeto que a Biologia de Sistemas está equipada para analisar.

A abordagem reducionista — estudar partes isoladas — e a abordagem sistêmica — estudar interações e comportamento emergente — não são opostas, mas complementares. O sucesso da biologia molecular do século XX gerou a matéria-prima; a Biologia de Sistemas fornece o enquadramento teórico para integrá-la em modelos preditivos. Nos capítulos seguintes, veremos como equações diferenciais, teoria de redes e algoritmos computacionais se transformam nas ferramentas centrais dessa integração.

1.5 Exercícios

Exercício 1.1. Descreva as quatro propriedades fundamentais dos sistemas biológicos — complexidade hierárquica, emergência, autorregulação e adaptabilidade — e dê um exemplo concreto de cada uma em sistemas celulares.

Exercício 1.2. A constante de Michaelis K_M de uma enzima é 2 mM para o substrato A e 20 mM para o substrato B. (a) Qual substrato tem maior afinidade pela enzima? (b) A concentração intracelular típica de A é 1 mM e de B é 40 mM. Para qual substrato a enzima opera mais próximo de $V_{\max}$?

Exercício 1.3. Compare células procariontes e eucariontes quanto a: (a) organização do material genético, (b) presença de organelas, (c) tamanho típico e (d) exemplos de organismos. Por que a compartimentalização das eucariontes representa uma vantagem para processos metabólicos incompatíveis?

Exercício 1.4. O Dogma Central da biologia molecular descreve o fluxo de informação $\text{DNA} \rightarrow \text{RNA} \rightarrow \text{Proteína}$. Cite dois exemplos de processos biológicos que representam exceções ou extensões a esse fluxo linear, e explique como eles modificam o panorama da regulação gênica.

1.6 Notas Bibliográficas

A estrutura em dupla hélice do DNA foi publicada por Watson e Crick em 1953 na *Nature* (*A structure for deoxyribose nucleic acid*), um artigo de apenas uma página que revolucionou a biologia. A contribuição decisiva dos dados de difração de Rosalind Franklin é documentada em detalhe em Maddox (2002), *Rosalind Franklin: The Dark Lady of DNA*.

A cinética de Michaelis-Menten foi formulada originalmente em Michaelis e Menten (1913), *Die Kinetik der Invertinwirkung*, *Biochemische Zeitschrift*. A derivação moderna usando o estado de equilíbrio quasi-estacionário é atribuída a Briggs e Haldane (1925). Uma discussão rigorosa encontra-se em Segel (1988), *On the validity of the steady state assumption of enzyme kinetics*.

Para uma introdução abrangente à Biologia de Sistemas, recomenda-se Alon (2007), *An Introduction to Systems Biology*, que cobre motifs de redes e princípios de design de redes biológicas. O livro de Klipp et al. (2009), *Systems Biology: A Textbook*, oferece cobertura mais ampla dos aspectos computacionais e quantitativos. A perspectiva histórica é bem narrada em Noble (2006), *The Music of Life: Biology Beyond Genes*.

Capítulo 2

Modelagem Matemática em Biologia

Modelar é uma das atividades mais antigas da ciência: antes de fazer um experimento caro, lento ou impossível, constrói-se uma representação simplificada do sistema e estuda-se seu comportamento. Na biologia, essa prática ganhou impulso extraordinário a partir do século XX, quando ficou claro que processos como crescimento populacional, dinâmica de infecções e regulação genética obedecem a leis quantitativas que podem ser formalizadas em linguagem matemática. Este capítulo introduz os fundamentos da modelagem matemática em biologia, das equações diferenciais ordinárias aos métodos numéricos, terminando com um modelo estocástico clássico: o reptation do DNA.

2.1 O que é um Modelo Matemático

Definição 2.1 (Modelo Matemático). Um modelo matemático é uma representação abstrata de um sistema real usando linguagem matemática — equações, funções, grafos ou algoritmos — que permite fazer previsões quantitativas sobre o comportamento do sistema e testar hipóteses de modo rigoroso.

Nenhum modelo é uma cópia fiel da realidade. George Box, estatístico britânico, formulou o axioma mais honesto da ciência quantitativa: “*All models are wrong, but some are useful.*” A questão não é se o modelo captura todos os detalhes do sistema — ele nunca o fará — mas se captura os aspectos relevantes para a pergunta que se quer responder. Um modelo simples, que responde corretamente uma pergunta delimitada, vale mais do que um modelo complexo que não responde nada com clareza.

A modelagem matemática oferece vantagens concretas ao biólogo experimental. Ela permite testar hipóteses *in silico*, antes de investir tempo e recursos em ban-

cada. Permite otimizar o design experimental, identificando quais medições trarão mais informação sobre parâmetros desconhecidos. Permite identificar componentes chave do sistema — aqueles cuja perturbação tem maior impacto no comportamento emergente. E permite, em última instância, fazer design racional de sistemas biológicos, orientando a engenharia metabólica e a biologia sintética com previsões fundamentadas.

O processo de modelagem é cíclico, não linear. Parte-se do sistema biológico, abstrai-se um modelo, analisam-se suas previsões por simulação, interpretam-se os resultados e retorna-se ao sistema para validação. Os dados experimentais refinam o modelo, que gera novas previsões, que guiam novos experimentos. Esse ciclo iterativo é a essência da abordagem quantitativa em biologia.

2.2 Tipos de Modelos

Os modelos matemáticos podem ser classificados segundo vários critérios, cada um relevante para uma escolha metodológica diferente.

2.2.1 Determinísticos e estocásticos

Nos modelos **determinísticos**, o estado futuro do sistema é completamente determinado pelo estado presente e pelos parâmetros: dado um conjunto inicial de condições, a trajetória é única. Esses modelos são apropriados quando o número de moléculas ou indivíduos é grande o suficiente para que as flutuações estatísticas sejam desprezíveis. Nos modelos **estocásticos**, em contraste, a aleatoriedade é incorporada explicitamente, e o que se obtém são distribuições de probabilidade sobre trajetórias possíveis. Quando o número de moléculas é pequeno — por exemplo, poucos fatores de transcrição numa célula — as flutuações deixam de ser ruído e passam a ser informação: a variabilidade célula a célula em expressão gênica é um fenômeno real, não um artefato, e só pode ser capturada por modelos estocásticos.

2.2.2 Estáticos e dinâmicos

Modelos **estáticos** descrevem o sistema em equilíbrio ou numa configuração que não varia com o tempo: redes de interação proteína-proteína, estruturas de equilíbrio de ligação, concentrações no estado estacionário. Modelos **dinâmicos** capturam a evolução temporal e são tipicamente formulados como equações diferenciais ordinárias (EDOs) para variáveis contínuas, ou como equações de diferenças para variáveis discretas no tempo.

2.2.3 Contínuos e discretos

A escolha entre modelos contínuos e discretos depende da escala do sistema. Quando as variáveis (concentrações, populações) são grandes, a aproximação contínua é válida e conveniente: a dinâmica é descrita por EDOs como

$$\frac{d[X]}{dt} = k_1 - k_2[X]. \quad (2.1)$$

Quando o sistema opera com poucos elementos discretos — dezenas de moléculas de uma espécie rara, eventos raros de mutação, divisão celular de uma única célula — a representação discreta é mais fiel:

$$X(t+1) = X(t) + \text{prod} - \text{deg}, \quad (2.2)$$

onde prod e deg são variáveis aleatórias inteiras.

2.2.4 Empíricos e mecanísticos

Os modelos **empíricos** são ajustados aos dados sem compromisso com os mecanismos subjacentes: regressões, modelos de machine learning, ajustes polinomiais. São rápidos e flexíveis, mas têm poder preditivo limitado fora da faixa de dados usada para o ajuste. Os modelos **mecanísticos**, ao contrário, representam explicitamente os processos físico-químicos e biológicos: leis de conservação, cinéticas de reação, fluxos metabólicos. São mais difíceis de construir e parametrizar, mas podem extrapolar seguros para condições não testadas experimentalmente — e, crucialmente, geram hipóteses mecanísticas que podem ser testadas experimentalmente.

2.3 Equações Diferenciais em Biologia

Definição 2.2 (Equação Diferencial Ordinária). Uma equação diferencial ordinária (EDO) relaciona uma função com suas derivadas em relação a uma única variável independente, geralmente o tempo. A forma geral de um sistema de n EDOs autônomas é

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, \mathbf{p}), \quad (2.3)$$

onde $\mathbf{x}(t) \in \mathbb{R}^n$ é o vetor de variáveis de estado, $\mathbf{p}$ é o vetor de parâmetros e $\mathbf{f}$ é a função que define a dinâmica.

A escolha de uma EDO de primeira ordem para modelar processos biológicos é justificada em muitas situações: a taxa de variação de uma concentração depende do estado presente, não do passado — o sistema é sem memória, ou markoviano. Quando

o passado importa, aparecem equações com retardo (*delay differential equations*), que descrevem, por exemplo, o tempo de maturação de células imunes antes de responder a um estímulo.

2.3.1 Crescimento exponencial e decaimento

O modelo mais simples de crescimento populacional supõe que cada indivíduo se reproduz a uma taxa constante r independentemente dos demais:

$$\frac{dN}{dt} = rN, \quad N(0) = N_0. \quad (2.4)$$

A solução analítica é a exponencial $N(t) = N_0 e^{rt}$. Para $r > 0$ a população cresce sem limite; para $r < 0$ ela decai; para $r = 0$ permanece estacionária. O tempo de duplicação, quando $r > 0$, é $t_2 = \ln 2/r$. O mesmo modelo descreve o decaimento radioativo, com $\lambda > 0$ substituindo $-r$:

$$\frac{dN}{dt} = -\lambda N \quad \Rightarrow \quad N(t) = N_0 e^{-\lambda t}. \quad (2.5)$$

A coincidência formal não é mero acidente: ambos os processos são proporcionais ao número de entidades presentes, sem interações.

2.3.2 Crescimento logístico

O crescimento exponencial é fisicamente impossível: recursos são finitos. Pierre-François Verhulst introduziu em 1838 a equação logística para capturar a limitação por capacidade de suporte K :

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right). \quad (2.6)$$

Quando $N \ll K$, o fator $(1 - N/K) \approx 1$ e o crescimento é praticamente exponencial. Quando $N \rightarrow K$, o fator se anula e o crescimento cessa. A solução analítica tem forma sigmoidal:

$$N(t) = \frac{K N_0 e^{rt}}{K + N_0(e^{rt} - 1)}, \quad (2.7)$$

com ponto de inflexão em $N = K/2$ — onde a taxa de crescimento dN/dt é máxima. Esse comportamento em S é ubíquo em biologia: crescimento de colônias bacterianas, difusão de inovações tecnológicas, proliferação tumoral em estágio inicial.

O seguinte código resolve e visualiza o modelo logístico usando o integrador `odeint` do SciPy:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def logistic(N, t, r, K):
6     """Crescimento logístico de Verhulst."""
7     return r * N * (1 - N / K)
8
9 r, K, N0 = 0.5, 100, 10
10 t = np.linspace(0, 20, 300)
11 N = odeint(logistic, N0, t, args=(r, K)).flatten()
12
13 plt.plot(t, N, 'b-', linewidth=2)
14 plt.axhline(K, color='r', linestyle='--', label=f'K_{K}')
15 plt.xlabel('Tempo'); plt.ylabel('N(t)')
16 plt.legend(); plt.grid(True); plt.tight_layout()

```

Listing 2.1: Modelo de crescimento logístico

2.4 Sistemas de EDOs: O Modelo Predador-Presa

Quando múltiplas espécies ou variáveis interagem, a dinâmica é descrita por um sistema acoplado de EDOs. O exemplo mais clássico é o modelo de Lotka-Volterra, formulado independentemente pelo matemático-estatístico Alfred Lotka (1920) e pelo matemático Vito Volterra (1926), este motivado por dados de capturas de pesca no Adriático durante a Primeira Guerra Mundial — período em que a pesca foi reduzida e a proporção de predadores aumentou, contrariando a intuição.

O sistema descreve a dinâmica de presas V (do italiano *vittime*) e predadores P (do italiano *predatori*):

$$\frac{dV}{dt} = \underbrace{\alpha V}_{\text{crescimento}} - \underbrace{\beta VP}_{\text{predação}} \quad (2.8)$$

$$\frac{dP}{dt} = \underbrace{\delta \beta VP}_{\text{reprodução}} - \underbrace{\gamma P}_{\text{morte}}. \quad (2.9)$$

As presas crescem exponencialmente na ausência de predadores (taxa α) e são removidas a uma taxa proporcional ao número de encontros VP . Os predadores ganham biomassa ao consumir presas (eficiência de conversão δ) e morrem natural-

mente a uma taxa γ . O produto VP representa a probabilidade de encontro entre as duas populações em campo aleatório — uma hipótese simplificadora, mas robusta em muitas condições.

O sistema possui dois pontos de equilíbrio: a origem $(0,0)$ (extinção mútua) e o equilíbrio não-trivial $(V^*, P^*) = (\gamma/\delta\beta, \alpha/\beta)$. A análise do jacobiano nesse ponto revela autovalores puramente imaginários, indicando que as trajetórias são órbitas fechadas no plano de fase — oscilações periódicas conservativas, confirmadas qualitativamente pelos dados de Volterra.

```

1 from scipy.integrate import odeint
2 import numpy as np, matplotlib.pyplot as plt
3
4 def lotka_volterra(y, t, alpha, beta, delta, gamma):
5     V, P = y
6     return [alpha*V - beta*V*P,
7             delta*beta*V*P - gamma*P]
8
9 alpha, beta, delta, gamma = 1.0, 0.1, 0.75, 1.5
10 y0, t = [10, 5], np.linspace(0, 50, 2000)
11 sol = odeint(lotka_volterra, y0, t,
12             args=(alpha, beta, delta, gamma))
13
14 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
15 ax1.plot(t, sol[:,0], 'b-', label='Presas')
16 ax1.plot(t, sol[:,1], 'r-', label='Predadores')
17 ax1.set(xlabel='Tempo', ylabel='Populacao'); ax1.legend()
18
19 ax2.plot(sol[:,0], sol[:,1], 'g-') # retrato de fase
20 ax2.set(xlabel='Presas (V)', ylabel='Predadores (P)')
21 ax2.set_title('Retrato de fase')
22 plt.tight_layout()

```

Listing 2.2: Sistema predador-presa de Lotka-Volterra

2.5 Análise de Equilíbrio e Estabilidade

Definição 2.3 (Ponto de Equilíbrio). Um ponto de equilíbrio (ou estado estacionário) de um sistema dinâmico é um estado $\mathbf{x}^*$ tal que $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$ — a derivada temporal é nula e o sistema permanece indefinidamente nesse estado na ausência de perturbações.

A estabilidade de um equilíbrio é analisada por linearização. Dada uma perturbação pequena $\delta \mathbf{x} = \mathbf{x} - \mathbf{x}^*$, a dinâmica linearizada é

$$\frac{d\delta \mathbf{x}}{dt} = J(\mathbf{x}^*) \delta \mathbf{x}, \quad (2.10)$$

onde J é a matriz jacobiana avaliada no equilíbrio, com entradas $J_{ij} = \partial f_i / \partial x_j|_{\mathbf{x}^*}$. A solução geral é uma combinação de exponenciais $e^{\lambda_k t}$, onde λ_k são os autovalores de J . O equilíbrio é **assintoticamente estável** se todos os autovalores têm parte real negativa; é **instável** se ao menos um tem parte real positiva.

Para o crescimento logístico, o jacobiano é escalar: $J(N^*) = r(1 - 2N^*/K)$. No equilíbrio trivial $N^* = 0$, tem-se $J = r > 0$ — instável. No equilíbrio não-trivial $N^* = K$, tem-se $J = -r < 0$ — estável. A população converge, portanto, para a capacidade de suporte, independentemente da condição inicial (desde que $N_0 > 0$).

2.6 Métodos Numéricos

A grande maioria dos sistemas de EDOs biologicamente realistas não possui solução analítica em forma fechada. A não-linearidade, a alta dimensionalidade e a rigidez (*stiffness* — presença de constantes de tempo muito diferentes) tornam a integração numérica indispensável.

2.6.1 Método de Euler

O método de Euler é o integrador mais simples: aproxima a derivada por diferença finita à frente e avança o sistema um passo de tamanho h :

$$\mathbf{x}(t + h) = \mathbf{x}(t) + h \cdot \mathbf{f}(\mathbf{x}(t), t). \quad (2.11)$$

O erro local por passo é $O(h^2)$ e o erro global acumulado é $O(h)$ — diz-se que o método é de primeira ordem. Para passos pequenos, o erro é aceitável; para passos grandes, a imprecisão se acumula rapidamente e a solução pode tornar-se instável.

2.6.2 Runge-Kutta de 4ª ordem

O método de Runge-Kutta de quarta ordem (RK4) avalia a derivada em quatro pontos por passo e combina as avaliações com pesos cuidadosamente escolhidos:

$$k_1 = \mathbf{f}(\mathbf{x}_n, t_n) \quad (2.12)$$

$$k_2 = \mathbf{f}\left(\mathbf{x}_n + \frac{h}{2}k_1, t_n + \frac{h}{2}\right) \quad (2.13)$$

$$k_3 = \mathbf{f}\left(\mathbf{x}_n + \frac{h}{2}k_2, t_n + \frac{h}{2}\right) \quad (2.14)$$

$$k_4 = \mathbf{f}(\mathbf{x}_n + hk_3, t_n + h) \quad (2.15)$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad (2.16)$$

O erro local é $O(h^5)$ e o erro global é $O(h^4)$ — quatro ordens de grandeza mais preciso que Euler para o mesmo passo. Em Python, o `odeint` do SciPy implementa o método de Adams-Moulton com controle adaptativo de passo, que ajusta h automaticamente para manter o erro abaixo de uma tolerância especificada. Para sistemas rígidos, o `solve_ivp` com método Radau ou BDF é mais eficiente.

2.7 Análise de Sensibilidade

A análise de sensibilidade responde à pergunta: quão sensível é a saída do modelo a variações nos parâmetros de entrada? Ela é indispensável tanto para validar modelos (um modelo robusto não deve depender criticamente de parâmetros mal estimados) quanto para identificar alvos de intervenção (parâmetros aos quais a saída é muito sensível são candidatos naturais a alvos terapêuticos ou de engenharia).

O **coeficiente de sensibilidade local** de uma variável de estado x em relação a um parâmetro p é a derivada parcial normalizada:

$$S_{x,p} = \frac{\partial \ln x}{\partial \ln p} = \frac{p}{x} \frac{\partial x}{\partial p}. \quad (2.17)$$

Um coeficiente $S_{x,p} = 2$ significa que um aumento de 1% em p produz um aumento de 2% em x . A análise local é rápida e pode ser feita analiticamente, mas captura apenas o comportamento em torno de um ponto nominal de parâmetros.

A **análise de sensibilidade global** (métodos de Sobol, FAST, amostragem Latin Hypercube) explora todo o espaço de parâmetros e decompõe a variância total da saída em contribuições de cada parâmetro e de suas interações. É computacionalmente mais custosa, mas fornece uma imagem completa da robustez do modelo e é especialmente valiosa quando a incerteza paramétrica é grande.

2.8 Boas Práticas em Modelagem

A modelagem matemática é uma arte tanto quanto uma ciência. Algumas práticas, desenvolvidas ao longo de décadas de experiência coletiva, reduzem erros frequentes e aumentam a credibilidade dos resultados.

A primeira é a **parcimônia**: comece com o modelo mais simples que responde a pergunta, e adicione complexidade apenas quando os dados exigirem. Einstein teria dito que as coisas devem ser tão simples quanto possível, mas não mais simples. Um modelo com dez parâmetros ajustados a dez pontos de dados não tem poder preditivo; um modelo com dois parâmetros ajustados a dez pontos e validado em dez outros pontos independentes é genuinamente informativo.

A segunda é a **verificação de unidades**: toda equação deve ser dimensionalmente consistente. Somas de concentrações com taxas, ou de tempos com comprimentos, são sempre erros. A análise dimensional frequentemente revela quais combinações de parâmetros realmente importam.

A terceira é a **validação cruzada**: ajustar o modelo a um conjunto de dados e testá-lo em dados independentes, idealmente de condições diferentes das usadas no ajuste. Um alto R^2 no conjunto de treinamento não garante poder preditivo.

A quarta é a **quantificação de incerteza**: reportar não apenas os valores ótimos dos parâmetros, mas também seus intervalos de confiança, e propagar essa incerteza para as predições. Modelos que apresentam predições pontuais sem incerteza transmitem uma confiança que raramente é justificada.

2.9 Exercícios

Exercício 2.1. Considere um modelo de crescimento logístico com colheita constante H :

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right) - H.$$

(a) Encontre os pontos de equilíbrio em função de r , K e H . (b) Analise a estabilidade para $H < rK/4$ e $H > rK/4$. (c) O que acontece biologicamente quando $H > rK/4$? Implemente e visualize.

Exercício 2.2. Implemente o método de Euler para resolver a EDO $dy/dt = -y + \sin(t)$, $y(0) = 1$. Compare com a solução analítica $y(t) = \frac{1}{2}(\sin t - \cos t) + \frac{3}{2}e^{-t}$ para passos $h \in \{0.5, 0.1, 0.01\}$. Plote o erro absoluto em função do tempo para cada passo e verifique que o erro global é proporcional a h .

Exercício 2.3. Considere o sistema de competição entre duas espécies:

$$\begin{aligned}\frac{dN_1}{dt} &= r_1 N_1 \left(1 - \frac{N_1 + \alpha N_2}{K_1} \right), \\ \frac{dN_2}{dt} &= r_2 N_2 \left(1 - \frac{N_2 + \beta N_1}{K_2} \right).\end{aligned}$$

(a) Implemente e simule para $\alpha = \beta = 0.5$ e $\alpha = \beta = 1.5$. (b) Encontre os pontos de equilíbrio analiticamente. (c) Demonstre numericamente que quando $\alpha\beta < 1$ a coexistência é possível, e quando $\alpha\beta > 1$ uma espécie exclui a outra (princípio de exclusão de Gause).

Exercício 2.4. Estenda o modelo SIR para incluir vacinação à taxa ν (fração de suscetíveis vacinados por unidade de tempo):

$$\begin{aligned}\frac{dS}{dt} &= -\beta \frac{SI}{N} - \nu S, \\ \frac{dI}{dt} &= \beta \frac{SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I + \nu S.\end{aligned}$$

(a) Derive a expressão para o R_0 efetivo em função de ν . (b) Determine a taxa de vacinação mínima ν^* para controlar a epidemia ($R_0^{\text{eff}} < 1$). (c) Simule e compare o pico de infectados com e sem vacinação.

Exercício 2.5. Simule o modelo de reptation para DNA com $N = 50$, $N = 100$ e $N = 200$ segmentos. (a) Para campo nulo ($E = 0$), calcule o deslocamento quadrático médio (MSD) em função do tempo e verifique o regime difusivo $\langle x^2 \rangle \propto t$. (b) Extraia os coeficientes de difusão $D(N)$ e verifique a relação $D \propto N^{-2}$. (c) Para campo elétrico $E = 0.2 k_B T / (qb)$, calcule a velocidade de deriva e verifique $\mu \propto N^{-1}$.

2.10 Notas Bibliográficas

A obra de referência para modelagem matemática em biologia é Murray (2002), *Mathematical Biology I: An Introduction*, Springer — texto canônico com cobertura rigorosa de modelos populacionais, reação-difusão e padrões espaciais. Para biologia celular e molecular, Edelstein-Keshet (2005), *Mathematical Models in Biology* (SIAM), é acessível e rica em exemplos.

O modelo SIR original está em Kermack e McKendrick (1927), *A contribution to the mathematical theory of epidemics*, *Proc. R. Soc. Lond. A*. Uma revisão moderna acessível encontra-se em Anderson e May (1992), *Infectious Diseases of Humans*, Oxford University Press.

O modelo de reptation foi introduzido por de Gennes (1971), *Reptation of a polymer chain in the presence of fixed obstacles*, *J. Chem. Phys.*, e estendido ao caso com campo elétrico por Duke (1989), *Tube model of field-inversion gel electrophoresis*, *Phys. Rev. Lett.*

Para métodos numéricos, Press et al. (2007), *Numerical Recipes*, é a referência clássica. Em Python, a documentação do `scipy.integrate` cobre os integradores disponíveis com orientações práticas sobre a escolha do método mais adequado para diferentes regimes de rigidez.

Capítulo 3

Redes Estáticas e Teoria de Grafos

3.1 Introdução

Visão Geral do Capítulo (1/2).

Objetivos de Aprendizagem

Compreender os fundamentos de teoria dos grafos, Analisar propriedades topológicas de redes biológicas e Explorar modelos de redes small-world e scale-free.

Visão Geral do Capítulo (2/2).

Por que Estudar Redes Biológicas?

Entender organização e função de sistemas biológicos, Identificar alvos terapêuticos e biomarcadores, Predizer efeitos de perturbações no sistema e Descobrir princípios de design evolutivo.

O que são Redes Biológicas?.

Definição (Rede Biológica).

Uma rede biológica é uma representação matemática de interações entre componentes biológicos (genes, proteínas, metabólitos) como um grafo, onde nós representam entidades e arestas representam relações.

Estrutura modular, Hierarquia organizacional, Robustez a perturbações e Propriedades emergentes.

Tipos de Redes Biológicas (1/2).

Redes de Interação Proteica (PPI)

Nós: proteínas, Arestas: interações físicas, Não-direcionadas e Ex: complexos proteicos.

Redes Metabólicas

Nós: metabólitos ou reações, Arestas: conversões enzimáticas, Direcionadas e Ex: glicólise, ciclo de Krebs.

Tipos de Redes Biológicas (2/2).

Redes Regulatórias Gênicas

Nós: genes/fatores de transcrição, Arestas: regulação transcricional, Direcionadas (ativação/repressão) e Ex: circuitos genéticos.

Redes de Sinalização

Nós: proteínas sinalizadoras, Arestas: cascatas de sinalização, Direcionadas e Ex: vias MAPK, PI3K-AKT.

3.2 Fundamentos de Teoria dos Grafos

Conceitos Básicos de Grafos.

Definição (Grafo).

Um grafo $G = (V, E)$ é composto por:

V : conjunto de vértices (nós) e E : conjunto de arestas (conexões) entre vértices.

— Grafo Não-Direcionado —

Arestas sem direção: $(u, v) = (v, u)$

Grafo Direcionado (Dígrafo)

Arestas com direção: $(u, v)(v, u)$

Representação de Grafos: Matriz de Adjacência.

Definição (Matriz de Adjacência).

Matriz A de dimensão nn onde $n = |V|$: $A_{ij} = 1$ se existe aresta de i para j , e $A_{ij} = 0$ caso contrário.

$A =$

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

Propriedades

Simétrica para grafos não-direcionados, Diagonal zero (sem auto-loops) e $\sum_j A_{ij} = k_i$ (grau do vértice i).

Representação de Grafos: Matriz de Incidência.

Definição (Matriz de Incidência).

Matriz B de dimensão nm onde $n = |V|$ e $m = |E|$:

$$B_{ie} = \begin{cases} 1 & \text{se vértice } i \text{ é origem da aresta } e \\ -1 & \text{se vértice } i \text{ é destino da aresta } e \\ 0 & \text{caso contrário} \end{cases}$$

Vantagens

Útil para representar grafos esparsos, Facilita análise de fluxos em redes metabólicas e Cada coluna representa uma reação bioquímica.

Grau de um Vértice.

Definição (Grau (Degree)).

O grau de um vértice é o número de arestas conectadas a ele.

$$k_i = \sum_j A_{ij}.$$

-

In-degree: $k_i^{\text{in}} = \sum_j A_{ji}$ (arestas que chegam) e Out-degree: $k_i^{\text{out}} = \sum_j A_{ij}$ (arestas que saem).

$$k_A = 4$$

$$k = 1$$

— IMP:

$$\text{Grau médio: } k = \frac{1}{n} \sum_i k_i = \frac{2|E|}{|V|}$$

Caminhos e Ciclos.

Definição (Caminho).

Sequência de vértices $v_1, v_2, \dots, v_k$ onde existe aresta entre v_i e v_{i+1} .

número de arestas e sem repetição de vértices.

Definição (Ciclo).

Caminho fechado: $v_1 = v_k$

Importância Biológica

Caminhos: vias de sinalização, rotas metabólicas e Ciclos: feedback loops, ciclos metabólicos.

Componentes Conectados.

Definição (Componente Conectado).

Subconjunto máximo de vértices tal que existe um caminho entre qualquer par de vértices.

— IMP:

O maior componente conectado é chamado de . Em redes biológicas, geralmente contém a maioria dos nós.

Diâmetro e Comprimento Médio de Caminho.

Definição (Distância entre Vértices).

$d(u, v)$ = comprimento do menor caminho entre u e v (geodésica)

Definição (Diâmetro).

Maior distância entre qualquer par de vértices:

$$D = \max_{u, v \in V} d(u, v)$$

Definição (Caminho Médio).

$$L = \frac{1}{n(n-1)} \sum_{u \neq v} d(u, v)$$

— Propriedade Small-World —

Muitas redes biológicas têm:

Diâmetro pequeno, $L(n)$ e Alcance rápido entre nós.

Fenômeno Six Degrees

Na maioria das redes sociais e biológicas, $L6$

Criando e Analisando Grafos com NetworkX (1/2).

```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 # Criar grafo nao-direcionado
5 G = nx.Graph()
6
7 # Adicionar nos e arestas
8 G.add_nodes_from([1, 2, 3, 4, 5])
9 G.add_edges_from([(1,2), (1,3), (2,3), (3,4), (4,5)])
10
11 # Propriedades basicas
12 print(f"Numero de nos: {G.number_of_nodes()}")
13 print(f"Numero de arestas: {G.number_of_edges()}")
```

Listing 3.1: Exemplo básico com NetworkX

Criando e Analisando Grafos com NetworkX (2/2).

```
1 # Calcular distancias
2 print(f"Diametro: {nx.diameter(G)}")
3 print(f"Caminho medio: {nx.average_shortest_path_length(G)}")
4
5 # Visualizar
6 nx.draw(G, with_labels=True, node_color='lightblue',
7         node_size=500, font_size=16)
8 plt.show()
```


Análise de Matriz de Adjacência (1/2).

```

1 import numpy as np
2 import networkx as nx
3
4 # Criar grafo de exemplo
5 G = nx.Graph()
6 G.add_edges_from([(1,2), (1,3), (2,3), (3,4)])
7
8 # Obter matriz de adjacência
9 A = nx.adjacency_matrix(G).todense()
10 print("Matriz de Adjacência:")
11 print(A)
12
13 # Calcular graus a partir da matriz
14 degrees = np.sum(A, axis=1)
15 print(f"\nGraus dos vertices: {degrees.T}")

```

Análise de Matriz de Adjacência (2/2).

```

1 # Matriz de adjacência elevada ao quadrado
2 A2 = np.linalg.matrix_power(A, 2)
3 print("\nMatriz A^2 (caminhos de comprimento 2):")
4 print(A2)
5
6 # Centralidade de autovetor
7 eigenvalues, eigenvectors = np.linalg.eig(A)
8 principal_eigenvector = eigenvectors[:, 0]
9 print(f"\nCentralidade de autovetor: {principal_eigenvector}")

```

3.3 Propriedades Topológicas de Redes

Distribuição de Graus.

Definição (Distribuição de Graus).

$P(k)$ = fração de nós na rede com grau k

— Rede Aleatória (Erdős-Rényi) —

Distribuição de Poisson, $P(k) = k!$ e A maioria dos nós tem grau próximo a k .

— Rede Scale-Free —

Lei de potência (power-law), $P(k)k^{-\gamma}$,

tipicamente entre 2 e 3 e Presença de hubs (nós altamente conectados).

— IMP:

A maioria das redes biológicas segue distribuição scale-free!

Visualizando Distribuições de Grau.

— Identificando Power-Law —

Gráfico log-log: $P(k) = -k + c$ (linha reta)

Coefficiente de Agrupamento (Clustering).

Definição (Clustering Local).

$$C_i = 2E_i / k_i(k_i - 1)$$

Definição (Global).

$$C = nC_i$$

— IMP: Indica modularidade!

Redes Small-World (Watts-Strogatz).

Alto coeficiente de agrupamento (como rede regular), Pequeno caminho médio (como rede aleatória), CC_{random} e LL_{random} .

Modelo WS

1. Começar com anel regular

2. Reconectar cada aresta com probabilidade p

3. $p[0, 1]$: regular

aleatória

0

— IMP:

Redes small-world permitem comunicação eficiente com conexões locais e alguns atalhos globais.

Redes Scale-Free (Barabási-Albert).

Modelo de Ligação Preferencial (Preferential Attachment)

Começar com pequeno número de nós m_0 , Adicionar novo nó com m arestas e Conectar a nós existentes com probabilidade:.

$$\Pi(k_i) = \frac{k_i}{\sum_j k_j}$$

“Rico fica mais rico” (hubs atraem mais conexões).

$P(k)k^{-3}$, Hubs naturalmente emergem e Rede sem escala característica.

Exemplos Biológicos

Redes de interação proteica, Redes metabólicas, Redes de regulação gênica e Internet, redes sociais.

Motivos de Rede (Network Motifs).

Definição (Motivo de Rede).

Padrões de interconexão recorrentes que aparecem significativamente mais frequentes que em redes aleatórias.

1

Funções Biológicas

filtro de ruído, detecção de persistência, switch biestável, memória e homeostase, oscilações.

Nós Hub e Vulnerabilidade.

Definição (Hub).

Nó com grau muito maior que a média da rede ($k_i k$)

Centrais para conectividade, Alvos de drogas e terapias, Remoção causa fragmentação e Evolutivamente conservados.

2

— IMP:

Redes scale-free são robustas a falhas aleatórias mas vulneráveis a ataques direcionados em hubs!

Calculando Propriedades de Rede com NetworkX.

```

1 import networkx as nx
2 import numpy as np
3
4 # Criar rede scale-free (Barabasi-Albert)
5 G = nx.barabasi_albert_graph(n=100, m=2)
6
7 # Analise
8 degrees = [G.degree(n) for n in G.nodes()]
9 print(f"Grau_médio: {np.mean(degrees):.2f}")
10
11 # Clustering e Caminho Médio
12 C_global = nx.average_clustering(G)
13 print(f"Clustering_global: {C_global:.3f}")
14
15 if nx.is_connected(G):
16     L = nx.average_shortest_path_length(G)
17     print(f"Caminho_médio: {L:.3f}")
18
19 # Identificar hubs (nos com grau > 2*media)
20 threshold = 2 * np.mean(degrees)
21 hubs = [n for n in G.nodes() if G.degree(n) > threshold]
22 print(f"Hubs_encontrados: {len(hubs)}")

```

Gerando Diferentes Modelos de Rede (1/2).

```

1 import networkx as nx

```

```
2
3 # 1. Rede Aleatoria (Erdos-Renyi)
4 G_random = nx.erdos_renyi_graph(n=50, p=0.1)
5
6 # 2. Rede Small-World (Watts-Strogatz)
7 G_sw = nx.watts_strogatz_graph(n=50, k=4, p=0.3)
8
9 # 3. Rede Scale-Free (Barabasi-Albert)
10 G_sf = nx.barabasi_albert_graph(n=50, m=2)
```

Gerando Diferentes Modelos de Rede (2/2).

```
1 # Comparar propriedades
2 models = [G_random, G_sw, G_sf]
3 names = ['Random', 'Small-World', 'Scale-Free']
4
5 for G, name in zip(models, names):
6     C = nx.average_clustering(G)
7     L = nx.average_shortest_path_length(G) if nx.is_connected(G)
8         else float('inf')
9     degrees = [d for n, d in G.degree()]
10
11     print(f"\n{name}: Clustering: {C:.3f}, Path: {L:.3f}")
```

3.4 Redes Biológicas

Redes de Interação Proteína-Proteína (1/2).

Nós: proteínas, Arestas: interações físicas, Não-direcionadas, Scale-free e Modularidade alta.

Yeast two-hybrid (Y2H), Affinity purification MS e Co-immunoprecipitation.

3

Redes de Interação Proteína-Proteína (2/2).

Aplicações Principais

Predição de função proteica por homologia de interação, Identificação de complexos proteicos e Descoberta de alvos terapêuticos.

Bases de Dados Principais

string-db.org — score de confiança integrado, thebiogrid.org — interações experimentais e ebi.ac.uk/intact — curada manualmente.

Redes Metabólicas.

Duas Representações Principais

nós = metabólitos, arestas = reações e nós = reações, arestas = metabólitos compartilhados.

4

Redes Regulatórias Gênicas (GRN).

Genes alvos, TFs, Elementos (promotores).

Direcionadas, Hierárquicas e Motivos regulatórios, FFLs.

5

Redes de Sinalização Celular (1/2).

Características

Transmissão de sinais extracelulares para resposta intracelular, Cascatas de fosforilação (quinases) e amplificação e Cross-talk entre vias, direcionadas e dinâmicas.

Redes de Sinalização Celular (2/2).

6

Bases de Dados: PPI e Metabolismo.

Redes PPI

string-db.org, thebiogrid.org e ebi.ac.uk/intact.

Redes Metabólicas

genome.jp/kegg, reactome.org e biocyc.org.

Bases de Dados: Regulação e Sinalização.

Redes Regulatórias

E. coli, TFs e Elementos.

Redes de Sinalização

-

vias de sinalização e cross-talk.

— IMP:

Integração de múltiplas bases de dados permite análise multi-ômica!

Acessando Bases de Dados com Python (1/2).

```
1 import requests
2 import networkx as nx
3
4 # Exemplo: Rede PPI do STRING
5 species, gene, thresh = '9606', 'TP53', 700
6 url = f'https://string-db.org/api/json/network?identifiers={
7     gene}&species={species}&required_score={thresh}'
8 response = requests.get(url)
9 data = response.json()
```

Acessando Bases de Dados com Python (2/2).

```
1 # Construir grafo
```

```
2 G = nx.Graph()
3 for interaction in data:
4     n1, n2 = interaction['preferredName_A'], interaction['
        preferredName_B']
5     G.add_edge(n1, n2, weight=interaction['score'])
6
7 print(f"Rede_{gene}: {G.number_of_nodes()}_nos, {G.
    number_of_edges()}_arestas")
```

Exemplo: Rede de Interação de TP53.

Supressor tumoral, “Guardião do genoma”, Hub na rede PPI humana, Interage com > 100 proteínas e Mutado em > 50

MDM2 (regulação negativa), ATM, ATR (sensores de dano), p21 (ciclo celular) e BAX, PUMA (apoptose).

7

— IMP:

Perda de TP53 desconecta múltiplas vias críticas!

3.5 Análise de Controle Metabólico

Introdução à Análise de Controle Metabólico (MCA).

Definição (MCA (Metabolic Control Analysis)).

Teoria quantitativa que analisa como mudanças em parâmetros individuais (atividade enzimática, concentrações) afetam propriedades sistêmicas (fluxos, concentrações).

Questões Fundamentais

Qual enzima controla o fluxo em uma via metabólica?, Como perturbações locais afetam o sistema global?, Onde devemos intervir terapeuticamente? e Como prever efeitos de mutações ou drogas?.

Desenvolvida por

Henrik Kacser, Jim Burns (1973) e Reinhart Heinrich, Tom Rapoport (1974)

Coefficientes de Controle.

Definição (Coeficiente de Controle de Fluxo).

Mede como mudança na atividade da enzima i afeta o fluxo J :

$$C_i^J = \frac{\partial J/J}{\partial E_i/E_i} = \frac{\partial \ln J}{\partial \ln E_i}$$

Elasticidade normalizada do fluxo em relação à enzima.

Definição (Coeficiente de Controle de Concentração).

Mede como mudança na atividade da enzima i afeta a concentração de metabólito S :

$$C_i^S = \frac{\partial S/S}{\partial E_i/E_i} = \frac{\partial \ln S}{\partial \ln E_i}$$

— IMP:

Coeficientes de controle são propriedades sistêmicas, não locais!

Coeficientes de Elasticidade.

Definição (Elasticidade).

Propriedade local que mede sensibilidade da taxa de reação v_i a mudanças em metabólito S :

$$\varepsilon_S^{v_i} = \frac{\partial v_i/v_i}{\partial S/S} = \frac{\partial \ln v_i}{\partial \ln S}$$

Interpretação

> 0 : S ativa a reação (substrato ou ativador), < 0 : S inibe a reação (inibidor),
 $|| > 1$: alta sensibilidade (cooperatividade) e $|| < 1$: baixa sensibilidade.

— Para Cinética de Michaelis-Menten —

$$v = \frac{V_{max}S}{K_M + S} \Rightarrow \varepsilon_S^v = \frac{K_M}{K_M + S}$$

Teoremas de Soma: Concentração.

Controle de Concentração

Para metabólito intermediário S :

$$\sum_{i=1}^n C_i^S = 0$$

Aumento e diminuição de concentração devem se balancear.

— IMP:

Não há uma única enzima limitante! O controle é distribuído.

Teorema da Conectividade.

Relaciona Controle e Elasticidade

Para metabólito intermediário S :

$$\sum_{i=1}^n C_i^J \varepsilon_S^{v_i} = 0$$

Para reações consecutivas $v_1 S v_2$:

$$C_1^J \varepsilon_S^{v_1} + C_2^J \varepsilon_S^{v_2} = 0$$

Interpretação

Liga propriedades locais (elasticidades) a globais (controle), Permite calcular coeficientes de controle a partir de elasticidades e Fundamental para modelagem modular.

Exemplo: Via Linear Simples.

8

$v_1 = v_2 = v_3 = J$ (fluxo constante)

Usando Teoremas de MCA

$$C_1^J + C_2^J + C_3^J = 1 \quad (\text{Soma})$$

$$C_1^J \varepsilon_{S_1}^{v_1} + C_2^J \varepsilon_{S_1}^{v_2} = 0 \quad (\text{Conectividade para } S_1)$$

$$C_2^J \varepsilon_{S_2}^{v_2} + C_3^J \varepsilon_{S_2}^{v_3} = 0 \quad (\text{Conectividade para } S_2)$$

Cálculo de Coeficientes de Controle.

$X_0 S_1 S_2 X_1$

Sistema de Equações

Supondo v_1 não depende de S_1 e v_3 não depende de S_2 :

$$\begin{aligned} \varepsilon_{S_1}^{v_1} &= 0, & \varepsilon_{S_2}^{v_3} &= 0 \\ C_1^J + C_2^J + C_3^J &= 1 \\ C_2^J \varepsilon_{S_1}^{v_2} &= 0 \quad \Rightarrow \quad C_2^J = 0 \text{ ou } \varepsilon_{S_1}^{v_2} = \infty \end{aligned}$$

Se todas elasticidades são similares

Aproximação: controle distribuído igualmente

$$C_1^J \approx C_2^J \approx C_3^J \approx \frac{1}{3}$$

Cada enzima contribui 33

Exemplo Prático: O Início da Glicólise.

Enunciado do Problema

Queremos analisar quantitativamente o controle do fluxo na etapa inicial da glicólise em células tumorais (efeito Warburg). O sistema consiste na fosforilação da Glicose (G) extracelular pela enzima (E_1), produzindo (S), que é então isomerizada pela (E_2) em (P).

$(G)(S)(P)$

Parâmetros do Modelo

Concentração de Glicose mantida fixa por transporte celular: $G = 5.0$ mM, Hexocinase (E_1): $v_1 = GK_M1 + G$, com $V_{max1} = 10.0$ mM/min, $K_M1 = 1.0$ mM e Isomerase (E_2): $v_2 = SK_M2 + S$, com $V_{max2} = 15.0$ mM/min, $K_M2 = 2.0$ mM.

Resolução: Passo 1 (Estado Estacionário).

Determinar o Fluxo da Via (J) e Concentração de G6P (S)

No estado estacionário, as taxas de consumo e produção devem se equilibrar:

$$v_1 = v_2 = J.$$

$\therefore$

Como G é fixo, v_1 funciona como fluxo de entrada constante:

$$v_1 = \frac{10.0 \cdot 5.0}{1.0 + 5.0} = \frac{50.0}{6.0} \approx 8.33 \text{ mM/min} \Rightarrow J = 8.33 \text{ mM/min}$$

$\therefore$

$$v_2(S) = J \Rightarrow \frac{15.0 \cdot S}{2.0 + S} = \frac{25.0}{3.0}$$

Multiplicando em cruz:

$$45.0 \cdot S = 50.0 + 25.0 \cdot S \Rightarrow 20.0 \cdot S = 50.0 \Rightarrow S = 2.50 \text{ mM}$$

: O fluxo estacionário da via é $J = 8.33$ mM/min e a concentração de G6P é $S = 2.5$ mM.

Resolução: Passos 2 e 3 (Elasticidade e Controle).

Passo 2: Calcular as Elasticidades Locais

Mede a sensibilidade de cada reação em relação ao intermediário G6P (S):

$\varepsilon_{S1}^v = 0$ (a Hexocinase não depende de S neste modelo linear simplificado) e $\varepsilon_{S2}^v = K_M2 + S = 2.0 + 2.5 = 4.5044$.

Passo 3: Resolver os Teoremas da MCA

$$\begin{aligned} C_1^J \varepsilon_S^{v_1} + C_2^J \varepsilon_S^{v_2} &= 0 \quad (\text{Conectividade}) \Rightarrow C_1^J \cdot (0) + C_2^J \cdot (0.44) = 0 \Rightarrow C_2^J = 0 \\ C_1^J + C_2^J &= 1 \quad (\text{Soma}) \Rightarrow C_1^J + 0 = 1 \Rightarrow C_1^J = 1 \end{aligned}$$

: $C_H K^J$ (Hexocinase) = 1.0 e $C_P G I^J$ (Isomerase) = 0.0. A Hexocinase detém todo o controle sobre o fluxo; aumentar a Isomerase não altera o fluxo J da via.

Feedback alostérico e a Realidade Celular.

A Realidade Biológica: Inibição por Produto

In vivo, a Hexocinase é inibida por feedback alostérico pelo seu próprio produto S (G6P):

$$v_1 = \frac{V_{max1} \cdot G}{K_{M1} + G} \cdot \frac{1}{1 + S/K_I} \Rightarrow \varepsilon_S^{v_1} < 0 \quad (\text{elasticidade negativa})$$

: A inibição por produto (ε_{S1}^v) redistribui o controle. O Teorema da Conectividade passa a dar:

$$C_1^J \varepsilon_S^{v_1} + C_2^J \varepsilon_S^{v_2} = 0 \Rightarrow \frac{C_1^J}{C_2^J} = -\frac{\varepsilon_S^{v_2}}{\varepsilon_S^{v_1}} > 0$$

Ambas as enzimas passam a compartilhar o controle de fluxo, ilustrando o princípio central da MCA de que !

Simulando MCA com Python.

```

1 import numpy as np
2 from scipy.optimize import fsolve
3
4 # Taxas da Hexocinase (v_hk) e Isomerase (v_pgi)
5 def v_hk(G, Vmax1=10.0, Km1=1.0): return Vmax1 * G / (Km1 + G)
6 def v_pgi(S, Vmax2=15.0, Km2=2.0): return Vmax2 * S / (Km2 + S)
7
8 # Estado estacionario (v_hk = v_pgi)
9 G_fixed = 5.0
10 S_ss = fsolve(lambda S: v_hk(G_fixed) - v_pgi(S), 2.5)[0]
11 J_ss = v_pgi(S_ss)
12
13 print(f"G6P_estacionario_{S_ss}={S_ss:.2f}mM")
14 print(f"Fluxo_estacionario_{J_ss}={J_ss:.2f}mM/min")

```

Cálculo de Coeficientes de Controle (Python).

```

1 def calc_C(idx, delta=0.01):
2     def get_J(v_hk_func, v_pgi_func):
3         S = fsolve(lambda S: v_hk_func(G_fixed) - v_pgi_func(S), 2.5)[0]
4         return v_pgi_func(S)
5
6     if idx == 1:
7         Jb = get_J(lambda g: v_hk(g, 10.0), v_pgi)
8         Jp = get_J(lambda g: v_hk(g, 10.0*(1+delta)), v_pgi)
9     else:
10        Jb = get_J(v_hk, lambda s: v_pgi(s, 15.0))
11        Jp = get_J(v_hk, lambda s: v_pgi(s, 15.0*(1+delta)))
12    return ((Jp - Jb) / Jb) / delta
13
14 print(f"C_HK^J={calc_C(1):.3f}")
15 print(f"C_PGI^J={calc_C(2):.3f}")

```

Limitações da MCA Tradicional.

Desafios da Modelagem Cinética Clássica

Embora a MCA forneça uma descrição quantitativa rigorosa do controle do

metabolismo, sua aplicação prática enfrenta barreiras significativas:

: Exige dados cinéticos detalhados (como K_M , V_{max} , constantes de inibição) para todas as enzimas envolvidas na rede, : Muitos desses parâmetros são difíceis ou impossíveis de se medir de forma confiável in vivo em condições fisiológicas, : As conclusões da MCA (elasticidades e coeficientes de controle) são linearizações válidas estritamente em torno de um . Perturbações grandes ou transições metabólicas invalidam a análise linear local e : Modelar redes metabólicas em escala genômica (GEMs) com cinética detalhada é computacional e experimentalmente inviável atualmente.

A Abordagem Baseada em Restrições (Palsson).

Definição (Abordagem de Palsson (COBRA)).

Proposta por Bernhard Palsson e colaboradores, a modelagem (Constraint-Based Reconstruction and Analysis) foca na reconstrução de redes metabólicas em escala genômica usando propriedades estequiométricas fundamentais, eliminando a dependência de parâmetros cinéticos individuais.

Princípios das Restrições

: Define o estado estacionário estrutural baseado estritamente na estequiometria (Matriz S) e : Delimita o espaço de fluxos permitidos por meio de termodinâmica e capacidade enzimática ($v_{min}v_{max}$).

Espaço de Soluções

As restrições físicas definem um . O fenótipo metabólico deve operar dentro deste espaço. Ferramentas como o FBA (Flux Balance Analysis) buscam soluções ótimas dentro deste cone maximizando objetivos biológicos (ex: produção de biomassa).

Comparativo: MCA vs. Abordagem de Palsson.

[h]

!

Principais Aplicações do FBA.

Definição (FBA (Flux Balance Analysis)).

O FBA é a ferramenta central da abordagem baseada em restrições para calcular fluxos metabólicos estacionários sem requerer cinética detalhada.

Previsões e Resultados Possíveis com FBA

: Predição da taxa máxima de crescimento de biomassa sob diferentes fontes de nutrientes, : Identificação in silico de knockouts de genes ou reações letais (relevante para novos fármacos), : Redirecionamento de fluxos para superproduzir subprodutos industriais (ex: etanol, aminoácidos) e : Mapeamento de relações de troca (envelopes de produção) entre crescimento e produtos.

A modelagem COBRA/FBA delimita a fronteira física de capacidades da rede; a MCA descreve como o controle dinâmico opera localmente dentro dela.

3.6 Exercícios

Exercícios - Parte 1: Teoria dos Grafos.

Questão 1: Matriz de Adjacência

Dada a matriz de adjacência abaixo, desenhe o grafo correspondente e calcule:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

O grau de cada vértice, o grau médio e o número de triângulos.

Questão 2: Caminho e Diâmetro

Determine a distância entre 1 e 4, o diâmetro e o comprimento médio.

Exercícios - Parte 2: Propriedades de Rede.

Questão 3: Coeficiente de Agrupamento

Para um nó com grau $k = 4$ que tem 3 arestas entre vizinhos, calcule C_i .

Questão 4: Distribuição de Graus

Uma rede de 100 nós tem 60 com grau 2, 30 com grau 3, 8 com grau 5, 2 com grau 10.

Calcule k e justifique se é aleatória ou scale-free.

Exercícios - Parte 3: NetworkX.

Questão 5: Implementação

Gere uma rede BA com $n = 50$, $m = 2$. Calcule a distribuição de graus, identifique hubs e compare com rede ER de mesmo tamanho.

Questão 6: Robustez

Simule a remoção de nós (aleatória vs. hubs) em rede scale-free. Qual estratégia fragmenta mais rápido?

Exercícios - Parte 4: MCA.

Questão 7: Elasticidade

Para $v = 100S/(10 + S)$, calcule ε_S^v para $S = 5, 10, 50$.

Questão 8: Teorema da Soma

Em via com 5 enzimas: $C_1^J = 0.4$, $C_2^J = 0.3$, $C_3^J = 0.2$, $C_4^J = 0.05$. Qual é C_5^J ?

Exercícios - Parte 5: Aplicação.

Projeto Prático

Escolha uma via metabólica e construa o grafo no NetworkX. Compare previsões topológicas com dados de MCA da literatura.

3.7 Notas Bibliográficas

Referências Principais.

99

Barabási A.L. (2016). Network Science. Cambridge University Press. Disponível em:

Alon U. (2019). An Introduction to Systems Biology: Design Principles of Biological Circuits, 2nd ed. Chapman & Hall/CRC.

Klipp E. et al. (2016). Systems Biology: A Textbook, 2nd ed. Wiley-VCH.

Fell D. (1997). Understanding the Control of Metabolism. Portland Press.

Heinrich R., Schuster S. (1996). The Regulation of Cellular Systems. Chapman & Hall.

Leitura Complementar.

Livros

Newman M.E.J. (2018). Networks, 2nd ed. Oxford University Press, Voit E.O. (2017). A First Course in Systems Biology, 2nd ed e Palsson B.O. (2015). Systems Biology: Properties of Reconstructed Networks. Cambridge.

Artigos Fundamentais

Barabási A.L., Albert R. (1999). Emergence of scaling in random networks. Science 286:509-512, Watts D.J., Strogatz S.H. (1998). Collective dynamics of small-world networks. Nature 393:440-442 e Kacser H., Burns J.A. (1973). The control of flux. Symp. Soc. Exp. Biol. 27:65-104.

Recursos Online.

Ferramentas e Software

-

(visualização de redes), (R e Python) e (análise interativa).

Bases de Dados

-

-

-

-

Obrigado!

Capítulo 3: Modelos de Redes Estáticas

Modelos de Redes Dinâmicas e Simulação

Dúvidas? Perguntas?

Parte II

Ferramentas Matemáticas

Capítulo 4

A Matemática dos Sistemas Biológicos

A matemática é a linguagem com que a Biologia de Sistemas formula seus argumentos mais precisos. Enquanto a descrição verbal de um processo biológico pode ser ambígua, uma equação diferencial é inequívoca: especifica exatamente como as variáveis de estado evoluem no tempo, dado um conjunto de parâmetros e condições iniciais. Mas a matematização da biologia não é um exercício de formalismo vazio — é uma ferramenta para prever, distinguir hipóteses e projetar experimentos. Este capítulo desenvolve as ferramentas matemáticas indispensáveis para a modelagem de sistemas dinâmicos biológicos, desde os mapas discretos até as bifurcações e a análise de sensibilidade.

4.1 Modelos Discretos e Recursivos

Definição 4.1 (Sistema Dinâmico em Tempo Discreto). Um sistema dinâmico em tempo discreto é aquele em que o tempo avança em passos inteiros $t = 0, 1, 2, \dots$. A dinâmica é descrita por uma relação de recorrência

$$x_{t+1} = f(x_t), \tag{4.1}$$

onde f é uma função que mapeia o estado atual x_t no estado do próximo instante.

Muitos processos biológicos ocorrem naturalmente em etapas discretas: gerações não-sobrepostas de insetos sazonais, divisão celular sincronizada em culturas, ciclos de replicação viral, ou simplesmente coletas experimentais em pontos fixos de tempo. Para esses sistemas, a modelagem discreta é mais natural do que as equações diferenciais contínuas.

4.1.1 O Mapa Logístico e o Caos Determinístico

Em 1976, o ecologista Robert May publicou na *Nature* um artigo que se tornaria clássico: "*Simple mathematical models with very complicated dynamics*". Nele, May demonstrava que uma equação de uma única variável — o mapa logístico — era capaz de exibir um leque de comportamentos que ia da extinção ao equilíbrio estável, passando por oscilações periódicas e chegando ao caos determinístico. O mapa logístico é definido por

$$x_{t+1} = r x_t(1 - x_t), \quad (4.2)$$

onde $x_t \in [0, 1]$ representa a densidade populacional normalizada e $r > 0$ é a taxa de crescimento intrínseca. Apesar de sua aparente simplicidade, o comportamento do sistema depende criticamente do valor de r .

Para $0 < r < 1$, a população se extingue: $x_t \rightarrow 0$. Para $1 < r < 3$, o sistema converge para o ponto fixo estável $x^* = 1 - 1/r$. Quando r ultrapassa 3, emerge um ciclo estável de período 2 — a população oscila entre dois valores. Aumentando r além de aproximadamente 3,44, o período dobra novamente para 4, depois para 8, num processo de duplicação de período (Feigenbaum, 1978) que conduz ao caos para $r > 3,57$. Na região caótica, trajetórias iniciadas em condições iniciais arbitrariamente próximas divergem exponencialmente — o famoso efeito borboleta de Lorenz, aqui num sistema unidimensional.

A estabilidade de um ponto fixo x^* do mapa logístico — onde $f(x^*) = x^*$ — é determinada pela derivada de f nesse ponto. Expandindo em série de Taylor em torno de x^* , a perturbação $\xi_t = x_t - x^*$ evolui segundo

$$\xi_{t+1} \approx f'(x^*) \xi_t \quad \Rightarrow \quad \xi_t \approx [f'(x^*)]^t \xi_0. \quad (4.3)$$

O ponto fixo é **estável** se $|f'(x^*)| < 1$ e **instável** se $|f'(x^*)| > 1$. Para o ponto fixo não-trivial $x^* = 1 - 1/r$, tem-se $f'(x^*) = 2 - r$, de modo que a estabilidade é garantida para $|2 - r| < 1$, ou seja, $1 < r < 3$.

O código a seguir simula o mapa logístico e gera um diagrama de bifurcação, visualizando como o comportamento assintótico varia com r :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 r_vals = np.linspace(2.5, 4.0, 1000)
5 x = 0.5 * np.ones(len(r_vals))
6
7 for _ in range(500):                # transiente
```

```

8     x = r_vals * x * (1 - x)
9
10    r_plot, x_plot = [], []
11    for _ in range(200):          # regime permanente
12        x = r_vals * x * (1 - x)
13        r_plot.extend(r_vals)
14        x_plot.extend(x)
15
16    plt.figure(figsize=(10, 5))
17    plt.plot(r_plot, x_plot, ',k', alpha=0.2)
18    plt.xlabel('r'); plt.ylabel('x*')
19    plt.title('Diagrama de bifurcação o mapa logístico')
20    plt.tight_layout(); plt.show()

```

Listing 4.1: Diagrama de bifurcação do mapa logístico

4.1.2 Processos Estocásticos e Cadeias de Markov

Nem todo processo biológico é determinístico. Quando o número de moléculas ou indivíduos é pequeno, as flutuações estocásticas tornam-se relevantes. Um framework natural para modelar esse regime é o das **cadeias de Markov**, nas quais o estado futuro do sistema depende apenas do estado presente, e não de toda a história passada.

Uma cadeia de Markov em tempo discreto é definida por um conjunto de estados $\{s_1, s_2, \dots, s_n\}$ e por uma *matriz de transição* P , onde P_{ij} é a probabilidade de transitar do estado s_i para o estado s_j num único passo. A distribuição de probabilidade dos estados evolui como

$$\pi_{t+1} = \pi_t P, \quad (4.4)$$

onde π_t é o vetor linha de probabilidades no instante t . No limite de longo prazo, uma cadeia de Markov ergódica converge para a distribuição estacionária π^* satisfazendo $\pi^* = \pi^* P$.

Em biologia, cadeias de Markov descrevem processos como mutações ao longo de gerações, transições entre estados conformacionais de proteínas, dinâmica de canais iônicos (aberto/fechado), ou modelos de expressão gênica em células individuais. No contexto de epidemiologia discreta, os modelos SIR (Suscetível-Infetado-Recuperado) podem ser formulados como cadeias de Markov sobre populações finitas, capturando flutuações de extinção que o modelo determinístico ignora.

4.2 Sistemas de EDOs e Retratos de Fase

A transição dos modelos discretos para os contínuos é natural quando o número de indivíduos é grande e o tempo pode ser tratado como uma variável contínua. O **sistema de equações diferenciais ordinárias** (EDOs) é a linguagem padrão para modelar a dinâmica contínua de sistemas biológicos:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, \boldsymbol{\theta}), \quad (4.5)$$

onde $\mathbf{x}(t) \in \mathbb{R}^n$ é o vetor de variáveis de estado (concentrações, populações, potenciais elétricos) e $\boldsymbol{\theta}$ é o vetor de parâmetros.

O **retrato de fase** é a representação geométrica das trajetórias do sistema no espaço de estados, independentemente do tempo. Cada ponto do espaço de fase representa um estado possível do sistema; as setas indicam a direção e magnitude da derivada temporal. As *nulclinas* — curvas onde $\dot{x}_i = 0$ — são ferramentas gráficas poderosas para identificar pontos de equilíbrio e analisar qualitativamente a dinâmica sem resolver as equações analiticamente.

4.2.1 O Sistema de Lotka-Volterra

O sistema predador-presa de Lotka-Volterra, proposto independentemente por Alfred Lotka (1920) e Vito Volterra (1926), é um dos modelos de EDOs mais estudados em biologia:

$$\frac{dx}{dt} = \alpha x - \beta xy \quad (4.6)$$

$$\frac{dy}{dt} = \delta xy - \gamma y, \quad (4.7)$$

onde x é a população de presas (crescimento intrínseco α , predação β) e y é a população de predadores (eficiência de conversão δ , mortalidade γ). O sistema possui dois pontos de equilíbrio: a origem $(0, 0)$ — extinção de ambas as espécies — e o ponto não-trivial $(x^*, y^*) = (\gamma/\delta, \alpha/\beta)$.

A análise de estabilidade desse ponto de equilíbrio, via linearização do sistema em torno de (x^*, y^*) , revela que os autovalores do jacobiano são puramente imaginários — indicando oscilações conservativas, não amortecidas. As trajetórias formam órbitas fechadas no plano de fase: as populações oscilam indefinidamente, sem convergir para um equilíbrio fixo nem divergir. Dados experimentais de populações naturais, como os registros centenários de lince e lebre do Canadá (Elton e Nicholson, 1942), mostram oscilações qualitativamente consistentes com esse modelo.

4.3 Análise de Estabilidade e Bifurcações

A análise de estabilidade de um ponto de equilíbrio $\mathbf{x}^*$ — onde $\mathbf{f}(\mathbf{x}^*, \boldsymbol{\theta}) = \mathbf{0}$ — é feita por linearização. Seja $\boldsymbol{\xi} = \mathbf{x} - \mathbf{x}^*$ a perturbação em torno do equilíbrio. Para $\boldsymbol{\xi}$ pequeno, a dinâmica é aproximada pelo sistema linear

$$\frac{d\boldsymbol{\xi}}{dt} \approx J(\mathbf{x}^*) \boldsymbol{\xi}, \quad (4.8)$$

onde J é o **jacobiano** da função $\mathbf{f}$ avaliado no equilíbrio, $J_{ij} = \partial f_i / \partial x_j|_{\mathbf{x}^*}$. A estabilidade é determinada pelos autovalores λ_k de J : o equilíbrio é **estável** se todos os autovalores têm parte real negativa ($\text{Re}(\lambda_k) < 0$ para todo k), **instável** se ao menos um tem parte real positiva, e **marginalmente estável** no caso limite.

Uma **bifurcação** ocorre quando, ao variar um parâmetro de controle μ , a estrutura qualitativa do sistema muda — surgem ou desaparecem pontos de equilíbrio, ou sua estabilidade se altera. As bifurcações mais importantes em biologia são:

Sela-nó (saddle-node): dois equilíbrios — um estável e um instável — se aproximam e se aniquilam ao variar μ . Modelos genéticos com bistabilidade exibem frequentemente esse tipo de bifurcação, criando dois estados atratores (“liga” e “desliga”) separados por uma separatriz.

Hopf: um equilíbrio estável perde estabilidade e dá origem a um ciclo limite estável. É o mecanismo gerador de oscilações biológicas: o oscilador circadiano, o ritmo cardíaco e as oscilações do NF- κ B são exemplos.

Pitchfork: um equilíbrio se divide em três, dois estáveis flanqueando um instável. Ocorre em sistemas com simetria e é relevante para modelos de diferenciação celular.

4.4 Análise de Sensibilidade

A análise de sensibilidade quantifica como as saídas do modelo respondem a variações nos parâmetros — pergunta fundamental tanto para validação de modelos quanto para identificar os “pontos de controle” terapêuticos de um sistema.

O **coeficiente de sensibilidade local** de uma variável de estado x_i em relação a um parâmetro θ_j é definido como a derivada parcial normalizada:

$$S_{ij} = \frac{\partial \ln x_i}{\partial \ln \theta_j} = \frac{\theta_j}{x_i} \frac{\partial x_i}{\partial \theta_j}. \quad (4.9)$$

Um valor $S_{ij} = 2$ significa que um aumento de 1% em θ_j produz um aumento de 2% em x_i . Coeficientes próximos de zero indicam parâmetros aos quais o modelo é robusto; valores grandes indicam vulnerabilidades — ou, na perspectiva terapêutica, alvos de intervenção promissores.

Para incerteza paramétrica ampla, a **análise de sensibilidade global** (métodos de Sobol, FAST, ou amostragem Latin Hypercube) explora todo o espaço de parâmetros, decompondo a variância total da saída em contribuições de cada parâmetro e de suas interações. Esse approach é computacionalmente mais custoso, mas fornece uma imagem mais completa da robustez e fragilidade do modelo.

4.5 Exercícios

Exercício 4.1. Simule o mapa logístico $x_{t+1} = r x_t(1 - x_t)$ para $r \in \{2.5, 3.2, 3.5, 3.9\}$ e $x_0 = 0.3$. (a) Plote as 100 primeiras iterações para cada valor de r . (b) Construa o diagrama de bifurcação. (c) Para $r = 3.9$, compare trajetórias iniciadas em $x_0 = 0.3$ e $x_0 = 0.30001$ — quantas iterações são necessárias para que diverjam visivelmente?

Exercício 4.2. Para o sistema de Lotka-Volterra com $\alpha = 0.4$, $\beta = 0.02$, $\delta = 0.01$, $\gamma = 0.2$: (a) calcule o ponto de equilíbrio não-trivial; (b) calcule o jacobiano nesse ponto e determine os autovalores; (c) simule o sistema com SciPy e plote o retrato de fase para diferentes condições iniciais.

Exercício 4.3. Calcule os coeficientes de sensibilidade local de $V_{\max}$ e K_M na equação de Michaelis-Menten $v = V_{\max}[S]/(K_M + [S])$ em relação à velocidade v , para $[S] = K_M$. Interprete os resultados biologicamente.

4.6 Notas Bibliográficas

O artigo seminal de May (1976), *Simple mathematical models with very complicated dynamics*, publicado na *Nature*, é leitura obrigatória e acessível. A teoria das bifurcações é tratada com rigor em Strogatz (1994), *Nonlinear Dynamics and Chaos* — o texto introdutório mais recomendado para biólogos com formação matemática básica. Para aplicações biológicas específicas, Keener e Sneyd (2009), *Mathematical Physiology*, cobrem desde modelos enzimáticos até eletrofisiologia cardíaca. A análise de sensibilidade global é detalhada em Saltelli et al. (2008), *Global Sensitivity Analysis: The Primer*.

Capítulo 5

Estimação de Parâmetros

5.1 Introdução

5.1 Por que Estimação de Parâmetros?.

Objetivos de Aprendizagem

Compreender o problema inverso em biologia de sistemas, Dominar métodos de otimização local e global, Aplicar técnicas de estimação a modelos biológicos, Avaliar identificabilidade e incerteza de parâmetros e Validar modelos com critérios estatísticos.

Definição (Estimação de Parâmetros).

Processo de determinar valores numéricos de parâmetros de um modelo matemático a partir de dados experimentais, ajustando o modelo para reproduzir o comportamento observado do sistema.

O Problema Inverso.

Parâmetros conhecidos, Condições iniciais dadas, Simular

Previsões Determinístico.

Exemplo

Dados $k_1 = 0.5$, $k_2 = 0.1$

Simular: $dt = k_1 - k_2x$

obter

$x(t)$

Dados experimentais, Parâmetros desconhecidos, Estimar

Ajuste Mal – posto (ill – posed).

Exemplo

Dados $x(t)$ experimentais

Modelo: $dt = k_1 - k_2x$

encontrar

k_1, k_2

— IMP:

O problema inverso é geralmente mais difícil que o direto!

Desafios na Estimação de Parâmetros.

Principais Obstáculos

múltiplas soluções, medições imprecisas, poucas observações, otimização não-convexa, parâmetros colineares, simulações e quantificação de confiança.

Visão Geral dos Métodos.

Escolha do Método

locais (rápido), globais (robusto) e busca em grade (simples).

5.2 Formulação do Problema

Função Objetivo: Conceito.

Definição (Função Objetivo (Cost Function)).

Medida quantitativa da discrepância entre dados experimentais e previsões do modelo. O objetivo na estimação é encontrar parâmetros que minimizam esta função.

Componentes Principais

y_i : dados experimentais observados no ponto i , $i()$: previsão ou simulação gerada pelo modelo,

$\mathbf{p}$: vetor de parâmetros a serem ajustados/estimados

n : número total de observações experimentais.

Função Objetivo: Formulação.

A função objetivo quantifica o erro total entre os dados e as previsões:

— DEST:

$J(\boldsymbol{\theta})$ = medida de discrepância entre $\{y_i\}$ e $\{\hat{y}_i(\boldsymbol{\theta})\}$

A estimação é formulada matematicamente como a busca pelo argumento minimizador de $J()$:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Mínimos Quadrados Ordinários (OLS).

Método Mais Comum

Minimização da soma dos quadrados dos desvios entre o modelo e os dados:

— DEST:

$$J(\boldsymbol{\theta}) = \text{SSE}(\boldsymbol{\theta}) = \sum_{i=1}^n [y_i - \hat{y}_i(\boldsymbol{\theta})]^2$$

Resíduo

O resíduo r_i representa o erro de previsão individual no ponto i :

$$r_i = y_i - \hat{y}_i(\boldsymbol{\theta}) \quad \Rightarrow \quad \text{SSE}(\boldsymbol{\theta}) = \sum_{i=1}^n r_i^2$$

Mínimos Quadrados Ordinários (OLS): Propriedades.

A formulação OLS clássica possui características importantes:

Simples e intuitivo, Bem estabelecido na literatura, Solução analítica para modelos lineares e Interpretação estatística direta.

Erros gaussianos (ruído branco), Variância constante (homocedasticidade), Erros independentes e Outliers são muito problemáticos.

Mínimos Quadrados Ponderados (WLS).

Quando usar?

Quando as observações têm diferentes níveis de incerteza ou confiabilidade.

— DEST:

$$J(\boldsymbol{\theta}) = \sum_{i=1}^n w_i [y_i - \hat{y}_i(\boldsymbol{\theta})]^2$$

$w_i = 1/\sigma_i^2$: inverso da variância (comum), $w_i = 1/y_i$: proporcional ao valor (dados log-scale) e w_i customizado: baseado no erro experimental.

Concentrações baixas têm maior erro relativo.

usarpesos

$w_i = 1/i$ ou $w_i = 1/i^2$.

Maximum Likelihood Estimation (MLE): Conceito.

Abordagem Estatística

Encontrar parâmetros que maximizam a probabilidade (likelihood) de observar os dados reais obtidos.

$$\mathcal{L}(\boldsymbol{\theta}) = P(\text{dados}|\boldsymbol{\theta})$$

$$\ln \mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^n \ln P(y_i|\boldsymbol{\theta})$$

Maximum Likelihood Estimation (MLE): Caso Gaussiano.

Caso Gaussiano

Assumindo que os erros de medição ϵ_i são independentes e normalmente distribuídos, $\epsilon_i(0, \sigma^2)$:

$$\ln \mathcal{L}(\boldsymbol{\theta}) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n [y_i - \hat{y}_i(\boldsymbol{\theta})]^2$$

— IMP:

Sob a hipótese de ruído gaussiano, maximizar a log-verossimilhança () é matematicamente equivalente a minimizar a soma dos quadrados dos resíduos (SSE) do método de OLS!

Abordagem Bayesiana: Teorema e Termos.

Incorpora Conhecimento Prévio

Usa o Teorema de Bayes para atualizar a nossa crença sobre as distribuições dos parâmetros à medida que novos dados são observados.

— DEST:

$$P(\boldsymbol{\theta}|\text{dados}) = \frac{P(\text{dados}|\boldsymbol{\theta}) \cdot P(\boldsymbol{\theta})}{P(\text{dados})}$$

$P(|)$: (distribuição pós-dados), $P(|)$: (verossimilhança dos dados), $P()$: (conhecimento prévio sobre os parâmetros) e $P()$: evidência (constante de normalização).

Abordagem Bayesiana: Vantagens e Desafios.

Vantagens

Permite incorporar dados históricos ou da literatura nas distribuições priors, Quantifica incerteza de forma natural (probabilidade dos parâmetros) e Previne overfitting agindo como uma regularização natural.

Desafio

Requer a escolha de distribuições priors adequadas e demanda alto custo compu-

tacional com algoritmos de amostragem (como MCMC).

5.3 Busca em Grade (Grid Search)

Conceito de Busca em Grade.

Definição (Grid Search).

Método de otimização que avalia sistematicamente a função objetivo em uma grade regular de valores de parâmetros.

Definir limites para os parâmetros, Criar grade regular com $n_1 n_2$ pontos, Avaliar a função objetivo $J()$ em cada nó da grade e Selecionar o ponto com o menor valor de J .

Vantagens e Desvantagens.

Vantagens

Simples de implementar, Facilmente paralelizável, Garantia de encontrar o mínimo global e Sem derivadas necessárias.

Desvantagens

Custo exponencial: $O(n^p)$, Inviável para muitos parâmetros, Resolução da grade limitada e Maldição da dimensionalidade.

Grid Search: Custo Computacional.

O custo computacional cresce exponencialmente com o número de parâmetros estimáveis.

— IMP:

assumindo 10 valores testados por parâmetro:

2 parâmetros: $10^2 = 100$ avaliações de J , 5 parâmetros: $10^5 = 100.000$ avaliações de J e 10 parâmetros: 10^{10} avaliações (impraticável em tempo viável!).

A busca em grade é útil para exploração inicial de poucos parâmetros (1 ou 2), mas inadequada para problemas de alta dimensão.

Implementação: Grid Search 1D.

```

1 import numpy as np
2
3 def objective_function(theta):
4     return (theta - 3)**2 + 5
5
6 # Definir grade e avaliar
7 theta_values = np.linspace(0, 6, 50)
8 J_values = [objective_function(t) for t in theta_values]
9
10 # Encontrar mínimo
11 idx_min = np.argmin(J_values)
12 theta_best = theta_values[idx_min]
13 J_best = J_values[idx_min]
14
```

```

15 print(f"Parametro_otimo:_{theta_best:.3f}")
16 print(f"Valor_da_funcao:_{J_best:.3f}")

```

Listing 5.1: Busca em grade unidimensional

Implementação: Grid Search 2D.

```

1 def sse_model(params, t, y):
2     a, b = params
3     return np.sum((y - a * np.exp(-b * t))**2)
4
5 # Dados experimentais e grade
6 t = np.array([0, 1, 2, 3, 4, 5])
7 y = np.array([10, 6.1, 3.7, 2.2, 1.4, 0.8])
8 a_range = np.linspace(5, 15, 20)
9 b_range = np.linspace(0.1, 1.0, 20)
10 SSE_grid = np.array([[sse_model([a, b], t, y) for b in b_range] for a in
11                        a_range])
12
13 # Encontrar minimo
14 idx = np.unravel_index(SSE_grid.argmin(), SSE_grid.shape)
15 a_best, b_best = a_range[idx[0]], b_range[idx[1]]
16 print(f"Parametros_otimos:_a={a_best:.3f},_b={b_best:.3f}")

```

Listing 5.2: Busca em grade bidimensional

Visualização: Contour Plot.

```

1 # Criar contour plot
2 A, B = np.meshgrid(a_range, b_range)
3
4 plt.figure(figsize=(8, 5))
5 contour = plt.contourf(A, B, SSE_grid.T, levels=20, cmap='viridis')
6 plt.colorbar(contour, label='SSE')
7
8 # Marcar minimo e contornos
9 plt.plot(a_best, b_best, 'r*', markersize=15,
10          label=f'Minimo:_{a_best:.2f},_{b_best:.2f}')
11 plt.contour(A, B, SSE_grid.T, levels=10,
12             colors='white', linewidths=0.5, alpha=0.5)
13
14 plt.xlabel('Parametro_a')
15 plt.ylabel('Parametro_b')
16 plt.title('Landscape_da_Funcao_Objeto')
17 plt.legend()
18 plt.grid(True, alpha=0.3)

```

Listing 5.3: Mapa de contorno da funcao objetivo

5.4 Métodos de Gradiente

Gradient Descent (Descida do Gradiente).

Definição (Gradient Descent).

Método iterativo que se move na direção oposta ao gradiente para encontrar um mínimo local da função objetivo.

— DEST:

$$\theta^{(k+1)} = \theta^{(k)} - \alpha \nabla J(\theta^{(k)})$$

$\theta^{(k)}$: parâmetros na iteração k , $\alpha > 0$: taxa de aprendizado (passo) e $J = [J_1, J_2, \dots]^T$: vetor gradiente.

Escolha da Taxa de Aprendizado: Impacto.

Escolha da Taxa de Aprendizado: Estratégias.

— IMP:

A escolha de

é crucial para a convergência!

Estratégias

constante (simples, pode não convergir),

$\alpha_k = \alpha_0 / (1 + k)$ (garante convergência), otimizar

em cada iteração (mais robusto) e ajustar baseado no progresso (Adam, RMSprop).

Critérios de Convergência.

Quando Parar?

Teste múltiplos critérios simultaneamente:

$$\|J^{(k)}\| < \epsilon_g.$$

Indica ponto estacionário.

$$\|\theta^{(k+1)} - \theta^{(k)}\| < \epsilon.$$

Parâmetros não mudam mais.

$$|J^{(k+1)} - J^{(k)}| < \epsilon_J.$$

Função objetivo estabilizou.

$$k > k_{\max}.$$

Prevenir loops infinitos.

Valores Típicos

$$\epsilon_g = 10^{-6}, \epsilon_{\theta} = 10^{-6}, \epsilon_J = 10^{-8}, k_{\max} = 1000$$

Implementação: Gradient Descent.

```
1 def gradient_descent(f, theta0, alpha=0.01, max_iter=1000, tol=1e-6):
2     theta = np.array(theta0, dtype=float)
3     history = [theta.copy()]
4
5     for k in range(max_iter):
```

```

6      # Calcular gradiente numericamente
7      grad = numerical_gradient(f, theta)
8
9      # Atualizar parametros e checar convergencia
10     theta_new = theta - alpha * grad
11     if np.linalg.norm(theta_new - theta) < tol:
12         print(f"Convergiu em {k} iteracoes")
13         break
14
15     theta = theta_new
16     history.append(theta.copy())
17
18     return theta, np.array(history)

```

Listing 5.4: Descida do gradiente principal

Implementação: Gradiente Numérico.

```

1 def numerical_gradient(f, theta, h=1e-5):
2     """Gradiente por diferencas finitas"""
3     grad = np.zeros_like(theta)
4     for i in range(len(theta)):
5         theta_plus = theta.copy()
6         theta_plus[i] += h
7         grad[i] = (f(theta_plus) - f(theta)) / h
8     return grad

```

Listing 5.5: Calculo numerico do gradiente

Método de Newton-Raphson.

Ideia

Usar informação de segunda ordem (curvatura) para convergência mais rápida
— DEST:

$$\theta^{(k+1)} = \theta^{(k)} - \mathbf{H}^{-1}(\theta^{(k)}) \nabla J(\theta^{(k)})$$

: matriz Hessiana (derivadas segundas) e

$$H_{ij} = \theta_i \theta_j.$$

Convergência quadrática, Rápido perto do mínimo e Não precisa de step size.

Calcular

é caro, Inverter

também e Não-convergência longe do mínimo.

Método de Gauss-Newton.

Específico para Mínimos Quadrados

Aproxima a Hessiana usando apenas o Jacobiano das previsões do modelo.

Para $J() = r_i^2()$ com resíduos $r_i = y_i - \hat{y}_i()$:

— DEST:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + (\mathbf{J}^T \mathbf{J})^{-1} \mathbf{J}^T \mathbf{r}$$

: Jacobiana(

$\mathbf{J}_{ij} = r_i / \theta_j$) e

: vetores de resíduos individuais.

Vantagens

Evita o cálculo da Hessiana completa, Excelente para mínimos quadrados e Base do algoritmo Levenberg-Marquardt.

Levenberg-Marquardt (LM): Algoritmo.

— DEF: Levenberg-Marquardt —

Combina Gauss-Newton e gradient descent, interpolando entre ambos via parâmetro de amortecimento.

— DEST:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + (\mathbf{J}^T \mathbf{J} + \lambda \mathbf{I})^{-1} \mathbf{J}^T \mathbf{r}$$

: parâmetro de amortecimento (damping),

grande

gradientdescent(pequenos passos, robusto) e

pequeno

Gauss – Newton(passos grandes, rápido).

Levenberg-Marquardt (LM): Estratégia.

Se J diminui: aceitar passo, diminuir

(mais agressivo) e Se

Jaumenta : rejeitar passo, aumentar (mais conservador).

— IMP:

LM é o método padrão para ajuste não-linear de curvas!

Otimização Local: Michaelis-Menten (curve_fit).

```
1 from scipy.optimize import curve_fit
2 import numpy as np
3
4 # Modelo de Michaelis-Menten
```

```

5 def michaelis_menten(S, Vmax, Km):
6     return Vmax * S / (Km + S)
7
8 # Dados experimentais
9 S_data = np.array([0.5, 1, 2, 5, 10, 20, 50])
10 v_data = np.array([0.4, 0.7, 1.2, 1.8, 2.1, 2.3, 2.4])
11
12 # Otimizacao via curve_fit
13 params, cov = curve_fit(michaelis_menten, S_data, v_data,
14                          p0=[3, 5])
15 Vmax_fit, Km_fit = params
16
17 print(f"Vmax={Vmax_fit:.3f}")
18 print(f"Km={Km_fit:.3f}")

```

Listing 5.6: Ajuste simples usando curve_fit

Otimização Local: Michaelis-Menten (least_squares).

```

1 from scipy.optimize import least_squares
2 import numpy as np
3
4 # Reuso dos dados anteriores e definicao de residuos
5 def residuals(params):
6     Vmax, Km = params
7     return v_data - michaelis_menten(S_data, Vmax, Km)
8
9 result = least_squares(residuals, x0=[3, 5], method='lm')
10 print(f"Parametros otimos: {result.x}")
11 print(f"Custo final: {result.cost:.6f}")
12 print(f"Sucesso: {result.success}")

```

Listing 5.7: Ajuste via least_squares (LM)

Visualização do Ajuste.

```

1 # Gerar curva do modelo ajustado
2 S_smooth = np.linspace(0, 60, 200)
3 v_fit = michaelis_menten(S_smooth, Vmax_fit, Km_fit)
4
5 # Plotar dados e ajuste
6 plt.figure(figsize=(10, 6))
7 plt.plot(S_data, v_data, 'ko', markersize=10, label='Dados_exp')
8 plt.plot(S_smooth, v_fit, 'b-', linewidth=2,
9          label=f'Ajuste: Vmax={Vmax_fit:.2f}, Km={Km_fit:.2f}')
10
11 # Linhas de referencia
12 plt.axhline(y=Vmax_fit, color='r', linestyle='--', alpha=0.5, label='Vmax')
13 plt.axvline(x=Km_fit, color='g', linestyle='--', alpha=0.5, label='Km')
14 plt.axhline(y=Vmax_fit/2, color='gray', linestyle=':')
15
16 plt.xlabel('[Substrato] (mM)'); plt.ylabel('Velocidade (umol/min)')
17 plt.title('Ajuste de Michaelis-Menten')
18 plt.legend(); plt.grid(True, alpha=0.3)

```


Algoritmo evolutivo que cria novos candidatos através de diferenças entre membros da população.

Escolher 3 indivíduos:

,
,
,
, *Mutação* :
= + F(-), *CruzamentousandotaxaCReSeleção* : *manteromelhor*.
Fator de escala $F[0.5, 1.0]$ e Crossover $CR[0.5, 0.9]$.

Vantagens

Simples de implementar, Poucos parâmetros de ajuste, Excelente robustez, Não requer gradientes e Ótimo para múltiplos mínimos.

Particle Swarm Optimization (PSO).

Inspiração

Comportamento social de bandos de pássaros ou cardumes de peixes.

Cada partícula tem posição i e velocidade i e Movimento influenciado por: melhor posição própria (i) e melhor posição global (

).

— DEST:

$$\begin{aligned} \mathbf{v}_i^{(k+1)} &= w\mathbf{v}_i^{(k)} + c_1r_1(\mathbf{p}_i - \mathbf{x}_i^{(k)}) + c_2r_2(\mathbf{g} - \mathbf{x}_i^{(k)}) \\ \mathbf{x}_i^{(k+1)} &= \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)} \end{aligned}$$

w : peso de inércia | c_1, c_2 : coeficientes cognitivo e social e r_1, r_2 : números aleatórios uniformes em $[0, 1]$.

Implementação: Differential Evolution (Função).

```
1 from scipy.optimize import differential_evolution
2 import numpy as np
3
4 # Funcao objetivo
5 def objective(params, t_data, y_data):
6     """Modelo exponencial:  $y = a * \exp(-b*t)$ """
7     a, b = params
8     y_pred = a * np.exp(-b * t_data)
9     sse = np.sum((y_data - y_pred)**2)
10    return sse
11
```

```

12 # Dados experimentais
13 t_data = np.array([0, 1, 2, 3, 4, 5])
14 y_data = np.array([10, 6.1, 3.7, 2.2, 1.4, 0.8])

```

Listing 5.9: Modelo e funcao objetivo para DE

Implementação: Differential Evolution (Otimização).

```

1 # Definir bounds para parametros: [(a_min, a_max), (b_min, b_max)]
2 bounds = [(1, 20), (0.01, 2.0)]
3
4 # Otimizar com DE
5 result = differential_evolution(
6     objective, bounds, args=(t_data, y_data),
7     strategy='best1bin', maxiter=1000, popsize=15,
8     tol=1e-7, mutation=(0.5, 1), recombination=0.7, seed=42
9 )
10
11 print(f"Parametros otimizados: a={result.x[0]:.3f}, b={result.x[1]:.3f}")
12 print(f"SSE minimo: {result.fun:.6f}")
13 print(f"Convergiu: {result.success}")

```

Listing 5.10: Otimizacao global usando scipy

Comparação de Métodos de Otimização.

Diretrizes para Estimação de Parâmetros.

Recomendações Práticas

Use Levenberg-Marquardt (rápido e local), Use Differential Evolution (robusto e global), Abordagem híbrida (rodar DE primeiro e usar resultado como inicial no LM) e Considere métodos Bayesianos (MCMC).

5.6 Análise de Identificabilidade

Identificabilidade Estrutural: Exemplos.

Definição (Identificabilidade Estrutural).

Propriedade teórica do modelo: os parâmetros podem ser determinados unicamente a partir de dados perfeitos (infinitos e sem ruído)?

Identificável

$$\frac{dx}{dt} = k_1 - k_2x$$

Ambos k_1 e k_2 podem ser determinados unicamente.

Não Identificável

$$\frac{dx}{dt} = (k_1k_2) - k_2x$$

Apenas o produto $k_1 k_2$ é identificável, não k_1 e k_2 separadamente.

Identificabilidade Estrutural: Diretrizes.

— IMP:

Antes de estimar parâmetros, verifique se o modelo é estruturalmente identificável! eliminação de variáveis via álgebra computacional e DAISY, GenSSI, STRIKE-GOLDD.

Identificabilidade Prática.

Definição (Identificabilidade Prática).

Podem os parâmetros ser determinados com precisão razoável a partir de dados experimentais reais (finitos e com ruído)?

Dados insuficientes/baixa qualidade, Parâmetros altamente correlacionados, Modelo super-parametrizado (overfitting) e Baixa sensibilidade a parâmetros.

Modelo: $y = ae^{-bt}$

Se t é pequeno: $e^{-bt} \approx 1 - bt$

$$ya(1 - bt) = a - abt$$

Difícil separar a de ab (produto correlacionado).

Matriz de Informação de Fisher (FIM).

Quantifica Informação sobre Parâmetros

Mede quanta informação os dados contêm sobre cada parâmetro.

— DEST:

$$\mathbf{F}_{ij} = \sum_{k=1}^n \frac{1}{\sigma_k^2} \frac{\partial \hat{y}_k}{\partial \theta_i} \frac{\partial \hat{y}_k}{\partial \theta_j}$$

k : previsão do modelo no ponto k , σ_k^2 : variância do erro no ponto k e θ_i, θ_j : parâmetros.

FIM grande

alta precisão, FIM pequena

alta incerteza e $\rightarrow 0$

não - identificável.

Intervalos de Confiança: Teoria.

Estimativa Assintótica

Para grandes amostras, $(*,)$ onde:

— DEST:

$$\mathbf{C} = \text{Cov}(\boldsymbol{\theta}) \approx \mathbf{F}^{-1}$$

= *erro padrão do parâmetro* i

Off-diagonal: $C_{i,j}$ = correlação entre parâmetros i e j .

— IMP:

Intervalos de confiança estreitos

modelo correto!

Apenas indicam precisão do ajuste.

Profile Likelihood.

Definição (Profile Likelihood).

Método robusto para calcular intervalos de confiança explorando o espaço de parâmetros sistematicamente.

Fixar θ_i em valor específico, Otimizar os outros parâmetros, Registrar $J(\theta_i)$, Repetir para vários valores e Plotar J vs θ_i .

Implementação: Visualização de Correlação.

```

1 from scipy.optimize import curve_fit
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # Ajustar e extrair correlacao
6 params, cov = curve_fit(model_func, x_data, y_data)
7 std = np.sqrt(np.diag(cov))
8 corr = cov / np.outer(std, std)
9
10 # Plotar heatmap
11 param_names = ['k1', 'k2', 'k3']
12 plt.figure(figsize=(8, 6))
13 sns.heatmap(corr, annot=True, fmt='.2f',
14             xticklabels=param_names, yticklabels=param_names,
15             cmap='coolwarm', center=0, vmin=-1, vmax=1)
16 plt.title('Matriz de Correlacao')
```

Listing 5.11: Heatmap de correlacao usando seaborn

Implementação: Alertas de Correlação.

```

1 # Alerta de alta correlacao
2 for i in range(len(params)):
3     for j in range(i+1, len(params)):
4         if abs(corr[i,j]) > 0.9:
5             print(f"ALERTA: {param_names[i]} e {param_names[j]}
6                 \n
7                     altamente correlacionados (r={corr[i,j]:.2f})")
```

Listing 5.12: Detecção de parâmetros colineares

Implementação: Cálculo de Intervalos de Confiança.

```

1 from scipy import stats
2 import numpy as np
3
4 params_best = params
5 errors = np.sqrt(np.diag(cov))
6
7 # Intervalo de confiança 95%
8 alpha = 0.05
9 t_val = stats.t.ppf(1-alpha/2, len(x_data)-len(params))
10 ci = t_val * errors

```

Listing 5.13: Calcular ICs usando distribuição t

Implementação: Plot de Parâmetros e CIs.

```

1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(10, 6))
4 x_pos = np.arange(len(param_names))
5
6 ax.bar(x_pos, params_best, yerr=ci, alpha=0.7, capsize=10,
7       color='steelblue', error_kw={'linewidth': 2, 'capthick': 2})
8 ax.set_xticks(x_pos)
9 ax.set_xticklabels(param_names)
10 ax.set_ylabel('Valor do Parâmetro')
11 ax.set_title('Parâmetros Estimados com ICs 95%')
12 ax.grid(axis='y', alpha=0.3)
13
14 for name, val, err in zip(param_names, params_best, ci):
15     print(f"{name} = {val:.4f} +/- {err:.4f}")
16     print(f"({100*err/val:.1f}% erro relativo)")

```

Listing 5.14: Visualizar parâmetros com barras de erro

5.7 Validação e Critérios de Ajuste

Análise de Resíduos: Padrões.

Verificação de Hipóteses

Examinar resíduos $r_i = y_i - \hat{y}_i$ para validar suposições do modelo.

Bom Ajuste

Distribuição aleatória, Sem padrão sistemático, Centrados em zero, Variância constante e Aproximadamente normais.

Problemas

Padrão curvo: inadequação, Funil: heterocedasticidade, Outliers: dados anômalos e Não-normalidade do erro.

Análise de Resíduos: Visualização.

Coefficiente de Determinação (R^2): Definição.

Definição (R^2).

Proporção da variância nos dados explicada pelo modelo. Varia de 0 (nenhum ajuste) a 1 (ajuste perfeito).

— DEST:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

: soma dos quadrados dos resíduos, : soma dos quadrados totais e

: *médiados dados experimentais.*

Coefficiente de Determinação (R^2): Limitações.

Limitações Importantes

R^2 sempre aumenta ao adicionar novos parâmetros (mesmo irrelevantes), Um R^2 alto não garante que o modelo é correto ou preditivo, Pode ser altamente enganoso para modelos não-lineares e Use o R^2 ajustado para comparar modelos com diferentes números de parâmetros.

R^2 Ajustado: Definição.

Penaliza Complexidade do Modelo

Ajusta o R^2 baseando-se no número de graus de liberdade.

— DEST:

$$R_{\text{adj}}^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

onde:

n : número de observações (tamanho da amostra) e p : número de parâmetros estimáveis.

Pode diminuir ao adicionar parâmetros ruins ou irrelevantes e Mais apropriado para comparação de modelos.

Akaike Information Criterion (AIC): Teoria.

Definição (AIC).

Critério que balanceia a qualidade do ajuste com a complexidade do modelo.

— DEST:

$$AIC = 2p - 2 \ln(\mathcal{L}) \approx n \ln \left(\frac{SSE}{n} \right) + 2p$$

p : número de parâmetros,

: *máxima verossimilhança* e SSE : *soma dos resíduos quadrados*.

Menor AIC = melhor modelo, Penaliza complexidade extra e > 10 : evidência forte.

AIC Corrigido (AICc)

Para amostras pequenas ($n/p < 40$):

$$AICc = AIC + \frac{2p(p+1)}{n-p-1}$$

Bayesian Information Criterion (BIC): Definição.

Definição (BIC (ou Schwarz Criterion)).

Similar ao AIC, mas penaliza mais fortemente a complexidade do modelo.

— DEST:

$$BIC = p \ln(n) - 2 \ln(\mathcal{L}) \approx n \ln \left(\frac{SSE}{n} \right) + p \ln(n)$$

BIC penaliza complexidade mais forte (termo $p(n)$ vs $2p$ do AIC), Para $n > 7$, o BIC seleciona modelos mais parcimoniosos (menores) e O AIC busca otimizar a previsão futura, enquanto o BIC tenta identificar o "modelo verdadeiro".

Comparação de Critérios de Seleção (AIC vs BIC).

Resumo Prático

Na biologia de sistemas, onde os dados costumam ser limitados e o ruído é alto, o ****AICc**** e o ****BIC**** são usados em conjunto. Se divergirem, prioriza-se o modelo mais simples (BIC) para evitar overfitting.

Cross-Validation (Validação Cruzada).

Teste de Capacidade Preditiva

Avaliar o desempenho do modelo em dados não vistos durante o treinamento para evitar overfitting.

Dividir dados em K folds e Para cada fold k :

Treinar modelo nos outros $K - 1$ folds, Testar no fold k e Registrar erro de previsão.

Calcular erro médio de CV.

0

Diretrizes

Típico: $K = 5$ ou $K = 10$. Se $K = n$, chamamos de Leave-One-Out (LOO), muito custoso.

Overfitting vs Underfitting.

1

Sintomas de Overfitting

Erro de treinamento muito menor que o erro de teste, Modelo complexo com excesso de parâmetros livres e ICs excessivamente largos; previsões ruins fora do domínio.

Implementação: Cálculo de Métricas (RMSE, R2).

```

1 import numpy as np
2
3 def calculate_metrics(y_true, y_pred, n_params):
4     n = len(y_true)
5     residuals = y_true - y_pred
6
7     # SSE e MSE
8     sse = np.sum(residuals**2)
9     mse = sse / n
10    rmse = np.sqrt(mse)
11
12    # R-squared e R-squared ajustado
13    ss_tot = np.sum((y_true - np.mean(y_true))**2)
14    r2 = 1 - (sse / ss_tot)
15    r2_adj = 1 - (1-r2)*(n-1)/(n-n_params-1)

```

Listing 5.15: Cálculo de RMSE e R-squared ajustado

Implementação: Cálculo de Métricas (AIC, BIC).

```

1 # Calculo continuacao: AIC e BIC
2 aic = n*np.log(sse/n) + 2*n_params
3 bic = n*np.log(sse/n) + n_params*np.log(n)
4
5 return {
6     'SSE': sse, 'MSE': mse, 'RMSE': rmse,
7     'R2': r2, 'R2_adj': r2_adj, 'AIC': aic, 'BIC': bic,
8     'residuals': residuals
9 }
10
11 # Executar e exibir resultados
12 metrics = calculate_metrics(y_data, y_pred, n_params=2)
13 for key, val in metrics.items():
14     if key != 'residuals':
15         print(f"{key}: {val:.4f}")

```

Listing 5.16: Cálculo de AIC/BIC e execucao

5.8 Exercícios

Exercícios - Parte 1: Busca em Grade.

Questão 1: Busca em Grade

Implemente busca em grade para encontrar parâmetros a e b do modelo $y = ax^b$ usando os dados:

Use: $a[1, 3]$, $b[1.5, 2.5]$, grade 20x20.

Exercícios - Parte 1: Descida do Gradiente.

Questão 2: Gradient Descent

Implemente gradient descent manualmente para minimizar $J() = (-5)^2 + 3$:

Use $\theta_0 = 0$ e $\epsilon = 0.1$, Plote trajetória de convergência, Teste diferentes valores de

$(0.01, 0.1, 0.5, 1.0)\epsilon$ Compare número de iterações até convergência.

Exercícios - Parte 2: Levenberg-Marquardt.

— Questão 3: Levenberg-Marquardt —

Ajuste parâmetros da cinética de Michaelis-Menten $v = [S]/K_M + [S]$ aos dados:

Use `scipy.optimize.curve_fit`, Calcule intervalos de confiança e Plote dados vs modelo ajustado e faça análise de resíduos.

Exercícios - Parte 2: Differential Evolution.

Questão 4: Differential Evolution

Compare DE com LM no problema anterior:

Use bounds amplos: $V[0.1, 10]$, $K_M[0.1, 10]$, Qual método é mais rápido? Qual encontra melhor solução? e Use DE para encontrar boa estimativa inicial para o LM.

Exercícios - Parte 3: Modelo Logístico.

Questão 5: Crescimento Logístico

Considere dados de crescimento bacteriano:

Gere dados sintéticos: $N(t) = 1 + (N_0 - 1)e^{-rt}$ com $N_0 = 1, r = 0.5, K = 100, t[0, 20]$ + ruído, Estime r e K usando três métodos diferentes e Analise identificabilidade e profile likelihood.

Exercícios - Parte 3: Validação Cruzada.

Questão 6: Validação Cruzada

Implemente 5-fold cross-validation para o modelo logístico:

Divida dados em 5 folds, Calcule RMSE médio de CV, Compare com RMSE de treinamento (checar overfitting) e Teste com modelos de complexidade crescente.

Exercícios - Parte 4: Descrição do Projeto.

— Projeto: Sistema de Lotka-Volterra —

Estime parâmetros do sistema predador-presa:

$$\begin{aligned}\frac{dV}{dt} &= \alpha V - \beta V P \\ \frac{dP}{dt} &= \delta \beta V P - \gamma P\end{aligned}$$

Gere séries temporais com parâmetros conhecidos + ruído gaussiano.

Exercícios - Parte 4: Tarefas do Projeto.

Tarefas Recomendadas

Implementar função de custo (SSE de ambas variáveis), Estimar 4 parâmetros usando Differential Evolution, Refinar estimativas com Levenberg-Marquardt, Analisar identificabilidade (matriz de correlação de Fisher), Calcular intervalos de confiança assintóticos e plotá-los e Validar o ajuste simulando o modelo vs dados ruidosos.

5.9 Notas Bibliográficas

Referências Principais.

99

Nocedal J., Wright S.J. (2006). Numerical Optimization, 2nd ed. Springer.

Raue A. et al. (2009). Structural and practical identifiability analysis of partially observed dynamical models. Bioinformatics.

Voit E.O. et al. (2015). 150 years of the mass action law. PLoS Computational Biology.

Moles C.G. et al. (2003). Parameter estimation in biochemical pathways. Genome Research.

Burnham K.P., Anderson D.R. (2002). Model Selection and Multimodel Inference, 2nd ed. Springer.

Leitura Complementar.

Livros

Press W.H. et al. (2007). Numerical Recipes. Cambridge, Bard Y. (1974). Nonlinear Parameter Estimation e Seber G.A.F., Wild C.J. (2003). Nonlinear Regression. Wiley.

Artigos Importantes

Gutenkunst R.N. (2007). Universally sloppy parameter sensitivities e Villaverde A.F. (2016). Cooperative parameter estimation.

Software e Tutoriais

-
-
-

Obrigado!

Capítulo 5: Estimação de Parâmetros

Sistemas Gênicos

Dúvidas? Perguntas?

Parte III

Sistemas Biológicos

Capítulo 6

Sistemas Gênicos

6.1 Dogma Central da Biologia Molecular

6.1 Dogma Central da Biologia Molecular.

— IMP:

O fluxo de informação genética segue a direção: DNA

RNA

Proteína

Exceção: Retrovírus

Transcriptase reversa: RNA

DNA

Visão Geral dos Sistemas Gênicos.

Objetivos de Aprendizagem

Compreender a estrutura e organização dos genes, Entender os mecanismos de regulação gênica, Conhecer os diferentes tipos de RNA e suas funções, Aprender técnicas de medição de expressão gênica e Explorar bancos de dados genômicos (GenBank).

Definição (Gene).

Unidade fundamental da hereditariedade: sequência de DNA que codifica uma molécula funcional (RNA ou proteína) e inclui regiões regulatórias.

6.2 Características do DNA

6.2 Características do DNA (DNA Features).

Promotor, Região codificante (ORF), Íntrons e éxons, UTRs (5' e 3'), Terminador e Enhancers/Silencers.

Procariotos vs Eucariotos

genes sem íntrons, operons e íntrons, splicing alternativo.

Promotores e Elementos Regulatórios.

Promotor

Região onde a RNA polimerase se liga para iniciar a transcrição (Pribnow box):

·
TATAAT

:

·
TTGACA

controle negativo.

(-25):

·
TATAAA

(-75).

-

(distantes).

— DEST:

5'—[Enhancer]—[CAAT]—[TATA]—[+1]—Gene—3'

Open Reading Frame (ORF).

Definição (ORF).

Sequência contínua de códons entre um códon de início (ATG) e um códon de parada (TAA, TAG, TGA), sem códons de parada internos.

Exemplo

5'-ATG GCT AAA TGG CCA TAA-3'

Met-Ala-Lys-Trp-Pro-STOP

15 nucleotídeos = 5 códons

Identificação de ORFs

Critérios:

Começa com ATG (códon de início), Termina com TAA, TAG ou TGA (códon de parada), Tamanho mínimo (geralmente 300 nt / 100 aa) e Mesma fase de leitura (frame).

Busca de ORFs Computacional.

```
1 def find_orfs(sequence, min_length=100):
```



```

2      """
3      Encontra ORFs em uma sequencia de DNA
4      """
5      start_codon = "ATG"
6      stop_codons = ["TAA", "TAG", "TGA"]
7      orfs = []
8
9      # Para cada frame (0, 1, 2)
10     for frame in range(3):
11         for i in range(frame, len(sequence)-2, 3):
12             codon = sequence[i:i+3]
13
14             # Encontrou inicio
15             if codon == start_codon:
16                 # Procurar stop codon
17                 for j in range(i+3, len(sequence)-2, 3):
18                     stop = sequence[j:j+3]
19                     if stop in stop_codons:
20                         orf_len = j - i + 3
21                         if orf_len >= min_length*3:
22                             orfs.append({
23                                 'start': i,
24                                 'end': j+3,
25                                 'length': orf_len,
26                                 'frame': frame
27                             })
28                     break
29     return orfs

```

Listing 6.1: Identificação de ORFs

6.3 Regulação Gênica

6.3 Regulação Gênica.

Níveis de Regulação

controle da síntese de RNA, splicing, estabilidade do mRNA, controle da síntese proteica, modificações, degradação e metilação, acetilação de histonas.

— IMP:

A regulação da expressão gênica permite:

Resposta a sinais ambientais, Diferenciação celular, Desenvolvimento embrionário e Homeostase.

Regulação Transcricional em Procariotos.

Sistema clássico de regulação negativa e positiva

repressor LacI bloqueia transcrição e lactose liga a LacI, liberando operador.

Regulação Transcricional em Eucariotos.

Ativadores, Repressores, Mediadores e Remodeladores de cromatina.

Eucromatina (aberta), Heterocromatina (fechada) e Nucleossomos.

Modificações Epigenéticas

Acetilação: ativa transcrição e Metilação: silencia genes.

Modelo Matemático de Regulação Gênica.

Equação de Hill

Modelo para regulação cooperativa:

$$\frac{d[mRNA]}{dt} = \frac{V_{max}[TF]^n}{K_d^n + [TF]^n} - \delta[mRNA]$$

onde:

$[TF]$ = concentração do fator de transcrição, n = coeficiente de Hill (cooperatividade), K_d = constante de dissociação e

$$= \text{taxa de degradação de } mRNA.$$

Interpretação de n

$n = 1$: sem cooperatividade (Michaelis-Menten), $n > 1$: cooperatividade positiva (switch-like) e $n < 1$: cooperatividade negativa.

Simulação de Regulação Gênica.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def hill_function(TF, Vmax, Kd, n):
5     """Funcao de Hill para regulacao genica"""
6     return Vmax * TF**n / (Kd**n + TF**n)
7
8 # Parametros
9 Vmax = 100
10 Kd = 10
11 TF = np.linspace(0, 50, 200)
12
13 # Plotar diferentes cooperatividades
14 plt.figure(figsize=(10, 6))
15 for n in [1, 2, 4, 8]:
16     mRNA = hill_function(TF, Vmax, Kd, n)
17     plt.plot(TF, mRNA, linewidth=2, label=f'n={n}')
18
19 plt.axhline(y=Vmax/2, color='gray', linestyle='--')
20 plt.axvline(x=Kd, color='gray', linestyle='--')
21 plt.xlabel('[Fator de Transcricao]')
22 plt.ylabel('Taxa de Transcricao')
23 plt.title('Regulacao Genica: Efeito da Cooperatividade')
24 plt.legend()
25 plt.grid(True)

```

Listing 6.2: Função de Hill

6.4 Tipos de RNA

6.4-6.6 Tipos de RNA.

Classificação Funcional

mensageiro

.

(proteínas)

ribossomal

.

(estrutura, catálise)

transferência

.

(adaptador)

microRNA

.

(silenciamento)

RNA interferente

.

(degradação)

longo não-codificante

.

(regulação cromossômica)

nucleolar pequeno

.

(modificação de rRNA)

6.4 RNA Mensageiro (mRNA).

Estrutura do mRNA em Eucariotos

proteção, reconhecimento para tradução, regulação traducional, sequência co-dificante (Coding Sequence), estabilidade, localização e estabilidade, exportação nuclear.

Processamento do mRNA em Eucariotos.

6.5 RNA de Transferência (tRNA).

Braço acceptor (3' CCA), Braço D, Braço anticódon, Braço T

CeBraçovariável.

Transporta aminoácidos, Reconhece códons via anticódon e Adaptador molecular.

Wobble Pairing

Flexibilidade na 3ª posição do códon

6.6 RNA Ribossomal (rRNA).

Componente Estrutural dos Ribossomos

Estrutura do ribossomo, Atividade peptidil-transferase, Ligação de tRNA e Translocação.

— IMP:

rRNA é uma : RNA com atividade catalítica!

RNAs Regulatórios: microRNA (miRNA).

Biogênese do miRNA

pri-miRNA (longo, hairpin), Drosha

pre-miRNA(

70 nt), Exportina-5, Dicer

miRNA Maduro(

22 nt) e complexo de silenciamento.

Mecanismo de Ação

clivagem do mRNA-alvo, repressão traducional e Um miRNA pode regular centenas de mRNAs.

6.5 Medição de Expressão Gênica

6.7 Medição de Expressão Gênica.

Técnicas Principais

Northern blot, RT-PCR / qPCR, Microarrays, RNA-seq e Single-cell RNA-seq.

Western blot, Imunofluorescência, Espectrometria de massa e Flow cytometry.

— IMP:

mRNA

Proteínas sempre!

Regulação pós-transcricional e pós-traducional

Microarrays de DNA.

Sondas fixadas em chip, Hibridização de cDNA marcado, Detecção fluorescente e Análise de intensidade.

Perfil de expressão global, Comparação de condições, Classificação de amostras e Descoberta de genes.

RNA-seq: Sequenciamento de RNA.

Vantagens sobre Microarrays

Maior faixa dinâmica, Sem necessidade de sondas pré-definidas, Detecção de novos transcritos, Resolução de nucleotídeo único, Identificação de variantes de splicing e Quantificação absoluta (não relativa).

— Pipeline RNA-seq —

Extração de RNA, Preparação de biblioteca (cDNA), Sequenciamento (Illumina, PacBio, etc.), Alinhamento a genoma de referência, Contagem de reads por gene e Análise de expressão diferencial.

Análise de Dados de RNA-seq.

```
1 import pandas as pd
2 import numpy as np
```

```

3 from scipy import stats
4
5 # Carregar contagens (genes x amostras)
6 counts = pd.read_csv('gene_counts.csv', index_col=0)
7
8 # Normalizar (TPM, RPKM, ou DESeq2)
9 def tpm_normalize(counts, gene_lengths):
10     """Normalizar para TPM (Transcripts Per Million)"""
11     # Reads per kilobase
12     rpk = counts / (gene_lengths / 1000)
13     # Scaling factor
14     scaling_factor = rpk.sum() / 1e6
15     tpm = rpk / scaling_factor
16     return tpm
17
18 # Teste t para expressao diferencial
19 def differential_expression(control, treatment):
20     """Teste t para cada gene"""
21     t_stat, p_value = stats.ttest_ind(
22         control, treatment, axis=1
23     )
24     log_fc = np.log2(
25         treatment.mean(axis=1) / control.mean(axis=1)
26     )
27     return log_fc, p_value
28
29 # Aplicar
30 log_fc, p_val = differential_expression(
31     counts.iloc[:, :3], # controle
32     counts.iloc[:, 3:] # tratamento
33 )

```

Listing 6.3: Análise de expressão diferencial

Visualização: Volcano Plot.

```

1 import matplotlib.pyplot as plt
2
3 # Calcular -log10(p-value)
4 neg_log_p = -np.log10(p_val)
5
6 # Criar volcano plot
7 plt.figure(figsize=(10, 8))
8 plt.scatter(log_fc, neg_log_p, alpha=0.5, s=10)
9
10 # Threshold lines
11 plt.axhline(y=-np.log10(0.05), color='r',
12             linestyle='--', label='p=0.05')
13 plt.axvline(x=-1, color='b', linestyle='--')
14 plt.axvline(x=1, color='b', linestyle='--',
15             label='|log2FC|>1')
16
17 # Destacar genes significativos
18 sig = (p_val < 0.05) & (np.abs(log_fc) > 1)
19 plt.scatter(log_fc[sig], neg_log_p[sig],
20             color='red', s=20, label='Significativo')
21
22 plt.xlabel('log2(FoldChange)')
23 plt.ylabel('-log10(p-value)')
24 plt.title('VolcanoPlot: ExpressaoDiferencial')
25 plt.legend()
26 plt.grid(True, alpha=0.3)

```

Listing 6.4: Volcano plot para DE genes

6.6 Localização de Genes

6.8 Localização de Genes (Gene Localization).

Predição de Genes Ab Initio

Métodos computacionais que identificam genes baseando-se apenas na sequência de DNA:

Busca de ORFs, Detecção de sinais (promotores, splice sites), Modelos de Markov Ocultos (HMM) e Redes neurais.

Predição Baseada em Evidências

Usa dados experimentais:

Alinhamento de ESTs (Expressed Sequence Tags), Alinhamento de proteínas homólogas, Dados de RNA-seq e Sintenia (comparação genômica).

Ferramentas de Predição de Genes.

-
-
-
-

Características

Genes densos, Poucos íntrons, Operons e Mais simples.

-
-
-

(pipeline).

-

Desafios

Íntrons, Splicing alternativo, Genes esparsos e DNA repetitivo.

Exemplo: Usando BioPython para Anotação.

```
1 from Bio import SeqIO
2 from Bio.SeqUtils import GC
3
4 def annotate_sequence(fasta_file):
5     """Anotacao basica de sequencia"""
6     for record in SeqIO.parse(fasta_file, "fasta"):
7         seq = record.seq
8         print(f"ID: {record.id}")
9         print(f"Comprimento: {len(seq)} bp")
10        print(f"GC%: {GC(seq):.2f}%")
11
12        # Buscar ORFs em todas as 6 frames
13        for strand, nuc in [(+1, seq), (-1, seq.reverse_complement())]:
14            for frame in range(3):
15                length = 3 * ((len(seq) - frame) // 3)
16                for pro in nuc[frame:frame+length].translate().split("*"):
17                    if len(pro) >= 100: # min 100 aa
18                        print(f"Frame {strand}-{frame}: ORF de {len(pro)} aa")
19
20 # Usar
21 annotate_sequence("genome.fasta")
```

Listing 6.5: Busca de ORFs com BioPython

6.7 GenBank

GenBank: Banco de Dados de Sequências.

O que é GenBank?

Banco de dados público do NCBI (National Center for Biotechnology Information), Coleção abrangente de sequências de DNA, Anotações de genes, proteínas, características, Atualizado diariamente e Livre acesso.

Estatísticas (2024)

350 milhões de sequências, 10 trilhões de pares de bases e Crescimento exponencial.

URL

Formato de Arquivo GenBank.

Estrutura

LOCUS NM₀005185510*bpmRNA*

DEFINITION Homo sapiens hemoglobin beta

ACCESSION NM₀00518

VERSION NM₀00518.5

ORGANISM Homo sapiens

FEATURES Location/Qualifiers

gene 1..5510

/gene="HBB"

CDS 51..495

/product="hemoglobin beta"

/translation="MVHLTPEEKSAVTAL..."

ORIGIN

1 acatttgctt ctgacacaac tgtgttcact agcaacctca

41 aacagacacc atggtgcac tgactcctga ggagaagtct

...

//

Acessando GenBank com BioPython.

```

1 from Bio import Entrez, SeqIO
2
3 # Sempre fornecer email
4 Entrez.email = "seu_email@example.com"
5
6 # Buscar por gene
7 def search_genbank(query, max_results=10):
8     """Busca no GenBank"""
9     handle = Entrez.esearch(

```

```

10     db="nucleotide",
11     term=query,
12     retmax=max_results
13 )
14 record = Entrez.read(handle)
15 return record["IdList"]
16
17 # Download de sequencia
18 def fetch_sequence(genbank_id):
19     """Baixa sequencia do GenBank"""
20     handle = Entrez.efetch(
21         db="nucleotide",
22         id=genbank_id,
23         rettype="gb",
24         retmode="text"
25     )
26     record = SeqIO.read(handle, "genbank")
27     return record
28
29 # Exemplo
30 ids = search_genbank("Homo_sapiens_HBB")
31 seq = fetch_sequence(ids[0])
32 print(f"Gene: {seq.annotations['organism']}")
33 print(f"Tamanho: {len(seq)}bp")

```

Listing 6.6: Download de sequências

Análise de Features no GenBank.

```

1 def extract_features(record):
2     """Extrai features de um registro GenBank"""
3     features_dict = {
4         'genes': [],
5         'CDS': [],
6         'mRNA': []
7     }
8
9     for feature in record.features:
10         if feature.type == 'gene':
11             gene_name = feature.qualifiers.get('gene', ['Unknown'])[0]
12             location = f"{feature.location.start}-{feature.location.end}"
13             features_dict['genes'].append({
14                 'name': gene_name,
15                 'location': location
16             })
17
18         elif feature.type == 'CDS':
19             product = feature.qualifiers.get('product', ['Unknown'])[0]
20             translation = feature.qualifiers.get('translation', [''])[0]
21             features_dict['CDS'].append({
22                 'product': product,
23                 'length': len(translation)
24             })
25
26     return features_dict
27
28 # Usar
29 features = extract_features(seq)
30 print(f"Genes encontrados: {len(features['genes'])}")
31 print(f"CDSs encontradas: {len(features['CDS'])}")

```

Listing 6.7: Extrair features

6.8 Análise Integrada

Análise de Sequências: Pipeline Completo.

Bancos de Dados Relacionados.

NCBI

sequências, curadas, expressão, sequenciamento e literatura.

EBI (Europa)

sequências, microarrays e genomas.

Especializados

vias metabólicas, ontologia, proteínas, microRNAs e RNAs não-codificantes.

Integração

Ferramentas como DAVID, Enrichr e WebGestalt integram múltiplos bancos

Ontologia Gênica (GO).

Definição (Gene Ontology).

Vocabulário controlado para descrever funções de genes em três domínios:

Molecular Function (função molecular), Biological Process (processo biológico) e Cellular Component (componente celular).

Exemplo: Hemoglobina

oxygen transporter activity (GO:0005344), oxygen transport (GO:0015671) e hemoglobin complex (GO:0005833).

Análise de Enriquecimento

Identificar processos/funções super-representados em conjunto de genes

6.9 Estudos de Caso

Caso 1: Análise de Expressão em Câncer.

Objetivo

Identificar genes diferencialmente expressos entre tecido normal e tumoral

Coletar dados de RNA-seq (GEO/TCGA), Normalizar e filtrar, Análise de expressão diferencial (DESeq2), Correção para múltiplos testes (FDR), Análise de enriquecimento (GO, KEGG) e Validação experimental (qPCR).

Resultado Esperado

Lista de genes super/sub-expressos, Vias biológicas alteradas, Potenciais biomarcadores e Alvos terapêuticos.

Caso 2: Descoberta de microRNAs Regulatórios.

Objetivo

Identificar miRNAs que regulam genes envolvidos em resposta ao estresse

Perfil de expressão de miRNAs (small RNA-seq), Perfil de expressão de mRNAs (RNA-seq), Predição de alvos (TargetScan, miRanda), Correlação inversa miRNA-mRNA e Validação experimental (luciferase assay).

Ferramentas

banco de miRNAs, predição de alvos e interações validadas.

6.10 Exercícios

Exercícios - Parte 1.

Questão 1: Cálculo de Códon

Uma região codificante tem 1200 nucleotídeos.

Quantos códon ela contém?, Quantos aminoácidos na proteína resultante? e Se a taxa de tradução é 15 aa/segundo, quanto tempo leva?.

Questão 2: Conteúdo GC

Por que o conteúdo GC é importante? Calcule o conteúdo GC da sequência:

5'-ATGCGATCGTAGCTAGCTA-3'

Questão 3: Função de Hill

Para uma função de Hill com $n = 4$, $K_d = 10$, $V_{max} = 100$:

Calcule a expressão quando $[TF] = 10$ e Em que concentração se atinge 90

Exercícios - Parte 2.

— Questão 4: Projeto Prático - Análise de Sequência —

Escolha um gene de interesse (ex: TP53, BRCA1, MYC):

Baixe a sequência do GenBank usando BioPython, Identifique e anote todas as features, Calcule estatísticas básicas (tamanho, GC Identifique potenciais sítios de ligação de fatores de transcrição no promotor e Visualize a estrutura do gene (exons/introns).

— Questão 5: RNA-seq —

Com dados de contagens de RNA-seq:

Normalize os dados (TPM ou DESeq2), Identifique genes diferencialmente expressos, Faça análise de enriquecimento GO e Crie volcano plot e heatmap.

Exercícios - Parte 3.

Questão 6: Predição de miRNA

Dada uma sequência 3' UTR de um mRNA:

Use TargetScan para prever miRNAs que a regulam, Analise a conservação evolutiva dos sítios, Verifique dados de expressão: correlação negativa? e Proponha experimento para validar a interação.

Projeto Final

Desenvolva um pipeline de análise completo:

Download de dados de RNA-seq do GEO, Controle de qualidade (FastQC), Alinhamento ao genoma de referência, Contagem de reads por gene, Análise de expressão diferencial, Análise de enriquecimento funcional e Visualização e relatório.

6.11 Notas Bibliográficas

Referências Principais.

99

Watson J.D. et al. (2013). Molecular Biology of the Gene, 7th ed. Pearson.

Alberts B. et al. (2022). Molecular Biology of the Cell, 7th ed. Garland Science.

Cock P.J.A. et al. (2009). Biopython: freely available Python tools for computational molecular biology and bioinformatics. Bioinformatics.

Love M.I. et al. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biology.

Bartel D.P. (2018). Metazoan MicroRNAs. Cell.

Leitura Complementar.

Livros

Lewin B. (2020). Genes XII. Jones & Bartlett, Lodish H. et al. (2021). Molecular Cell Biology, 9th ed e Mount D.W. (2004). Bioinformatics: Sequence and Genome Analysis.

Recursos Online

-
-
-
-
-

Obrigado!

Capítulo 6: Sistemas Gênicos

Protein Systems (Sistemas Proteicos)

Dúvidas? Perguntas?

Capítulo 7

Sistemas de Proteínas

7.1 Introdução

Visão Geral do Capítulo.

Objetivos de Aprendizagem

Aprofundar conhecimento sobre estrutura de proteínas, Dominar cinética enzimática avançada, Compreender cascatas de sinalização celular, Estudar modificações pós-traducionais, Explorar métodos de proteômica e Aplicar ferramentas bioinformáticas para análise de proteínas.

— IMP:

Proteínas são as máquinas moleculares da vida: executam, regulam e coordenam praticamente todos os processos celulares

Importância das Proteínas (Estruturais e Catalíticas).

0.5

Colágeno (tecido conectivo), Queratina (cabelo, unhas), Actina/Miosina (músculo) e Histonas (empacotamento DNA).

Enzimas metabólicas, DNA/RNA polimerases, Proteases e Quinases.

Importância das Proteínas (Regulatórias e de Sinalização).

0.5

Fatores de transcrição, Hormônios peptídicos, Receptores de membrana e Proteínas G.

Anticorpos, Citocinas, Neurotransmissores e Mensageiros intracelulares.

Papel Central das Proteínas.

Proteínas são o elo entre genótipo (DNA) e fenótipo (função)

7.2 Estrutura de Proteínas

7.1 Estrutura de Proteínas - Níveis.

Quatro Níveis Estruturais

sequência linear de aminoácidos, arranjos locais (

—*hélice*,

-folha), estrutura 3D completa de uma cadeia e arranjo de múltiplas subunidades.

7.1 Estrutura de Proteínas - Visualização.

Termodinâmica do Dobramento Proteico.

Energia Livre de Gibbs

O dobramento proteico é governado pela mudança de energia livre:

$$\Delta G = \Delta H - T\Delta S$$

Onde:

$G < 0$: processo espontâneo (proteína dobrada é estável), H : entalpia (ligações, interações) e TS : entropia (desordem).

Fatores Termodinâmicos

Hidrogênio, eletrostática e Van der Waals e Principal força motriz; resíduos hidrofóbicos ao interior para estabilidade estrutural.

Diagrama de Ramachandran.

Definição (Ramachandran Plot).

Gráfico que mostra as combinações permitidas dos ângulos diedros

(ϕ)e

(ψ) da cadeia principal proteica.

.

—60, —45

.

—120, +120

.

+60, +45

— IMP:

Glicina (sem cadeia lateral) permite todos os ângulos. Prolina (imino ácido) restringe —60

Domínios e Motivos Proteicos.

Domínios

Unidades estruturais e funcionais independentes:

Dobram-se independentemente, 50-200 aminoácidos, Evolução modular e Exemplos: domínio SH2, domínio quinase.

Motivos

Padrões estruturais recorrentes:

Hélice-volta-hélice, Dedos de zinco, Barril TIM e Leucine zipper.

Proteínas Intrinsecamente Desordenadas (IDPs).

Características

Proteínas ou regiões sem estrutura 3D definida em condições fisiológicas:

Alto conteúdo de resíduos polares e carregados, Baixo conteúdo hidrofóbico, Flexibilidade conformacional e 30-40

Sinalização celular, Regulação da transcrição, Montagem de complexos e Promiscuidade de ligação.

Exemplos

p53 (supressor tumoral),

— *sinucleína*(Parkinson), *Tau*(Alzheimer) e *Histonas*(*caudas* N — terminais).

— IMP:

IDPs desafiam o paradigma estrutura-função: função sem estrutura fixa!

Protein Data Bank (PDB).

O que é o PDB?

Repositório global de estruturas 3D, URL:, Mais de 200,000 estruturas, Livre acesso e Formatos: PDB, mmCIF.

Cristalografia de raios-X (85 Ressonância magnética nuclear (NMR), Microscopia crioeletrônica (Cryo-EM) e Modelagem computacional.

Informações no PDB

Coordenadas atômicas, Estrutura secundária, Ligantes e cofatores, Metadados experimentais e Literatura associada.

Resolução

Medida em Ångströms (Å):

Alta resolução: < 2, Média resolução: 2-3 e Baixa resolução: > 3.

Análise de Estruturas com BioPython.

```

1 from Bio.PDB import PDBParser, PDBIO, Select
2 from Bio.PDB.DSSP import DSSP
3 import numpy as np
4
5 # Carregar estrutura
6 parser = PDBParser(QUIET=True)
7 structure = parser.get_structure("1UBQ", "1ubq.pdb")
8
9 # Extrair informacoes basicas
10 model = structure[0]
```

```

11 for chain in model:
12     print(f"Cadeia_{chain.id}:{len(chain)}_residuos")
13
14     # Calcular centro de massa
15     coords = []
16     for residue in chain:
17         if residue.id[0] == '_': # residuos padrao
18             for atom in residue:
19                 coords.append(atom.coord)
20
21     coords = np.array(coords)
22     center_mass = coords.mean(axis=0)
23     print(f"Centro_de_massa:{center_mass}")
24
25 # Analisar estrutura secundaria (requer DSSP)
26 dssp = DSSP(model, "1ubq.pdb")
27 secondary = [dssp[key][2] for key in dssp.keys()]
28 print(f"Helices:{secondary.count('H')}")
29 print(f"Folhas_beta:{secondary.count('E')}")

```

Listing 7.1: Parser de arquivos PDB

Visualização de Proteínas com PyMOL.

```

1 from pymol import cmd
2
3 # Carregar estrutura
4 cmd.load("1ubq.pdb")
5
6 # Colorir por estrutura secundaria
7 cmd.color("red", "ss_h") # helices em vermelho
8 cmd.color("yellow", "ss_s") # folhas em amarelo
9 cmd.color("green", "ss_l+'') # loops em verde
10
11 # Mostrar superficie
12 cmd.show("surface")
13 cmd.set("transparency", 0.5)
14
15 # Destacar residuos ativos (exemplo: His68)
16 cmd.select("site", "resi_68")
17 cmd.show("sticks", "site")
18 cmd.color("magenta", "site")
19
20 # Ajustar visualizacao
21 cmd.bg_color("white")
22 cmd.set("ray_shadows", 0)
23
24 # Salvar imagem
25 cmd.png("protein_structure.png", width=1200, dpi=300, ray=1)
26
27 print("Visualizacao_criada!")

```

Listing 7.2: Script PyMOL para visualização

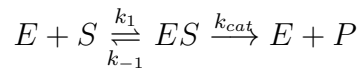
7.3 Cinética Enzimática Avançada

7.2 Cinética Enzimática - Revisão de Michaelis-Menten.

— Equação de Michaelis-Menten —

$$v = \frac{V_{max}[S]}{K_M + [S]}$$

Derivada do mecanismo:



$K_M = +k_{cat}/k_1$, $V_{max} = k_{cat}[E]_0$ e k_{cat} : número de turnover.

$$\frac{k_{cat}}{K_M} \quad (\text{M}^{-1}\text{s}^{-1})$$

Limite de difusão: 10^8 - $10^9 \text{ M}^{-1}\text{s}^{-1}$ e Enzimas perfeitas: anidrase carbônica, catalase.

Linearização: Gráfico de Lineweaver-Burk.

Duplo Recíproco

Invertendo a equação de MM:

$$\frac{1}{v} = \frac{K_M}{V_{max}} \cdot \frac{1}{[S]} + \frac{1}{V_{max}}$$

Forma linear: $y = mx + b$

Intercepto-y: $1/V_{max}$, Intercepto-x: $-1/K_M$ e Inclinação: K_M/V_{max} .

Desvantagens

Amplifica erros em v baixo e Distorce distribuição de erros.

Gráfico de Eadie-Hofstee.

Transformação Alternativa

Dividindo MM por $[S]$:

$$v = -K_M \cdot \frac{v}{[S]} + V_{max}$$

Vantagens:

Melhor distribuição de erros, Detecta desvios de MM e Identifica cooperatividade.

Comparação de Métodos

Hoje em dia, ajuste não-linear direto (regressão) é preferível às linearizações

Ligação Cooperativa e Equação de Hill.

Cooperatividade em Enzimas Oligoméricas

Quando ligação de substrato afeta afinidade de outros sítios:

$$v = \frac{V_{max}[S]^n}{K_d^n + [S]^n}$$

Onde n é o coeficiente de Hill:

$n = 1$: sem cooperatividade (Michaeliano), $n > 1$: cooperatividade positiva e

$n < 1$: cooperatividade negativa.

Ligação de O_2 é cooperativa: $n=2.8$

Primeira molécula de O₂ facilita ligação das demais (efeito sigmoide)

Gráfico de Hill.

Gráfico de Hill Linearizado

$$\log \frac{v}{V_{max} - v} = n \log[S] - \log K_d$$

Inclinação = n (coeficiente de Hill)

— IMP:

$n > 1$ resulta em resposta switch-like: pequena mudança em $[S]$ causa grande mudança em v

Regulação Alostérica: Modelo MWC (1/2).

Modelo MWC

Estados T (tenso) e R (relaxado), Transição concertada, Todas subunidades no mesmo estado e Ligante estabiliza estado R.

Regulação Alostérica: Modelo KNF (2/2).

Modelo KNF

Transição sequencial, Subunidades mudam individualmente, Ligação induz mudança conformacional e Afeta subunidades vizinhas.

Inibição Enzimática: Tipos.

Inibição Competitiva

Inibidor compete com substrato pelo sítio ativo

$$v = \frac{V_{max}[S]}{K_M \left(1 + \frac{[I]}{K_I}\right) + [S]}$$

V_{max} inalterado, K_M^{app} aumenta

— Inibição Não-Competitiva —

Inibidor liga em sítio diferente (não afeta ligação de S)

$$v = \frac{V_{max}[S]}{\left(1 + \frac{[I]}{K_I}\right) (K_M + [S])}$$

K_M inalterado, V_{max}^{app} diminui

Inibição Enzimática: Tipos Adicionais.

Inibição Incompetitiva

Inibidor liga apenas ao complexo ES

$$v = \frac{V_{max}[S]}{K_M + [S] \left(1 + \frac{[I]}{K_I}\right)}$$

K_M e V_{max} ambos diminuem proporcionalmente

Inibição Mista

Inibidor pode ligar tanto a E quanto a ES, com afinidades diferentes

$$v = \frac{V_{max}[S]}{K_M \left(1 + \frac{[I]}{K_I}\right) + [S] \left(1 + \frac{[I]}{K_I'}\right)}$$

Afeta tanto K_M quanto V_{max}

Lineweaver-Burk: Identificando Tipo de Inibição.

Inibição Reversível vs Irreversível.

Reversível

Inibidor liga não-covalentemente

Equilíbrio dinâmico, Pode ser removido (diálise), Constante de dissociação K_I e

Exemplos: metabólitos regulatórios.

Inibidores competitivos reversíveis da HMG-CoA redutase (síntese de colesterol)

Irreversível

Inibidor forma ligação covalente

Inativação permanente, Requer síntese de nova enzima, Taxa de inativação k_{inact}

e Exemplos: toxinas, fármacos.

Acetila irreversivelmente Ser530 da COX (inibe inflamação)

— IMP:

IC_{50} : concentração de inibidor que reduz atividade em 50

K_I : constante de dissociação do complexo EI (afinidade)

Ajuste de Dados de Cinética Enzimática.

```

1 import numpy as np
2 from scipy.optimize import curve_fit
3 import matplotlib.pyplot as plt
4
5 # Funcao de Michaelis-Menten
6 def michaelis_menten(S, Vmax, Km):
7     return Vmax * S / (Km + S)
8
9 # Dados experimentais
10 S = np.array([0.5, 1, 2, 5, 10, 20, 50]) # mM
11 v = np.array([15, 25, 38, 60, 75, 85, 95]) # umol/min
12
13 # Ajuste nao-linear
14 params, covariance = curve_fit(michaelis_menten, S, v,
15                               p0=[100, 5]) # estimativas iniciais
16
17 Vmax_fit, Km_fit = params
18 print(f"Vmax={Vmax_fit:.2f} umol/min")
19 print(f"Km={Km_fit:.2f} mM")
20 print(f"kcat/Km={Vmax_fit/Km_fit:.2f} min^-1 mM^-1")
21
22 # Plotar
23 S_fit = np.linspace(0, 50, 200)
24 v_fit = michaelis_menten(S_fit, Vmax_fit, Km_fit)
25 plt.plot(S, v, 'o', label='Dados')
26 plt.plot(S_fit, v_fit, '--', label='Ajuste')
27 plt.xlabel('[S] (mM)')
28 plt.ylabel('v (umol/min)')
29 plt.legend()

```

Listing 7.3: Regressão não-linear para Michaelis-Menten

Cálculo de IC50 e Ki.

```

1 from scipy.optimize import curve_fit
2
3 # Funcao dose-resposta (Hill)
4 def dose_response(I, IC50, n):
5     """Inibicao em funcao de [I]"""
6     return 100 / (1 + (I/IC50)**n)
7
8 # Dados de inibicao
9 I = np.array([0.01, 0.1, 1, 10, 100, 1000]) # uM
10 activity = np.array([98, 90, 70, 30, 8, 2]) # % atividade
11
12 # Ajuste
13 params, _ = curve_fit(dose_response, I, activity, p0=[10, 1])
14 IC50, n = params
15
16 print(f"IC50={IC50:.2f}uM")
17 print(f"Coefficiente de Hill={n:.2f}")
18
19 # Para inibidor competitivo, calcular Ki
20 S = 5 # mM (concentracao de substrato usada)
21 Km = 2 # mM
22 Ki = IC50 / (1 + S/Km)
23 print(f"Ki={Ki:.2f}uM")
24
25 # Plotar curva dose-resposta
26 I_fit = np.logspace(-2, 3, 200)
27 activity_fit = dose_response(I_fit, IC50, n)
28 plt.semilogx(I, activity, 'o', label='Dados')
29 plt.semilogx(I_fit, activity_fit, '-', label='Ajuste')
30 plt.axhline(y=50, color='r', linestyle='--', label='IC50')
31 plt.axvline(x=IC50, color='r', linestyle='--')
32 plt.xlabel('[Inibidor](uM)')
33 plt.ylabel('Atividade(%)')
34 plt.legend()

```

Listing 7.4: Determinação de IC50

7.4 Cascatas de Sinalização Celular

7.3 Sinalização Celular: Visão Geral (1/2).

Definição (Transdução de Sinal).

Processo pelo qual uma célula converte um sinal extracelular em uma resposta intracelular específica.

Etapas da transdução

Ligante extracelular é reconhecido por um receptor, O receptor ativa transdutores intracelulares e O sinal chega a efetores e gera resposta celular.

7.3 Sinalização Celular: Visão Geral (2/2).

0

Tipos de Receptores de Membrana (1/2).

GPCRs

7 hélices transmembrana, Ligação ativa proteína G, Maior família de receptores e Alvos de 30-40

Exemplos: receptores adrenérgicos, dopaminérgicos, olfativos

RTKs

1 hélice transmembrana, Atividade quinase intrínseca, Dimerização por ligante e Fosforilação de tirosinas.

Exemplos: EGFR, VEGFR, insulina

Tipos de Receptores de Membrana (2/2).

Canais Iônicos

Ligante abre canal, Fluxo de íons (Na^+ , K^+ , Ca^{2+} , Cl^-) e Resposta rápida (ms).

Exemplos: receptor nicotínico, GABA_A

Receptores Nucleares

Intracelulares, Ligante lipofílico e Regulação direta de genes.

Exemplos: receptores de esteroides, tireoide

Via GPCR: Proteínas G Heterotriméricas.

1

$\text{G}\alpha_s$: estimula adenilil ciclase, $\text{G}\alpha_i$: inibe adenilil ciclase e $\text{G}\alpha_q$: ativa fosfolipase C.

Ativa canais iônicos, Regula adenilil ciclase e Ativa PI3K.

Segundos Mensageiros.

cAMP

Produzido por adenilil ciclase, Ativa PKA (proteína quinase A), Degradado por fosfodiesterases e Via $\text{G}\alpha_s$.

Ca^{2+}

Liberado do RE, Liga calmodulina, Ativa CaMK, PKC e Concentração: 100 nM

1

M.

IP_3 e DAG

Produzidos por fosfolipase C, IP_3 libera Ca^{2+} do RE, DAG ativa PKC e Via $\text{G}\alpha_q$.

cGMP

Produzido por guanilil ciclase, Ativa PKG, Via óxido nítrico (NO) e Vasodilatação.

Cascata MAPK: Via Clássica.

2

MAPK: Módulo de Amplificação (1/2).

— IMP:

Cascata de fosforilação amplifica sinal:

1 RTK ativa múltiplas moléculas de Ras, 1 Ras ativa múltiplas moléculas de Raf, 1 Raf fosforila múltiplas MEK e 1 MEK fosforila múltiplas ERK.

Amplificação em Cascata

Se cada quinase ativa 10 moléculas da próxima:

$$\text{Amplificação} = 10^3 = 1000\times$$

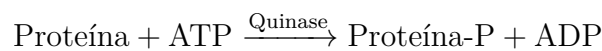
MAPK: Módulo de Amplificação (2/2).

Outras Vias MAPK

estresse, apoptose e inflamação, diferenciação.

Dinâmica de Fosforilação e Desfosforilação.

Ciclo de Fosforilação



Fosforilam Ser, Thr, Tyr,

518quinaseshumanas, Domínio catalítico conservado e Regulação por fosforilação.

Removem fosfatos, Menos específicas, Regulação por subunidades e Antagonizam quinases.

— DEST:

Balanco quinase/fosfatase determina estado de fosforilação

Ultrassensibilidade de Goldbeter-Koshland.

Modelo Matemático

Quando quinase e fosfatase operam próximas à saturação:

$$\frac{[\text{Proteína-P}]}{[\text{Proteína}]_{\text{total}}} = \frac{v_1 - v_2}{v_1 + v_2}$$

Resposta switch-like mesmo sem cooperatividade!

Alta [Proteína] vs. K_M , Ciclo de fosforilação/desfosforilação e Saturação de ambas enzimas.

3

— IMP:

Permite decisões digitais (tudo-ou-nada) em sistemas analógicos

Modelagem de Cascatas de Sinalização.

```
1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def mapk_cascade(y, t, k1, k2, k3):
6     """
```

```

7      Modelo simplificado de cascata MAPK
8      y = [MAPKKK*, MAPKK*, MAPK*] (* = fosforilado/ativo)
9      """
10     MAPKKK_active, MAPKK_active, MAPK_active = y
11
12     # Assumindo input constante e desfosforilacao linear
13     dMAPKKK = k1 - 0.1*MAPKKK_active
14     dMAPKK = k2*MAPKKK_active - 0.1*MAPKK_active
15     dMAPK = k3*MAPKK_active - 0.1*MAPK_active
16
17     return [dMAPKKK, dMAPKK, dMAPK]
18
19 # Parametros
20 k1, k2, k3 = 1.0, 2.0, 3.0 # taxas de ativacao
21 y0 = [0, 0, 0] # condicoes iniciais
22 t = np.linspace(0, 50, 500)
23
24 # Resolver EDO
25 solution = odeint(mapk_cascade, y0, t, args=(k1, k2, k3))
26
27 # Plotar
28 plt.plot(t, solution[:, 0], label='MAPKKK*')
29 plt.plot(t, solution[:, 1], label='MAPKK*')
30 plt.plot(t, solution[:, 2], label='MAPK*', linewidth=2)
31 plt.xlabel('Tempo (min)')
32 plt.ylabel('Concentracao (uM)')
33 plt.title('Cascata MAPK: Amplificacao de Sinal')
34 plt.legend()
35 plt.grid(True)

```

Listing 7.5: Cascata MAPK simplificada

7.5 Modificações Pós-Traducionais

7.4 Modificações Pós-Traducionais (PTMs) (1/2).

Definição (PTM).

Modificações químicas covalentes em proteínas após tradução, expandindo diversidade e regulação do proteoma.

Fosforilação (Ser, Thr, Tyr), Acetilação (Lys), Metilação (Lys, Arg), Ubiquitinação (Lys), Glicosilação (Asn, Ser, Thr) e SUMOilação (Lys).

Estatística

>400 tipos de PTMs conhecidas!

Proteína média: 5-10 PTMs diferentes

7.4 Modificações Pós-Traducionais (PTMs) (2/2).

— IMP:

PTMs regulam:

Atividade enzimática, Localização subcelular, Interações proteína-proteína, Estabilidade e Conformação.

Tipos adicionais

Nitrosilação (Cys) e Palmitoilação (Cys).

Fosforilação: PTM mais Estudada.

Características

Mais abundante e dinâmica, Reversível (quinases/fosfatases), Adição de grupo fosfato (PO_4^{3-}), Carga negativa: muda interações eletrostáticas e

30

90 10 <1 e His, Asp (procariotos).

Ativação/inativação enzimática, Criação de sítios de ligação (SH2, 14-3-3), Mudanças conformacionais e Marcação para degradação.

Acetilação e Metilação: Regulação Epigenética.

Acetilação

Principalmente em Lys, Neutraliza carga positiva, HATs: histona acetiltransferases e HDACs: histona deacetilases.

Reduz afinidade DNA-histona, Cromatina aberta (eucromatina) e Ativa transcrição.

Metilação

Lys (mono, di, tri-metil), Arg (mono, di-metil), HMTs: histona metiltransferases e HDMs: histona demetilases.

H3K4me3: ativação, H3K9me3: repressão e H3K27me3: silenciamento.

— DEST:

Código de histonas: combinação de PTMs determina estado transcricional

Ubiquitinação e Degradação Proteica.

— Sistema Ubiquitina-Proteassoma —

proteína pequena (76 aa) que marca alvos para degradação

4

K48: degradação (26S), K63: sinalização (não-degradativa) e Mono-Ub: endocitose, reparo DNA.

— IMP:

E3 ligases conferem especificidade:

>600 E3s humanas!

Glicosilação: Modificação Mais Complexa.

— N-Glicosilação —

Ocorre no RE, Oligossacarídeo pré-formado, Transferido en bloc e Processamento no Golgi.

— O-Glicosilação —

Construção sequencial, Golgi e citoplasma, Mais diversa e O-GlcNAc (citoplasma).

Dobramento proteico, Estabilidade, Reconhecimento celular, Adesão celular e Imunidade (anticorpos).

Importância Clínica

Hemoglobina glicosilada (HbA1c): diabetes, Glicosilação aberrante: câncer e Grupo sanguíneo: glicosilação de RBCs.

Crosstalk entre PTMs.

— IMP:

PTMs não agem isoladamente: interagem e se regulam mutuamente!

Exemplos de Crosstalk

mesmo resíduo (Ser/Thr vs. Lys adjacente), fosfo-degron, mesmas Ser/Thr, em histonas e mesmas Lys.

Acetilação (ativa) vs. Ubiquitinação (degrada) em Lys₃₈₂ regula estabilidade do supressor tumoral p53

Bancos de Dados de PTMs.

PhosphoSitePlus

PTMs experimentalmente verificadas, >500,000 sítios, Fosforilação, acetilação, ubiquitinação, etc, Regulação up/down e Informação funcional.

UniProt

PTMs anotadas, Integração com literatura e Variantes e mutações.

Outros Recursos

integração de PTMs, crosstalk, fosforilação, metilação de lisinas e O-GlcNAc.

Deteção

Espectrometria de massa (LC-MS/MS)

Identifica modificações, Mapeia sítios exatos e Quantificação relativa/absoluta.

PTMs no Controle do Ciclo Celular (1/2).

Progressão pelo ciclo celular é regulada por fosforilação:

5

PTMs no Controle do Ciclo Celular (2/2).

Principais Efeitos

libera E2F

transição G1/S, condensação cromossomale entrada em mitose.

7.6 Proteômica

7.5 Proteômica: Estudo do Proteoma.

Definição (Proteoma).

Conjunto completo de proteínas expressas por um genoma, célula ou tecido em determinado momento e condição.

mRNA

Proteína, PTMs não previstas por DNA, Abundância dinâmica, Interações proteína-proteína e Localização

Faixa dinâmica enorme (10^6), Diversidade química, Modificações pós-traducionais, Complexos transitórios e Sem amplificação (vs. PCR).

— IMP:

Proteoma humano:

20,000 *genes*

>1,000,000 proteoformas (com PTMs e isoformas)

Abordagens Proteômicas.

— Bottom-Up —

Digestão proteolítica (tripsina), Peptídeos analisados por MS, Identificação por busca em banco e Reconstrução de proteínas.

Mais sensível, Cobertura ampla e PTMs mapeadas.

— Top-Down —

Proteínas inteiras no MS, Fragmentação no espectrômetro e Análise direta.

Isoformas preservadas, Combinações de PTMs e Sem ambiguidade de montagem.

Menos sensível e Proteínas grandes difíceis.

Espectrometria de Massa: LC-MS/MS.

6

Massa/carga (m/z), Intensidade do íon e Seleção de precursores.

Fragmentação: CID, HCD, ETD, Espectro de fragmentos e Sequência do peptídeo.

Técnicas de Ionização.

ESI

Ionização suave, Íons múltiplos carregados, Acoplado a LC (LC-MS), Peptídeos e proteínas pequenas e Mais usado em proteômica.

— MALDI-TOF —

Laser dissolve amostra, Matriz absorve energia, Íons simples carregados, Time-of-Flight (ToF) e Bom para proteínas grandes.

Analísadores de Massa

alta resolução, alta acurácia, tandem MS, bom para PTMs e quantificação (SRM/MRM).

Quantificação Proteômica (1/2).

— Label-Free —

Intensidade do sinal MS1, Contagem de espectros (SpC), iBAQ, LFQ, Simples, econômico e Variabilidade maior.

SILAC

Aminoácidos marcados (^{13}C , ^{15}N), Incorporação metabólica, Multiplexação (até 3 condições), Quantificação acurada e Requer cultura celular.

Quantificação Proteômica (2/2).

Marcação Química

Tags isobáricas, Multiplexação (6-18 condições), Quantificação em MS2, Versátil (qualquer amostra) e Ratio compression.

Escolha do Método

descoberta, custo, acurácia, cultura e multiplexação, clínica.

Interações Proteína-Proteína.

Métodos Experimentais

In vivo em levedura, Interações binárias, Triagem em larga escala e Falsos positivos.

Anticorpo puxa proteína, Complexos co-purificados, Condições nativas e Validação de interações.

Tag (FLAG, HA, Strep), Purificação de complexos, Identificação por MS e Mapeamento de complexos.

Marcação por proximidade, Biotina em vizinhos, Interações transitórias e In vivo.

Bancos de Dados de Interações.

STRING

Interações conhecidas e preditas, Integração de evidências, Scores de confiança, Visualização de redes e Análise de enriquecimento.

BioGRID

Interações experimentais, Curado manualmente e Múltiplas espécies.

IntAct

Base do EBI, Dados estruturados e PSI-MI format.

CORUM

Complexos proteicos, Mamíferos e Composição e função.

— IMP:

Dados de diferentes métodos são complementares: integração é chave!

Análise de Dados Proteômicos.

```
1 import pandas as pd
2 import numpy as np
3 from scipy.stats import ttest_ind
4 import matplotlib.pyplot as plt
5
6 # Carregar dados de quantificacao (ex: MaxQuant output)
7 df = pd.read_csv('proteinGroups.txt', sep='\t')
8
9 # Selecionar colunas de intensidade
```

```

10 control_cols = [f'LFQ_intensity_Ctrl_{i}' for i in range(1,4)]
11 treatment_cols = [f'LFQ_intensity_Treat_{i}' for i in range(1,4)]
12
13 # Log2 transform
14 df[control_cols] = np.log2(df[control_cols].replace(0, np.nan))
15 df[treatment_cols] = np.log2(df[treatment_cols].replace(0, np.nan))
16
17 # Filtrar proteínas com muitos missing values
18 df = df.dropna(thresh=4, subset=control_cols+treatment_cols)
19
20 # Calcular fold change
21 df['log2FC'] = df[treatment_cols].mean(axis=1) - \
22               df[control_cols].mean(axis=1)
23
24 # Teste t
25 t_stat, p_val = [], []
26 for idx, row in df.iterrows():
27     ctrl = row[control_cols].dropna()
28     treat = row[treatment_cols].dropna()
29     if len(ctrl) >= 2 and len(treat) >= 2:
30         t, p = ttest_ind(ctrl, treat)
31         t_stat.append(t)
32         p_val.append(p)
33     else:
34         t_stat.append(np.nan)
35         p_val.append(np.nan)
36
37 df['p_value'] = p_val
38 df['-log10(p)'] = -np.log10(df['p_value'])

```

Listing 7.6: Processamento de dados MS

Visualização: Volcano Plot (1/2).

```

1 # Continuação do código anterior
2
3 # Thresholds
4 fc_thresh = 1 # log2 fold change
5 p_thresh = 0.05
6
7 # Classificar proteínas
8 df['significant'] = 'No'
9 df.loc[(df['log2FC'] > fc_thresh) & (df['p_value'] < p_thresh),
10        'significant'] = 'Up'
11 df.loc[(df['log2FC'] < -fc_thresh) & (df['p_value'] < p_thresh),
12        'significant'] = 'Down'

```

Listing 7.7: Volcano plot proteômico

Visualização: Volcano Plot (2/2).

```

1
2 # Volcano plot
3 plt.figure(figsize=(10, 8))
4 colors = {'No': 'gray', 'Up': 'red', 'Down': 'blue'}
5 for sig, group in df.groupby('significant'):
6     plt.scatter(group['log2FC'], group['-log10(p)'],
7                 c=colors[sig], label=sig, alpha=0.6, s=20)
8
9 # Threshold lines
10 plt.axhline(y=-np.log10(p_thresh), color='black',
11             linestyle='--', linewidth=1)
12 plt.axvline(x=fc_thresh, color='black',
13            linestyle='--', linewidth=1)
14 plt.axvline(x=-fc_thresh, color='black',
15            linestyle='--', linewidth=1)
16
17 plt.xlabel('log2(FoldChange)', fontsize=14)
18 plt.ylabel('-log10(p-value)', fontsize=14)
19 plt.title('Volcano Plot: Proteínas Diferencialmente Expressas')
20 plt.legend()
21 plt.grid(True, alpha=0.3)

```

Listing 7.8: Volcano plot proteômico

7.7 Exercícios

Exercícios - Parte 1: Cinética Enzimática.

Questão 1: Parâmetros de MM

Uma enzima apresenta $K_M = 5$ mM e $V_{max} = 100$

mol/min. Calcule :

A velocidade quando $[S] = 15$ mM, A concentração de substrato para $v = 75$

mol/min. Eficiência catalítica se

$[E]_0 = 1$

M.

Questão 2: Inibição

Um inibidor competitivo tem $K_I = 2$ mM. Se $K_M = 5$ mM e $[I] = 10$ mM:

Qual é o K_{Mpp} ?, Como isso afeta o gráfico de Lineweaver-Burk? e Que concentração de substrato é necessária para superar 90

Exercícios - Parte 2: Estrutura e Análise.

Questão 3: Ramachandran

Explique por que:

Glicina tem maior liberdade conformacional, Prolina restringe ângulos

e

e Regiões proibidas existem no diagrama.

— Questão 4: Projeto Prático - BioPython —

Escolha uma proteína do PDB:

Baixe e parse a estrutura com BioPython, Calcule proporção de estruturas secundárias (

,

, loop), Identifique resíduos hidrofóbicos vs. hidrofílicos, Visualize em PyMOL colorindo por propriedade e Analise cavidades potenciais de ligação.

Exercícios - Parte 3: Sinalização.

Questão 5: Cascata MAPK

Considerando amplificação 10x em cada nível:

Quantas moléculas de ERK são ativadas por 1 RTK?, Se 100 RTKs são ativados, quantas ERK fosforiladas? e Como feedback negativo afeta isso?.

Exercícios - Parte 3: PTMs.

Questão 6: PTMs

Uma proteína tem os seguintes sítios:

Lys₄₈: pode ser acetilada ou ubiquitinada e Ser₁₃₂: pode ser fosforilada ou O-GlcNAcilada.

Quantas combinações de PTMs são possíveis?, Como crosstalk entre PTMs regula função? e Que técnica usaria para detectar cada modificação?.

Exercícios - Parte 4: Proteômica.

Projeto Final: Pipeline Proteômico

Desenvolva análise completa de dados de proteômica:

Baixe dados públicos (PRIDE, ProteomeXchange), Pré-processamento: log-transform, normalização, filtro, Análise estatística: teste t, correção FDR, Volcano plot e heatmap, Análise de enriquecimento GO/KEGG, Rede de interações proteína-proteína (STRING), Identificação de hubs e módulos funcionais e Relatório com interpretação biológica.

Questão 7: Quantificação

Compare métodos label-free, SILAC e TMT:

Quando usar cada um?, Vantagens e limitações? e Qual é mais apropriado para amostras clínicas?.

7.8 Notas Bibliográficas

Referências Principais.

99

Berg J.M., Tymoczko J.L., Stryer L. (2019). Biochemistry, 9th ed. W.H. Freeman.
Voet D., Voet J.G., Pratt C.W. (2016). Fundamentals of Biochemistry, 5th ed. Wiley.

Lodish H. et al. (2021). Molecular Cell Biology, 9th ed. W.H. Freeman.

Aebersold R., Mann M. (2016). Mass-spectrometric exploration of proteome structure and function. Nature.

Mann M., Jensen O.N. (2003). Proteomic analysis of post-translational modifications. Nature Biotechnology.

Referências Complementares.

99

Cornish-Bowden A. (2012). Fundamentals of Enzyme Kinetics, 4th ed. Wiley-Blackwell.

Cox J., Mann M. (2011). Quantitative, high-resolution proteomics for data-driven systems biology. Annual Review of Biochemistry.

Koshland D.E. et al. (1966). Comparison of experimental binding data and theoretical models in proteins containing subunits. Biochemistry.

Goldbeter A., Koshland D.E. (1981). An amplified sensitivity arising from covalent modification in biological systems. PNAS.

Venne A.S. et al. (2014). The next level of complexity: Crosstalk of posttranslational modifications. Proteomics.

Leitura Complementar e Recursos.

Livros Especializados

Bairoch A. et al. (2016). The SWISS-PROT Protein Knowledgebase, Liebler D.C. (2001). Introduction to Proteomics e Cech T.R., Atkins J.F. (2011). RNA Worlds, 3rd ed.

Recursos Online

-
-
-
-
-
-

(enzimas).

Obrigado!

Capítulo 7: Protein Systems

Metabolic Systems (Sistemas Metabólicos)

Dúvidas? Perguntas?

Capítulo 8

Sistemas Metabólicos

8.1 Introdução

Visão Geral do Capítulo.

Objetivos de Aprendizagem

Compreender estrutura e dinâmica de redes metabólicas, Modelar cinéticas de reações enzimáticas, Aplicar análise de fluxo metabólico (MFA) e FBA e Interpretar dados metabolômicos.

Por que Estudar Metabolismo?

Produção de energia e biomoléculas, Engenharia metabólica e biotecnologia e Compreensão de doenças metabólicas e design de terapias.

O Papel Central do Metabolismo.

Definição (Metabolismo).

Conjunto de reações químicas interconectadas que convertem nutrientes em energia e blocos construtores para crescimento e manutenção celular.

— IMP:

Redes metabólicas são sistemas complexos com milhares de reações interconectadas

Redes Metabólicas como Sistemas Complexos.

centenas a milhares de reações, altamente interconectadas, múltiplos níveis de controle e redundância e flexibilidade.

dinâmica de reações individuais, balanços de massa, estrutura da rede e controle de fluxo.

Abordagens Computacionais

Modelagem cinética, FBA, MFA, análise de modos elementares, metabolômica

8.2 Cinéticas de Reação Metabólica

Visão Geral das Cinéticas Enzimáticas.

Por que Modelar Cinéticas?

Predizer comportamento dinâmico de vias metabólicas, Identificar passos limitantes e avaliar regulação alostérica e Orientar estratégias de engenharia metabólica.

Ação de massas, Michaelis-Menten, Hill cooperativo, Convenience kinetics e GMA/S-systems.

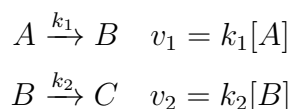
Cinética de Ação de Massas.

Lei de Ação de Massas

Para uma reação elementar $A + B \rightarrow C$:

$$v = k[A][B]$$

aplica-se a reações elementares; linear em cada reagente; base para derivar outras cinéticas.



EDOs: $= -k_1[A]; = k_1[A] - k_2[B]; = k_2[B]$

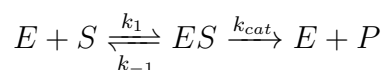
Cinética de Michaelis-Menten.

— DEF: Michaelis-Menten —

Modelo padrão para catálise enzimática com um substrato:

$$v = \frac{V_{max}[S]}{K_M + [S]}$$

onde $V_{max} = k_{cat}[E]_T$ e $K_M = (k_{-1} + k_{cat})/k_1$



Hipótese: t_0

K_M : afinidade (baixo K_M = alta afinidade), V_{max} : capacidade catalítica máxima e k_{cat}/K_M : eficiência catalítica.

Equação de Hill: Cooperatividade.

Cinética de Hill

Para enzimas com múltiplos sítios de ligação e cooperatividade:

$$v = \frac{V_{max}[S]^n}{K_{0.5} + [S]^n}$$

onde n é o coeficiente de Hill e $K_{0.5}$ é a constante de meia-saturação.

$n = 1$: sem cooperatividade (MM), $n > 1$: cooperatividade positiva e $n < 1$: cooperatividade negativa.

$n \gg 1$ (cooperatividade positiva forte)

Michaelis-Menten Reversível.

Reação Bidirecional

Para reação reversível S

equilíbrio P :

— DEST:

$$v = \frac{V_f \frac{[S]}{K_{MS}} - V_r \frac{[P]}{K_{MP}}}{1 + \frac{[S]}{K_{MS}} + \frac{[P]}{K_{MP}}}$$

onde V_f , V_r : velocidades máximas forward/reverse; K_{MS} , K_{MP} : constantes de Michaelis para S e P.

Relação com Termodinâmica

No equilíbrio ($v = 0$):

$$K_{eq} = \frac{[P]_{eq}}{[S]_{eq}} = \frac{V_f K_{MP}}{V_r K_{MS}}$$

Convenience Kinetics.

Forma Geral

Aproximação prática que incorpora inibição por produto e regulação:

$$v = E_T \cdot k_{cat} \cdot \frac{\frac{[S]}{K_S} \left(1 - \frac{[P]}{[S]K_{eq}}\right)}{\left(1 + \frac{[S]}{K_S} + \frac{[P]}{K_P}\right) \left(1 + \sum_i \frac{[I_i]}{K_{I_i}}\right)}$$

Termodinamicamente consistente, Incorpora múltiplos efetores e Parametrização mais simples.

Menos mecanística e Requer ajuste cuidadoso.

Generalized Mass Action (GMA) e S-Systems.

GMA (Lei de Potência)

$$v_i = \alpha_i \prod_{j=1}^n X_j^{g_{ij}}$$

$$\frac{dX_i}{dt} = \sum_k \alpha_k \prod_j X_j^{g_{kj}} - \sum_l \beta_l \prod_j X_j^{h_{lj}}$$

— S-Systems —

Forma especial de GMA:

$$\frac{dX_i}{dt} = \alpha_i \prod_{j=1}^n X_j^{g_{ij}} - \beta_i \prod_{j=1}^n X_j^{h_{ij}}$$

Termo de produção agregado e Termo de degradação agregado.

— IMP:

S-systems permitem análise algébrica de estado estacionário e são base da Biochemical Systems Theory (BST)

Comparação de Formalismos Cinéticos.

Escolha do Modelo

: Michaelis-Menten, Hill, : Convenience kinetics, : S-systems, FBA e : FBA (sem cinética).

Implementação: Cinética de Michaelis-Menten.

```

1 import numpy as np
2 from scipy.integrate import odeint
3
4 def michaelis_menten(S, t, Vmax, Km, k_deg):
5     """Modelo: S -> P com MM kinetics"""
6     v = Vmax * S / (Km + S)
7     dSdt = -v - k_deg * S
8     return dSdt
9
10 # Parametros
11 Vmax, Km, k_deg = 10.0, 2.0, 0.1 # umol/min, mM, 1/min
12 S0 = 20.0 # mM
13 t = np.linspace(0, 50, 500)
14
15 # Resolver e plotar
16 S = odeint(michaelis_menten, S0, t, args=(Vmax, Km, k_deg))
17
18 import matplotlib.pyplot as plt
19 plt.figure(figsize=(10, 6))
20 plt.plot(t, S, 'b-', linewidth=2, label='[S](t)')
21 plt.xlabel('Tempo (min)')
22 plt.ylabel('Concentrac o [S] (mM)')
23 plt.title('Cinetica de Michaelis-Menten')
24 plt.legend(); plt.grid(True, alpha=0.3)

```

Listing 8.1: Modelo cinético simples

Implementação: Via Metabólica Simples.

```

1 def metabolic_pathway(y, t, params):
2     """Via metabólica: A -> B -> C -> D"""
3     A, B, C, D = y
4     Vmax1, Km1, Vmax2, Km2, Vmax3, Km3 = params
5     v1 = Vmax1 * A / (Km1 + A)
6     v2 = Vmax2 * B / (Km2 + B)
7     v3 = Vmax3 * C / (Km3 + C)
8     return [-v1, v1-v2, v2-v3, v3]
9
10 params = [10.0, 1.0, 8.0, 1.5, 5.0, 2.0]
11 y0 = [50.0, 0.0, 0.0, 0.0]
12 t = np.linspace(0, 20, 500)
13 sol = odeint(metabolic_pathway, y0, t, args=(params,))
14
15 labels = ['A', 'B', 'C', 'D']
16 colors = ['b-', 'r-', 'g-', 'm-']
17 for i, (lbl, clr) in enumerate(zip(labels, colors)):
18     plt.plot(t, sol[:, i], clr, label=lbl, linewidth=2)

```

```
19 plt.legend()
```

Listing 8.2: Via de três passos

8.3 Análise de Fluxo Metabólico (MFA)

Análise de Fluxo Metabólico (MFA).

Definição (Metabolic Flux Analysis (MFA)).

Técnica experimental e computacional para determinar taxas de reação (fluxos) em uma rede metabólica usando dados de medições e balanços de massa.

Princípios Fundamentais

entrada = saída + acúmulo, metabólitos intracelulares constantes, taxas de consumo/produção e rastreamento de carbono-13.

Aplicações

Engenharia metabólica, identificação de gargalos, otimização de bioprocessos

Representação Estequiométrica.

— Matriz Estequiométrica

Representa a topologia da rede:

Linhas: metabólitos (m), Colunas: reações (n), $S_{ij} > 0$: metabólito i produzido por reação j e $S_{ij} < 0$: metabólito i consumido por reação j .

Reações:



Matriz estequiométrica:

$$\mathbf{S} = \begin{bmatrix} -1 & 0 & 0 \\ 1 & -1 & -1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{matrix} \leftarrow A \\ \leftarrow B \\ \leftarrow C \\ \leftarrow D \end{matrix}$$

Balanco de Massa.

Equação de Balanço

Para cada metabólito intracelular:

— DEST:

$$\frac{dc}{dt} = S \cdot v$$

onde

$= \text{concentrações}(\text{m})$

$= \text{estequiométrica}(\text{m})$; $= \text{fluxos}(\text{n})$

Hipótese de Estado Estacionário

Assumindo t :

$$S \cdot v = 0$$

Sistema de equações algébricas lineares!

Determinação de Fluxos.

Sistema Subdeterminado

Tipicamente, $n > m$ (mais fluxos que metabólitos):

Sistema com infinitas soluções, Espaço nulo de

não – trivial e Requer medições adicionais.

taxas de consumo/produção extracelular, ^{13}C -MFA, razões de fluxo, irreversibilidade e minimizar função objetivo.

— IMP:

^{13}C -MFA usa padrões de marcação para resolver ambiguidades

^{13}C .

C – MFA : Conceito

Metabolic Flux Analysis com ^{13}C

C

Alimentar células com substrato marcado (e.g., $[1, 2^{13}\text{C}]$ glicose), Medir distribuição de marcação em metabólitos via MS ou NMR, Simular propagação de marcação pela rede e Ajustar fluxos para reproduzir padrões experimentais.

Direção de fluxos reversíveis, Fluxos em vias paralelas e Ciclos e recirculação.

Metodologia de ^{13}C .

C – MFA

-

Escolher substrato marcado; cultivar em estado estacionário e Medir taxas extracelulares e padrões de marcação.

-

Definir topologia metabólica e mapear transições atômicas.

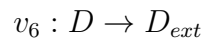
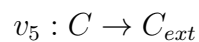
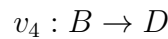
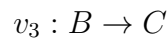
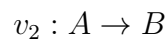
-

Calcular distribuição para fluxos dados e Usar modelos de isotopômeros ou cumômeros.

-

Minimizar desvio entre simulado e experimental e Mínimos quadrados ponderados + análise de incerteza.

Exemplo de MFA: Rede Simples.



$$A : v_1 - v_2 = 0$$

$$B : v_2 - v_3 - v_4 = 0$$

$$C : v_3 - v_5 = 0$$

$$D : v_4 - v_6 = 0$$

$v_1 = 10$ (uptake de A), $v_5 = 6$ (produção de C) e $v_6 = 4$ (produção de D).

$$v_1 = 10$$

$$v_2 = 10$$

$$v_3 = 6$$

$$v_4 = 4$$

$$v_5 = 6$$

$$v_6 = 4$$

Sistema completamente determinado!

Implementação: MFA Básico.

```
1 import numpy as np
```

```

2 from scipy.linalg import lstsq
3
4 # Matriz estequiometrica: metabolitos A,B,C,D x reacoes v1..v6
5 S = np.array([
6     [ 1, -1,  0,  0,  0,  0], # A
7     [ 0,  1, -1, -1,  0,  0], # B
8     [ 0,  0,  1,  0, -1,  0], # C
9     [ 0,  0,  0,  1,  0, -1]  # D
10 ])
11
12 # Medicoes: v1=10, v5=6, v6=4
13 # S*v=0 => S_unmeas*v_unmeas = -S_meas*v_meas
14 S_meas = S[:, [0, 4, 5]]
15 S_unmeas = S[:, [1, 2, 3]]
16 v_meas = np.array([10, 6, 4])
17
18 b = -S_meas @ v_meas
19 v_unmeas, residuals, rank, s = lstsq(S_unmeas, b)
20
21 v_all = np.zeros(6)
22 v_all[[0, 4, 5]] = v_meas
23 v_all[[1, 2, 3]] = v_unmeas
24
25 for i, flux in enumerate(v_all, 1):
26     print(f"v{i}_u={flux:.2f}")

```

Listing 8.3: Resolução de fluxos por balanço de massa

8.4 Análise de Balanço de Fluxo (FBA)

Análise de Balanço de Fluxo (FBA).

Definição (Flux Balance Analysis (FBA)).

Método computacional baseado em otimização linear para prever fluxos metabólicos em redes de escala genômica, sem requerer parâmetros cinéticos.

Características

estequiometria + limites de fluxo, =, tipicamente maximizar biomassa e milhares de reações — computacionalmente eficiente.

— IMP:

FBA permite análise de metabolismo em escala de sistemas sem conhecer todas as constantes cinéticas

Modelos Metabólicos de Escala Genômica (GEMs).

— Genome-scale Metabolic Models —

Reconstruídos a partir de anotação genômica, Integram conhecimento bioquímico e fisiológico e Incluem todas as reações conhecidas de um organismo.

Exemplos de GEMs

iJO1366 (2583 reações, 1805 genes), Yeast8 (4059 reações, 1150 genes) e Recon3D (13543 reações, 3288 genes).

Disponíveis em: BiGG Models ()

Formulação Matemática de FBA.

Problema de Otimização Linear

— DEST:

Maximizar: $c^T v$

Sujeito a: $S \cdot v = 0$

$v_{min} \leq v \leq v_{max}$

onde:

c = coeficientes objetivo;

v = fluxos;

S = estequiométrica;

v_{min}/v_{max} = limites

Função Objetivo Típica

Maximizar crescimento: $c_i = 1$ para reação de biomassa, $c_i = 0$ para outras

Reação de Biomassa.

— Pseudo-reação de Crescimento —

Representa a composição biomolecular da célula:

$$\sum_i a_i \cdot M_i \rightarrow 1 \text{ g biomassa}$$

onde M_i são precursores (aminoácidos, nucleotídeos, lipídios, etc.)

Proteínas:

55 RNA :

20 e DNA:

3

Lipídios:

9 Carboidratos :

3 e Pequenas moléculas:

10

— IMP:

A maximização de biomassa é apoiada por evidências evolutivas e valida bem contra dados experimentais

Restrições e Limites de Fluxo.

Tipos de Restrições**irreversibilidade.****Reações irreversíveis:** $v_i \geq 0$ e **Reações reversíveis:** $v_{min} \leq v_i \leq v_{max}$. $v_i \leq v_{i,max}$. $v_{glucose} = -10 \text{ mmol/gDW/h}$. $v_{ATPM8.39} = 8.39 \text{ mmol/gDW/h}$.**Irreversíveis:** $0 \leq v_i \leq 1000$ e **Reversíveis:** $-1000 \leq v_i \leq 1000$.**Análise de Variabilidade de Fluxo (FVA).****Definição (Flux Variability Analysis).**

Determina o intervalo de valores possíveis para cada fluxo mantendo a função objetivo em seu valor ótimo (ou próximo).

FormulaçãoPara cada reação j :

$$\begin{aligned} \text{Min/Max: } & v_j \\ \text{S.a.: } & \mathbf{S} \cdot \mathbf{v} = 0 \\ & \mathbf{v}_{min} \leq \mathbf{v} \leq \mathbf{v}_{max} \\ & \mathbf{c}^T \mathbf{v} \geq \gamma \cdot Z_{opt} \end{aligned}$$

onde Z_{opt} é o ótimo e $[0, 1]$ (tipicamente 0.9–1.0)**Interpretação**

Intervalo estreito: fluxo bem determinado e Intervalo amplo: múltiplas soluções alternativas.

Predição de Essencialidade Gênica.**Análise de Gene Knockout****Simular deleção de gene removendo reações associadas:**

Definir gene-protein-reaction (GPR) associations, Remover reação(ões) do gene deletado, Resolver FBA com nova rede e Comparar crescimento: μ_{KO}/μ_{WT} .

 $\mu_{KO} < (\text{crescimento nulo/mínimo}), \mu_{KO} \approx \mu_{WT}$ e $0 < \mu_{KO} < \mu_{WT}$.

— IMP:

FBA prediz corretamente

85 – 90

Implementação: FBA com COBRApy.

```
1 import cobra
2 from cobra.io import load_model
```

```

3
4 # Carregar modelo de E. coli
5 model = load_model("iJO1366")
6 print(f"Reacoes: {len(model.reactions)}, Genes: {len(model.genes)}")
7
8 # Definir meio de cultura (glicose aerobica)
9 model.medium = {"EX_glc__D_e": 10.0, "EX_o2_e": 20.0}
10
11 # Resolver FBA
12 solution = model.optimize()
13 print(f"Status: {solution.status}")
14 print(f"Taxa de crescimento: {solution.objective_value:.3f} 1/h")
15
16 # Fluxos principais
17 for rxn_id in ["EX_glc__D_e", "EX_o2_e", "EX_co2_e",
18               "BIOMASS_Ec_iJO1366_core_53p95M"]:
19     print(f"{rxn_id}: {solution.fluxes[rxn_id]:.2f}")

```

Listing 8.4: FBA básico com COBRApy

Implementação: FVA e Gene Knockout.

```

1 from cobra.flux_analysis import flux_variability_analysis
2 from cobra.flux_analysis import single_gene_deletion
3
4 # Flux Variability Analysis
5 fva_result = flux_variability_analysis(model, fraction_of_optimum=0.9)
6
7 # FVA para reacoes centrais
8 for rxn in ["PGI", "PFK", "FBA", "PYK"]:
9     mn = fva_result.loc[rxn, "minimum"]
10    mx = fva_result.loc[rxn, "maximum"]
11    print(f"{rxn}: [{mn:.2f}, {mx:.2f}]")
12
13 # Essencialidade genica
14 deletion_results = single_gene_deletion(model)
15 essential = deletion_results[deletion_results["growth"] < 0.01]
16 print(f"Genes essenciais: {len(essential)}/{len(model.genes)}")
17 for gene in essential.head(5)["ids"]:
18     print(f"{gene}")

```

Listing 8.5: FVA e análise de essencialidade

Implementação: Visualização de Fluxos.

```

1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 fluxes = solution.fluxes
5
6 # Selecionar subsistema (ex: glicolise)
7 glycolysis = model.groups.get_by_id("Glycolysis")
8 glycolysis_rxns = [rxn.id for rxn in glycolysis.members]
9 glycolysis_fluxes = fluxes[glycolysis_rxns]
10
11 fig, ax = plt.subplots(figsize=(10, 6))
12 glycolysis_fluxes.sort_values().plot(kind='barh', ax=ax)
13 ax.set_xlabel('Fluxo (mmol/gDW/h)')
14 ax.set_title('Fluxos na Glicolise')
15
16 # Heatmap em multiplas condicoes
17 conditions = ["aerobic", "anaerobic", "minimal"]
18 flux_matrix = pd.DataFrame()
19 for condition in conditions:
20     sol = model.optimize() # configurar condicao antes
21     flux_matrix[condition] = sol.fluxes[glycolysis_rxns]
22
23 import seaborn as sns
24 plt.figure(figsize=(8, 10))
25 sns.heatmap(flux_matrix, cmap='RdBu_r', center=0)
26 plt.title('Fluxos em Diferentes Condições')

```

Listing 8.6: Visualizar distribuição de fluxos

8.5 Modos Elementares e Vias Extremas

Modos Elementares de Fluxo (EFMs).

Definição (Elementary Flux Modes).

Conjuntos mínimos de reações que podem operar em estado estacionário, representando vias metabólicas funcionalmente independentes.

Propriedades

=, respeitam termodinâmica e remoção de qualquer reação quebra o estado estacionário.

— IMP:

EFMs representam todas as rotas metabólicas possíveis. Qualquer distribuição de fluxo pode ser expressa como combinação linear de EFMs.

Vias Extremas (Extreme Pathways).

Definição (Extreme Pathways).

Subconjunto dos EFMs que forma uma base convexa do espaço de fluxos viáveis. São os vetores geradores do cone de fluxos.

Diferenças: EFMs vs Extreme Pathways

Extreme pathways

Elementarymodes

Cálculo de EFMs.

Algoritmos

Baseado no espaço nulo de

, Construção iterativa e Eficiente para redes grandes.

Desafio Computacional

Número de EFMs cresce exponencialmente com tamanho da rede, Redes pequenas: milhares; redes médias: milhões a bilhões e Genoma completo: computacionalmente intratável.

Limitado a sub-redes (

50 – 100 reações)

Interpretação Biológica de EFMs.

Aplicações

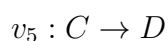
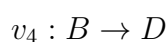
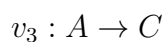
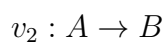
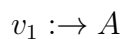
descobrir rotas metabólicas, redundância funcional, alvos de modificação, capacidades metabólicas e comparar estratégias metabólicas.

Em *E. coli*,

500 *EFMs* conectam glicose a biomassa :

ATP: respiração vs fermentação, Poder redutor: PP pathway vs TCA e Precursores biossintéticos.

Exemplo: Rede Simples.



:

$$\begin{bmatrix} 1 & -1 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{EFM}_1 : v_1 = v_2 = v_4 = v_6 = 1, v_3 = v_5 = 0$$

Via: $A \rightarrow B \rightarrow D$

$$\mathbf{EFM}_2 : v_1 = v_3 = v_5 = v_6 = 1, v_2 = v_4 = 0$$

Via: $A \rightarrow C \rightarrow D$

Duas rotas independentes de A para D

Implementação: Cálculo de EFMs (Conceitual).

```

1 import numpy as np
2 from scipy.linalg import null_space
3
4 # Matriz estequiometrica
5 S = np.array([
6     [ 1, -1, -1,  0,  0,  0], # A
7     [ 0,  1,  0, -1,  0,  0], # B
8     [ 0,  0,  1,  0, -1,  0], # C
9     [ 0,  0,  0,  1,  1, -1]  # D
10 ])
11
12 # Espaco nulo (base de solucoes)
13 null_S = null_space(S)
14 print("Espaco_nulo_de_S:", null_S)
```

```

15
16 # EFMs: combinacoes que respeitam v >= 0
17 # Para calculo real usar:
18 # - efmtool (Java): csb.ethz.ch/tools/efmtool
19 # - CellNetAnalyzer (MATLAB), efmtool-py
20
21 # from efmtool import calculate_efms
22 # efms = calculate_efms(S, reversibilities)
23 # for i, efm in enumerate(efms):
24 #     print(f"EFM {i+1}: {efm}")

```

Listing 8.7: EFMs com efmtool (interface conceitual)

8.6 Metabolômica

O que é Metabolômica?.

Definição (Metabolômica).

Estudo abrangente do conjunto completo de metabólitos (metaboloma) em uma amostra biológica em um dado momento.

Snapshot do estado metabólico, Reflete fenótipo bioquímico, Integra genoma, transcriptoma, proteoma e Sensível a perturbações ambientais. essenciais (glicose, ATP), não-essenciais, externos (drogas) e Faixa:

100Daa

1500 Da.

— IMP:

Metaboloma humano:

114,000metabólitosconhecidos(HMDB),

2,500 endógenos detectáveis

Plataformas Analíticas.

— LC-MS: Liquid Chromatography-Mass Spectrometry —

separação por cromatografia + detecção por massa, alta sensibilidade, ampla cobertura, metabólitos polares e apolares e HILIC, RP, tandem MS (MS/MS).

— GC-MS: Gas Chromatography-Mass Spectrometry —

volatilização + separação + MS, reprodutibilidade, bibliotecas extensas, requer derivatização e metabólitos voláteis e derivatizáveis.

NMR: Nuclear Magnetic Resonance

ressonância magnética nuclear, não-destrutivo, quantitativo, identificação estrutural e menor sensibilidade.

Targeted Metabolomics.

Targeted

conjunto predefinido de metabólitos

Alta precisão e acurácia, Quantificação absoluta e Validação com padrões.

Cobertura limitada e Viés de hipótese.

Validação de biomarcadores e Monitoramento de vias.

Foco: conjunto predefinido de metabólitos

Untargeted Metabolomics.

Untargeted

detecção abrangente

Descoberta sem viés, Ampla cobertura e Geração de hipóteses.

Quantificação relativa, Desafios de identificação e Grande volume de dados.

Estudos exploratórios e Descoberta de biomarcadores.

Foco: detecção abrangente

Pipeline de Análise Metabolômica.

-

Quenching metabólico (parar reações), Extração de metabólitos e Normalização (por massa, proteína, etc.).

-

Análise por LC-MS, GC-MS ou NMR e Controles de qualidade (QC).

-

Detecção de picos, Alinhamento de tempos de retenção e Normalização e correção de batch.

-

Matching com bibliotecas (m/z, RT) e MS/MS para confirmação estrutural.

-

Identificação de Metabólitos.

Níveis de Confiança (MSI Standards)

Identificado - padrão autêntico + 2 propriedades ortogonais, Putativo - match com biblioteca espectral, Classe putativa - características químicas e Desconhecido - m/z sem identificação.

Bancos de Dados

Human Metabolome Database (), biblioteca de MS/MS (), metabólitos em vias (), Chemical Entities of Biological Interest e NIH database.

Integração Multi-Omics.

Mapear níveis de RNA, proteína e metabólito em vias, Modelos de interação gene-metabolismo, contextualizar medições metabolômicas, predição de fenótipo multi-omics e identificar módulos regulatórios.

Implementação: Análise de Dados Metabolômicos.

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.decomposition import PCA
5
6 # Carregar dados (linhas=amostras, colunas=metabolitos)
7 data = pd.read_csv("metabolomics_data.csv", index_col=0)
8 metadata = pd.read_csv("sample_metadata.csv", index_col=0)
9 print(f"Amostras: {data.shape[0]}, Metabolitos: {data.shape[1]}")
10
11 # Pre-processamento
12 threshold = 0.2
13 data_clean = data.dropna(thresh=len(data)*(1-threshold), axis=1)
14 data_imputed = data_clean.fillna(data_clean.median())
15 data_log = np.log2(data_imputed + 1)
16
17 # Normalizacao Z-score
18 scaler = StandardScaler()
19 data_scaled = pd.DataFrame(
20     scaler.fit_transform(data_log),
21     index=data_log.index, columns=data_log.columns)
22 print("Dados processados!")

```

Listing 8.8: Processamento básico de dados

Implementação: PCA e Visualização.

```

1 # PCA
2 pca = PCA(n_components=2)
3 pca_result = pca.fit_transform(data_scaled)
4 print(f"PC1: {pca.explained_variance_ratio_[0]*100:.1f}%")
5 print(f"PC2: {pca.explained_variance_ratio_[1]*100:.1f}%")
6
7 # Plot PCA
8 fig, ax = plt.subplots(figsize=(10, 8))
9 groups = metadata["group"].unique()
10 colors = plt.cm.Set1(range(len(groups)))
11 for group, color in zip(groups, colors):
12     mask = metadata["group"] == group
13     ax.scatter(pca_result[mask, 0], pca_result[mask, 1],
14               c=[color], label=group, s=100, alpha=0.7)
15 ax.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]*100:.1f}%)')
16 ax.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]*100:.1f}%)')
17 ax.set_title('PCA Score Plot')
18 ax.legend()
19
20 # Top metabolitos por loading
21 loadings = pd.DataFrame(pca.components_.T,
22     columns=['PC1', 'PC2'], index=data_scaled.columns)
23 print(loadings['PC1'].abs().nlargest(10))

```

Listing 8.9: Análise de componentes principais

8.7 Exercícios

Exercícios - Parte 1.

Questão 1: Cinética Enzimática

Uma enzima tem $K_M = 5 \text{ mM}$ e $V_{max} = 100$

mol/min.

Calcule a velocidade quando $[S] = 10 \text{ mM}$, Qual $[S]$ dá $v = 75$ e Se $[E]$ dobrar, como mudam K_M e V_{max} ?

Questão 2: Balanço de Massa

Para a rede ABC com BD :

Escreva a matriz estequiométrica

, Equações de balanço em estado estacionário e Se

$v_{AB} = 10$ e $v_{BC} = 6$, calcule v_{BD} .

Exercícios - Parte 2.

Questão 3: FBA

Dado um modelo metabólico simples:

Biomassa requer 40 ATP, Respiração: 38 ATP/glc; Fermentação: 2 ATP/glc e Uptake de glicose

10 mmol/gDW/h.

Formule o problema de FBA, Crescimento máximo só com respiração?, E só com fermentação? e Como o modelo escolheria em anaerobiose?.

Exercícios - Parte 3.

Projeto: Análise de GEM com COBRApy

Escolha um organismo modelo (E. coli, S. cerevisiae, etc.):

Carregar GEM com COBRApy, Simular crescimento em meio rico vs mínimo e Análise de gene knockout:.

Identificar genes essenciais e Duplos knockouts sintéticos letais.

FVA: identificar reações flexíveis e Maximizar produção de metabólito mantendo crescimento mínimo.

8.8 Notas Bibliográficas

Referências Principais - Livros.

99

Palsson B.O. (2015). Systems Biology: Constraint-based Reconstruction and Analysis. Cambridge University Press.

Fell D.A. (1997). *Understanding the Control of Metabolism*. Portland Press.

Nielsen J., Jewett M.C. (2008). *Metabolic Engineering*. Springer.

Voit E.O. (2013). *Biochemical Systems Theory: A Review*. ISRN Biomathematics.

Stephanopoulos G., Aristidou A.A., Nielsen J. (1998). *Metabolic Engineering: Principles and Methodologies*. Academic Press.

Referências Principais - Artigos.

99

Orth J.D., Thiele I., Palsson B.O. (2010). What is flux balance analysis? *Nature Biotechnology*, 28(3):245-248.

Wiechert W. (2001). ^{13}C Metabolic Flux Analysis. *Metabolic Engineering*, 3(3):195-206.

Schuster S. et al. (1999). Detection of elementary flux modes in biochemical networks. *Bioinformatics*, 15(3):251-257.

Patti G.J., Yanes O., Siuzdak G. (2012). Metabolomics: the apogee of the omics trilogy. *Nature Reviews Molecular Cell Biology*, 13:263-269.

Ebrahim A., Lerman J.A., Palsson B.O., Hyduke D.R. (2013). CO-BRApy: COstraints-Based Reconstruction and Analysis for Python. *BMC Systems Biology*, 7:74.

Ferramentas e Recursos.

Software

-
-
-
-
-

Bancos de Dados

-
-
-
-
-

Leitura Complementar.

Tutoriais e Reviews

Bordbar A. et al. (2014). Constraint-based models predict metabolic and associated cellular functions. *Nature Reviews Genetics*.

Antoniewicz M.R. (2015). Methods and advances in metabolic flux analysis: a mini-review. *Journal of Industrial Microbiology & Biotechnology*.

Lewis N.E. et al. (2012). Constraining the metabolic genotype-phenotype relationship using a phylogeny of in silico methods. *Nature Reviews Microbiology*.

Zamboni N. et al. (2015). ^{13}C metabolic flux analysis in complex systems. *Current Opinion in Biotechnology*.

Cursos Online

Systems Biology (Icahn School of Medicine) e Introduction to Computational Biology.

-

Obrigado!

Capítulo 8: Sistemas Metabólicos

Redes de Regulação Gênica

Dúvidas? Perguntas?

Capítulo 9

Sistemas de Sinalização Celular

9.1 Introdução aos Sistemas de Sinalização.

Objetivos de Aprendizagem

Compreender os princípios básicos da sinalização celular, Conhecer os principais tipos de receptores e vias de sinalização, Entender a dinâmica e propriedades emergentes de redes de sinalização, Modelar matematicamente cascatas de sinalização e Simular sistemas de sinalização usando Python.

— IMP:

A sinalização celular é essencial para:

Coordenação entre células, Resposta a estímulos ambientais, Diferenciação e desenvolvimento e Homeostase e sobrevivência.

Papel da Sinalização Celular.

Definição (Sinalização Celular).

Processo pelo qual células detectam, transmitem e respondem a sinais químicos ou físicos, permitindo comunicação intra e intercelular.

Exemplos de Respostas Celulares

Alteração na expressão gênica, Modificação do metabolismo, Reorganização do citoesqueleto e Proliferação, diferenciação ou apoptose.

Tipos de Sinalização.

célula responde ao próprio sinal, sinal atua em células vizinhas (curta distância), hormônios transportados pelo sangue (longa distância) e contato direto entre células.

Exemplos

fatores de crescimento em câncer, neurotransmissores na sinapse, insulina, hormônios tireoidianos e sinalização Notch-Delta.

9.1 Receptores e Mecanismos de Ativação

9.2 Receptores Celulares: Visão Geral.

Classificação de Receptores

-
-
-
-
-

Receptores Acoplados à Proteína G (GPCRs).

7 domínios transmembrana, Maior família de receptores (

800 em humanos), Acoplados a proteínas G heterotriméricas. Alvos de

50

estimula adenilil ciclase

cAMP, inibe adenilil ciclase

cAMP, ativa fosfolipase C

IP

IP_3/DAG e : regulam o esqueleto.

Exemplos

Receptores adrenérgicos, Receptores muscarínicos, Receptores de olfato e Rodopsina (visão).

Mecanismo de Ativação de GPCRs.

— IMP:

G

$-GTP \rightarrow GDP + P_i$

ativam efetores downstream independentemente!

Receptores Tirosina Quinase (RTKs).

Domínio extracelular (ligação), Domínio transmembrana único, Domínio intracelular (quinase), Ativados por dimerização e Autofosforilação em tirosinas.

EGF receptor, PDGF receptor, FGF receptor, Insulin receptor e VEGF receptor.

Mutações/superexpressão de EGFR estão associadas a diversos cânceres. Inibidores de tirosina quinase (TKIs) como gefitinib são usados em

terapia.

Receptores de Canais Iônicos e Nucleares.

— Canais Iônicos Ligante-Dependentes —

Resposta ultrarrápida (ms), Alteração do potencial de membrana e Influxo/efluxo de íons.

Receptor nicotínico (ACh), Receptor GABA_A, Receptor glutamato (NMDA) e Receptor 5-HT₃ (serotonina).

Receptores Nucleares

Fatores de transcrição ativados por ligantes, Ligantes hidrofóbicos (atravessam membrana) e Resposta lenta (horas).

Receptores de esteroides (GR, ER, AR), Receptores de hormônios tireoidianos, Receptores de retinoides (RAR) e PPAR (metabolismo lipídico).

Mecanismo de Receptores Nucleares

Hormônio

Atravessamembrana

Liga ao receptor no citoplasma/núcleo

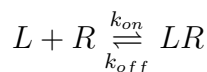
Dimerização

Liga ao DNA (HRE)

Ativa/reprimetranscrição

Ligação Ligante-Receptor: Cinética.

Modelo de Equilíbrio



$$K_d = \frac{k_{off}}{k_{on}} = \frac{[L][R]}{[LR]}$$

$$\theta = \frac{[LR]}{[R]_{total}} = \frac{[L]}{K_d + [L]}$$

Interpretação de K_d

K_d baixo

alta afinidade, Quando

$[L] = K_d, = 0.5$ e K_d típicos: nM -

$M.$

Capacidade Máxima

$$B_{max} = [R]_{total}$$

Número máximo de sítios de ligação disponíveis

Modelagem de Ligação Receptor-Ligante.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def receptor_binding(L, Kd, Bmax):
5     """Calcula ligacao pelo modelo de equilibrio"""
6     return (Bmax * L) / (Kd + L)
7
8 # Parametros
9 Kd = 10 # nM
10 Bmax = 100 # unidades arbitrias
11
12 # Concentracoes de ligante
13 L = np.logspace(-1, 3, 100) # 0.1 - 1000 nM
14 bound = receptor_binding(L, Kd, Bmax)
15
16 # Plotar
17 plt.figure(figsize=(10, 6))
18 plt.semilogx(L, bound, 'b-', linewidth=2)
19 plt.axhline(y=Bmax/2, color='r', linestyle='--',
20             label=f'50% ocupacao')
21 plt.axvline(x=Kd, color='r', linestyle='--',
22             label=f'Kd={Kd} nM')
23 plt.xlabel('[Ligante] (nM)')
24 plt.ylabel('Receptores Ocupados')
25 plt.title('Curva de Ligacao Ligante-Receptor')
26 plt.legend()
27 plt.grid(True, alpha=0.3)
```

Listing 9.1: Curva de ligação

Cinética de Ativação de Receptores.

Modelo com Estado Ativo/Inativo

$$\begin{aligned}\frac{d[R]}{dt} &= -k_{on}[L][R] + k_{off}[LR] + k_{inact}[R^*] \\ \frac{d[LR]}{dt} &= k_{on}[L][R] - k_{off}[LR] - k_{act}[LR] \\ \frac{d[R^*]}{dt} &= k_{act}[LR] - k_{inact}[R^*]\end{aligned}$$

Onde R^* é o receptor ativo que inicia a sinalização.

Estado Estacionário

No equilíbrio ($dt = 0$):

$$[R^*]_{ss} = \frac{k_{act} \cdot k_{on}[L][R]_{total}}{k_{inact}(k_{off} + k_{act}) + k_{act} \cdot k_{on}[L]}$$

9.2 Redes de Transdução de Sinal

9.3 Redes de Transdução: Visão Geral.

Principais Vias de Sinalização

proliferação, diferenciação, sobrevivência, metabolismo, resposta a citocinas, desenvolvimento, polaridade celular, decisões de destino celular, crescimento, diferenciação, inflamação, imunidade e padronização no desenvolvimento.

— IMP:

Estas vias não operam isoladamente! Existe extenso (diálogo cruzado) entre elas.

Via MAPK/ERK: Cascata Canônica.

Via PI3K/Akt: Sobrevivência e Metabolismo.
síntese proteica

, glicogêniosíntese

, apoptose

, gliconeogênese

e p53

Relevância Clínica

supressor tumoral frequentemente mutado, hiperativa em

30 *resistência à insulina via Akt.*

Via JAK-STAT: Sinalização de Citocinas.

— IMP:

Via rápida e direta: do receptor à transcrição gênica sem cascatas intermediárias complexas!

Via Wnt: Desenvolvimento e Polaridade.

-

-catenina fosforilada por complexo de destruição (GSK3, APC, Axina) e Ubiquitinação e degradação.

-

Wnt liga a Frizzled + LRP5/6, Dishevelled (Dvl) ativado, Complexo de destruição inibido,

-catenina acumula, Entra no núcleo e Ativa transcrição (TCF/LEF).

Mutações em APC (via Wnt) são encontradas em >80

Via Notch: Decisões de Destino Celular.

0

— IMP:

Notch é único: o receptor é clivado e o fragmento intracelular vai diretamente ao núcleo!

Via TGF-

/Smad.

1

(Smad6/7): inibem R-Smads (feedback negativo), degradação de R-Smads e desfosforilam Smads.

Diferenciação celular, Apoptose, Transição epitélio-mesênquima (EMT), Fibrose e Imunossupressão.

Dual Role

TGF-

age como estágios iniciais, mas promove estágios avançados.

Crosstalk entre Vias de Sinalização.

2

— IMP:

O crosstalk permite:

Integração de múltiplos sinais, Ajuste fino da resposta celular, Comportamentos complexos (biestabilidade, oscilações) e Robustez e redundância.

9.3 Cascatas de Proteínas Quinases

9.4 Cascatas de Fosforilação.

Definição (Cascata de Quinases).

Sequência de reações onde uma quinase ativa outra por fosforilação, resultando em amplificação de sinal.

3

— IMP:

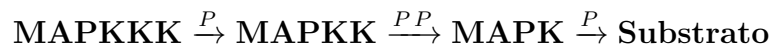
Cascatas proporcionam , e de sinais!

Cascata MAPK: MAPKKK

MAPKK

MAPK.

Estrutura da Cascata



Raf, MEKK, MLK, TAK1, MEK1/2, MKK3/6, MKK4/7 e ERK1/2, p38, JNK.

Dupla Fosforilação de MAPK

MAPK requer fosforilação em resíduos (Thr e Tyr no motif TXY) para ativação completa:



Isso confere !

Ultrassensibilidade: Função de Goldbeter-Koshland.

Modelo de GK

Para um ciclo de modificação (fosforilação/desfosforilação):

$$\frac{[X^*]}{[X]_{total}} = \frac{2vq}{v + q - v + q - (v - q)^2 + 4vq}$$

Onde:

$v = K_1 + [S]$ (quinase) e $q = K_2$ (fosfatase).

Quando $K_1, K_2[X]_{total}$: comportamento

$$n_H \approx 1 + \frac{2[X]_{total}}{K_1 + K_2}$$

Interpretação

$n_H = 1$: resposta hiperbólica (Michaelis-Menten), $n_H > 1$: ultrasensível (switch-like) e Cascatas de MAPK: n_H pode chegar a 5-10!.

— IMP:

Ultrasensibilidade permite decisões binárias (ON/OFF) mesmo com sinais graduais!

Amplificação de Sinal em Cascatas.

Modelo de Amplificação

Para uma cascata de n níveis, cada nível com ganho g_i :

$$\text{Amplificação Total} = \prod_{i=1}^n g_i = g_1 \times g_2 \times \cdots \times g_n$$

$$g_i = \frac{k_{cat,i}}{k_{inact,i}} \times \frac{[K_i]_{total}}{[K_{i-1}]_{ativo}}$$

Raf: 10 moléculas ativas, MEK: 100 moléculas ativas (ganho = 10), ERK: 1000 moléculas ativas (ganho = 10) e $10 \times 10 = 100$.

Fosforilação Distributiva vs Processiva.

Distributiva

Substrato se dissocia após cada fosforilação:

4

Mais comum, Gera e Coeficiente de Hill alto.

Processiva

Substrato permanece ligado:

5

Menos comum, Resposta mais gradual e Coeficiente de Hill 1.

MAPK é Distributiva

MEK fosforila ERK de forma distributiva

ultrasensibilidade!

Biestabilidade em Cascatas de Sinalização.

Definição (Biestabilidade).

Sistema com dois estados estáveis (ON e OFF) separados por ponto de instabilidade.

Ultrasensibilidade ($n_H > 1$), Feedback positivo e Saturação de fosfatases.

Memória celular, Decisões irreversíveis, Histerese e Resistência a ruído.

6

Histerese

Threshold para ativar

thresholdparadesativar!

Simulação de Cascata MAPK.

```

1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def mapk_cascade(y, t, stimulus):
6     """Modelo de cascata MAPK de 3 niveis"""
7     K1, K1p, K2, K2p, K3, K3p = y
8
9     # Parametros
10    k1 = 0.1 * stimulus # ativacao de K1
11    k2 = 0.5             # K1p ativa K2
12    k3 = 1.0             # K2p ativa K3
13    d1 = 0.1             # desfosforilacao
14    d2 = 0.2
15    d3 = 0.3
16
17    # ODEs
18    dK1 = -k1*K1 + d1*K1p
19    dK1p = k1*K1 - d1*K1p
20
21    dK2 = -k2*K1p*K2 + d2*K2p
22    dK2p = k2*K1p*K2 - d2*K2p
23
24    dK3 = -k3*K2p*K3 + d3*K3p
25    dK3p = k3*K2p*K3 - d3*K3p
26
27    return [dK1, dK1p, dK2, dK2p, dK3, dK3p]
28
29 # Condições iniciais (total = 100 para cada nivel)
30 y0 = [100, 0, 100, 0, 100, 0]
31 t = np.linspace(0, 50, 500)
32 stimulus = 1.0
33
34 sol = odeint(mapk_cascade, y0, t, args=(stimulus,))

```

Listing 9.2: Modelo simplificado de cascata MAPK

Simulação de Cascata MAPK (cont.).

```

1 # Plotar resultados
2 plt.figure(figsize=(12, 6))
3
4 plt.subplot(1, 2, 1)
5 plt.plot(t, sol[:, 1], 'b-', linewidth=2, label='K1*(MAPKKK)')
6 plt.plot(t, sol[:, 3], 'g-', linewidth=2, label='K2*(MAPKK)')
7 plt.plot(t, sol[:, 5], 'r-', linewidth=2, label='K3*(MAPK)')
8 plt.xlabel('Tempo')
9 plt.ylabel('Concentracao_(forma_ativa)')
10 plt.title('Dinamica_Temporal_da_Cascata_MAPK')
11 plt.legend()

```

```

12 plt.grid(True, alpha=0.3)
13
14 # Dose-response
15 plt.subplot(1, 2, 2)
16 stimuli = np.logspace(-2, 1, 20)
17 responses = []
18
19 for stim in stimuli:
20     sol = odeint(mapk_cascade, y0, t, args=(stim,))
21     responses.append(sol[-1, 5]) # K3p no equilibrio
22
23 plt.semilogx(stimuli, responses, 'ro-', linewidth=2)
24 plt.xlabel('Estimulo (log scale)')
25 plt.ylabel('MAPK ativa (equilibrio)')
26 plt.title('Curva Dose-Resposta (Ultrassensibilidade)')
27 plt.grid(True, alpha=0.3)
28 plt.tight_layout()

```

Listing 9.3: Visualização da cascata

9.4 Segundos Mensageiros

9.5 Segundos Mensageiros.

Definição (Segundo Mensageiro).

Molécula intracelular pequena que propaga o sinal de um receptor para efetores intracelulares, amplificando e difundindo o sinal.

Principais Segundos Mensageiros

(AMP cíclico), (GMP cíclico), $^{2+}$ (cálcio), $_3$ (inositol 1,4,5-trifosfato), (diacilglicerol) e (óxido nítrico).

— IMP:

Segundos mensageiros permitem:

1 receptor

milhares de moléculas de 2º mensageiro, sinal se espalha pelo citoplasma em múltiplas sinais convergem.

cAMP: Sinalização via Adenilil Ciclase.

GPCR ativa G_s -

, G

s^-

$-GTP$ ativa adenilil ciclase (AC), AC converte ATP

cAMP, cAMP ativa PKA (proteína quinase A), PKA fosforila alvos

downstream e Fosfodiesterase (PDE) degrada cAMP

AMP.

CREB (transcrição), Enzimas metabólicas e Canais iônicos.

7

Exemplos

Epinefrina

glicogenólise, ACTH

síntese de cortisol e Glucagon

liberação de glicose.

Sinalização por Ca^{2+} .

: *Oscilações e Ondas*

:

retículo endoplasmático (ER) e através de canais.

$[\text{Ca}^{2+}]_{\text{cito, repouso}}$

$100 \text{ nM}, [\text{Ca}$

$^{2+}]_E R$

$0.5 \text{ mM} e [\text{Ca}$

$^{2+}]_e \text{ extra}$

$1 - 2 \text{ mM}.$

:

-

(músculo) e (proteína quinase C).

-

8

Padrões de Ca^{2+}

pico único, picos periódicos e propagação espacial.

Frequência e amplitude codificam informação!

Via IP_3/DAG : Fosfolipase C.

9

— IMP:

IP_3 e DAG atuam sinergisticamente: IP_3 libera Ca^{2+} , $\text{Ca}^{2+} + \text{DAG}$ ativam PKC!

Modelo Matemático de Oscilações de Ca^{2+} .

Modelo Simplificado

$$\frac{d[\text{Ca}^{2+}]}{dt} = v_{in} - v_{out} + v_{release} - v_{uptake}$$

$$v_{release} = k_{rel} \cdot [\text{IP}_3] \cdot \frac{[\text{Ca}^{2+}]^2}{K_1^2 + [\text{Ca}^{2+}]^2}$$

$$v_{uptake} = k_{uptake} \cdot [\text{Ca}^{2+}]$$

Ca^{2+} estimula sua própria liberação (feedback positivo), Combinado com uptake (feedback negativo) e Resultado: oscilações!.

Codificação por Frequência (Frequency Encoding)

A das oscilações de Ca^{2+} codifica a intensidade do estímulo:

Estímulo fraco

baixa frequência e Estímulo forte

alta frequência.

Simulação de Oscilações de Ca^{2+} .

```

1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def ca_oscillations(y, t, IP3):
6     """Modelo simplificado de oscilacoes de Ca2+"""
7     Ca = y[0]
8
9     # Parametros
10    k_rel = 2.0      # liberacao do ER
11    k_uptake = 0.5    # recaptura para o ER
12    K1 = 0.3         # constante para CICR
13    v_in = 0.1       # influxo externo
14    v_out = 0.05     # efluxo
15
16    # Liberacao induzida por IP3 e Ca (CICR)
17    v_release = k_rel * IP3 * (Ca**2 / (K1**2 + Ca**2))
18
19    # Uptake pelo ER (SERCA pumps)
20    v_uptake_rate = k_uptake * Ca
21
22    # ODE
23    dCa = v_in - v_out + v_release - v_uptake_rate
24
25    return [dCa]
26
27 # Simular para diferentes niveis de IP3
28 t = np.linspace(0, 100, 1000)

```



```

29 IP3_levels = [0.5, 1.0, 1.5]
30
31 plt.figure(figsize=(12, 4))
32 for i, IP3 in enumerate(IP3_levels):
33     Ca0 = [0.1]
34     sol = odeint(ca_oscillations, Ca0, t, args=(IP3,))
35     plt.subplot(1, 3, i+1)
36     plt.plot(t, sol[:, 0], 'b-', linewidth=2)
37     plt.xlabel('Tempo')
38     plt.ylabel('Ca2+')
39     plt.title(f'IP3={IP3}')
40     plt.grid(True, alpha=0.3)

```

Listing 9.4: Modelo de oscilações de cálcio

cGMP e Sinalização por NO.

NO (óxido nítrico) produzido por NOS (NO sintase), NO difunde através de membranas, NO ativa guanilil ciclase solúvel (sGC), sGC converte GTP

cGMP, cGMP ativa PKG (proteína quinase G), PKG fosforila a lvas e Fosfodiesterase degrada cGMP

NO é gasoso, lipofílico, Meia-vida curta (

segundos) e Sinalização parácrina.

0

Aplicação Clínica

inibe fosfodiesterase que degrada cGMP

prolonga e faz de vasos dilatador!

9.5 Dinâmica e Propriedades Emergentes

9.6 Dinâmica Temporal de Sinalização.

Padrões Temporais

resposta rápida que retorna ao basal, resposta mantida enquanto estímulo presente, resposta diminui apesar de estímulo constante e ciclos periódicos de ativação/desativação.

1

Feedback Loops: Positivo e Negativo.

Feedback Negativo

2

Estabilidade, Homeostase, Adaptação e Reduz variabilidade.

Inibição por produto, Desensibilização de receptores e I-Smads na via TGF-

Feedback Positivo

3

Amplificação, Biestabilidade, Memória e Decisões irreversíveis.

CICR (Ca^{2+}), Ativação de caspases (apoptose) e Ciclo celular (check-points).

— IMP:

Combinações de feedbacks positivo e negativo geram oscilações!

Gradientes Espaciais e Morfógenos.

Definição (Morfógeno).

Molécula sinalizadora que forma gradiente de concentração e especifica diferentes destinos celulares dependendo da concentração local.

4

padrão ântero-posterior em *Drosophila*, tubo neural e membros e padrão dorso-ventral.

Oscilações em Sistemas de Sinalização.

Feedback negativo com atraso, Não-linearidade (Hill, saturação) e Separação de escalas temporais.

oscilações em resposta a TNF-

, pulso em resposta a dano ao DNA,

²⁺: oscilações citosólicas, pulsos vs sustentado e oscilações no relógio de segmentação.

Caso: p53

5

p53 ativa Mdm2

Mdm2 degrada p53

oscilações!

Pulsos de p53 permitem reparo do DNA, p53 sustentado

apoptose e Número de pulsos codifica severidade do dano.

Adaptação e Desensibilização.

Mecanismos de Adaptação

reduz afinidade/atividade, endocitose mediada, feedback negativo e exaustão de estoques.

GPCR ativo é fosforilado por (GPCR kinases), liga ao receptor fosforilado, Bloqueia acoplamento com proteína G, Promove internalização via endocitose e Receptor pode ser reciclado ou degradado.

— IMP:

Adaptação permite que células detectem no estímulo, não apenas intensidade absoluta!

Robustez e Sensibilidade.

Robustez

Capacidade do sistema manter função apesar de perturbações.

Feedback negativo, Redundância (vias paralelas), Buffering (sequestro) e Compartimentalização.

Exemplo

Via MAPK é robusta a variações em concentração de componentes devido a feedbacks múltiplos.

Sensibilidade

Capacidade de responder a pequenas mudanças no estímulo.

Amplificação (cascatas), Ultrassensibilidade (Hill), Feedback positivo e Cooperatividade.

Exemplo

Cascata MAPK amplifica sinal >100

exibe ultrassensibilidade (Hill)

n 5).

— IMP:

Sistemas de sinalização equilibram (resistência a ruído) com (detecção de sinais)!

9.6 Exercícios

Exercícios - Parte 1.

— Questão 1: Ligação Receptor-Ligante —

Um receptor tem $K_d = 5 \text{ nM}$ e $B_{max} = 1000 \text{ receptores/célula}$.

Calcule a ocupação quando $[\text{Ligante}] = 5 \text{ nM}$, Que concentração de ligante resulta em 90 e Se $k_{on} = 10^6 \text{ M}^{-1}\text{s}^{-1}$, qual é k_{off} ?

Questão 2: Amplificação em Cascata

Uma cascata MAPK tem 3 níveis. Cada quinase ativada fosforila 10 moléculas do próximo nível. Se 5 moléculas de MAPKKK são ativadas:

Quantas moléculas de MAPKK são ativadas?, Quantas moléculas de MAPK são ativadas? e Qual é o fator de amplificação total?.

Exercícios - Parte 2.

Questão 3: Ultrassensibilidade

Uma quinase exibe comportamento ultrassensível com coeficiente de Hill $n_H = 4$ e $K_{0.5} = 10$

M .

Escreva a equação de Hill para a ativação, Calcule a fração ativa quando $[S] = 5, 10, 15$

MeComparecomMichaelis – Menten(

$n_H = 1)$.

Questão 4: Segundos Mensageiros

cAMP é produzido a $v_{synth} = 0.5$

M/sedegradadocomconstante

$k_{deg} = 0.1 \text{ s}^{-1}$.

Escreva a equação diferencial para [cAMP], Calcule a concentração no estado estacionário e Quanto tempo leva para atingir 90

Exercícios - Parte 3.

Questão 5: Simulação de Cascata MAPK

Use Python para simular a cascata MAPK:

Implemente o modelo de 3 níveis (MAPKKK

$MAPKK$

MAPK), Inclua fosforilação e desfosforilação, Simule a dinâmica temporal para um estímulo em degrau, Gere curva dose-resposta variando intensidade do estímulo e Calcule o coeficiente de Hill efetivo a partir da curva dose-resposta.

Questão 6: Oscilações de Ca^{2+}

Modifique o código de oscilações de Ca^{2+} para:

Incluir feedback negativo via inativação do receptor IP_3 , Simular ondas de Ca^{2+} (1D ou 2D com difusão) e Investigar como parâmetros afetam frequência de oscilação.

Exercícios - Parte 4.

Projeto Prático: Modelagem de Via Completa

Escolha uma via de sinalização (MAPK, PI3K/Akt, JAK-STAT) e:

Desenhe diagrama completo da via com todas as interações, Identifique pontos de regulação (feedbacks, crosstalk), Construa modelo matemático (ODEs) com 5-10 componentes, Implemente o modelo em Python usando `scipy.integrate.odeint`, Simule perturbações (knockout de componente, inibidor farmacológico), Analise sensibilidade dos parâmetros, Compare previsões do modelo com dados experimentais da literatura e Gere visualizações: dinâmica temporal, espaço de fases, bifurcações.

9.7 Notas Bibliográficas

Referências Principais.

99

Alberts B. et al. (2022). *Molecular Biology of the Cell*, 7th ed. Garland Science.

Lodish H. et al. (2021). *Molecular Cell Biology*, 9th ed. W.H. Freeman.

Alon U. (2019). *An Introduction to Systems Biology: Design Principles of Biological Circuits*, 2nd ed. CRC Press.

Kholodenko B.N. (2006). Cell-signalling dynamics in time and space. *Nature Reviews Molecular Cell Biology* 7:165-176.

Ferrell J.E. (1996). Tripping the switch fantastic: how a protein kinase cascade can convert graded inputs into switch-like outputs. *Trends in Biochemical Sciences* 21:460-466.

Leitura Complementar.

Livros

Tyson J.J., Novak B. (2020). *The Cell Cycle: Principles of Control*. Oxford,

Klipp E. et al. (2016). *Systems Biology: A Textbook*, 2nd ed e

Fall C.P. et al. (2002). *Computational Cell Biology*. Springer.

Artigos Clássicos

Goldbeter A., Koshland D.E. (1981). Ultrasensitivity in biochemical systems. *PNAS*, Bhalla U.S., Iyengar R. (1999). Emergent properties of networks of signaling pathways. *Science* e Dolmetsch R.E. et al. (1998). Calcium oscillations increase efficiency of gene expression. *Nature*.

Recursos Online

-
-

-
-

Analogia-âncora: a cascata como amplificador de show.

Ligar o receptor é sussurrar no microfone

Imagine que cada célula é um e o ligante extracelular é um do técnico de som.

A cascata MAPK (Raf

$\rightarrow$ MEK

$\rightarrow$ ERK) é a :

cada camada fosforila a seguinte em 1000 cópias, transformando o sussurro inicial em um sinal nuclear suficiente para disparar transcrição gênica.

Por que isso importa?

A célula ganha (liga/desliga) + + . Sem amplificação, a resposta seria indistinguível de ruído — o mesmo problema do show sem microfone.

Recapitulação — Capítulo 9: Signaling Systems.

Ao final deste capítulo, você deve saber:

RTK, GPCR, ion channel — três famílias, uma lógica, MAPK, PI3K/Akt, JAK-STAT — amplificação + switch, cAMP, Ca^{2+} , IP_3 /DAG — oscilações como código (Dolmetsch 1998), Goldbeter-Koshland (1981) — cooperatividade gera sigmóide e módulos bifásicos (Bhalla-Iyengar 1999) — comportamento global > soma de partes.

como cascatas de sinalização viram controle alostérico de enzimas metabólicas.

Capítulo 10

Sistemas Populacionais e Epidemiologia

10.1 Introdução

Visão Geral do Capítulo.

Objetivos de Aprendizagem

Compreender modelos de dinâmica populacional, Analisar interações entre espécies, Estudar modelos epidemiológicos, Explorar dinâmica espacial de populações e Implementar simulações em Python.

Por que Estudar Sistemas Populacionais?

Conservação de espécies ameaçadas, Controle de epidemias, Gestão de recursos naturais e Compreensão de ecossistemas.

Dinâmica Populacional.

Definição (Sistema Populacional).

Conjunto de indivíduos da mesma espécie (ou espécies interagentes) cuja densidade varia no tempo e espaço devido a processos de nascimento, morte, migração e interações.

crescimento, regulação, predação, competição, mutualismo e metapopulações, difusão.

Ecologia teórica, Epidemiologia, Biologia evolutiva e Gestão ambiental.

Processos Fundamentais.

— DEST:

$t = - \quad + -$

10.2 Modelos de Crescimento Populacional

Crescimento Exponencial (Malthusiano).

Modelo Mais Simples

População cresce proporcionalmente ao seu tamanho:

$$\frac{dN}{dt} = rN$$

onde $r = b - d$ é a taxa intrínseca de crescimento per capita.

— DEST:

$$N(t) = N_0 e^{rt}$$

Crescimento ilimitado ($r > 0$), Decaimento exponencial ($r < 0$) e Tempo de duplicação: $t_d = r$.

Crescimento Logístico (Verhulst).

Modelo com Capacidade de Suporte

Incorpora limitação de recursos através da capacidade de suporte K :

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right)$$

$$N(t) = \frac{KN_0 e^{rt}}{K + N_0(e^{rt} - 1)} = \frac{K}{1 + \left(\frac{K - N_0}{N_0}\right) e^{-rt}}$$

Forma sigmoidal (curva S), NK quando t , Ponto de inflexão em $N = K/2$ e Taxa máxima em $N = K/2$.

Implementação: Crescimento Logístico.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def logistic(N, t, r, K):
6     """Modelo de crescimento logistico"""
7     return r * N * (1 - N/K)
8
9 # Parametros
10 r = 0.8      # taxa de crescimento
11 K = 1000     # capacidade de suporte
12 N0 = 50      # populacao inicial
13
14 # Tempo
15 t = np.linspace(0, 20, 500)
```



```

16
17 # Resolver EDO
18 N = odeint(logistic, N0, t, args=(r, K))
19
20 # Plotar
21 plt.figure(figsize=(10, 6))
22 plt.plot(t, N, 'b-', linewidth=2, label='N(t)')
23 plt.axhline(y=K, color='r', linestyle='--',
24             linewidth=2, label=f'K_{K}')
25 plt.xlabel('Tempo')
26 plt.ylabel('Populacao')
27 plt.title('Crescimento Logistico')
28 plt.legend()
29 plt.grid(True, alpha=0.3)

```

Listing 10.1: Modelo logístico em Python

Efeito Allee.

Crescimento com Densidade Crítica

Em algumas populações, baixas densidades reduzem a taxa de crescimento (dificuldade de encontrar parceiros, defesa contra predadores):

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right) \left(\frac{N}{A} - 1\right)$$

onde A é a densidade crítica (limiar de Allee).

Três equilíbrios: $N^* = 0, A, K$, $N = 0$ estável, $N = A$ instável, $N = K$ estável e Risco de extinção se $N < A$.

Modelos em Tempo Discreto.

Definição (Equações de Diferença).

Modelos onde o tempo avança em passos discretos: $N_{t+1} = f(N_t)$

Modelo de Ricker

$$N_{t+1} = N_t e^{r(1-N_t/K)}$$

Pode exibir caos, Usado para peixes com gerações discretas e Comportamento complexo para r alto.

— Modelo de Beverton-Holt —

$$N_{t+1} = \frac{RN_t}{1 + (R - 1)N_t/K}$$

Sempre estável, R = taxa de reprodução líquida e Converge monotonicamente para K .

Implementação: Modelo de Ricker.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def ricker(N, r, K):
5     """Modelo de Ricker"""
6     return N * np.exp(r * (1 - N/K))
7
8 # Parametros
9 r_values = [1.5, 2.0, 2.7] # diferentes taxas de
    crescimento
10 K = 100
11 N0 = 10
12 t_max = 50
13
14 # Simular
15 fig, axes = plt.subplots(1, 3, figsize=(15, 4))
16
17 for i, r in enumerate(r_values):
18     N = np.zeros(t_max)
19     N[0] = N0
20
21     for t in range(t_max - 1):
22         N[t+1] = ricker(N[t], r, K)
23
24     axes[i].plot(N, 'b.-', markersize=4)
25     axes[i].axhline(y=K, color='r', linestyle='--',
26                     label=f' $K_{\square}=\square\{K\}$ ')
27     axes[i].set_xlabel('Geracao  $\square t$ ')
28     axes[i].set_ylabel('N(t)')
29     axes[i].set_title(f' $r_{\square}=\square\{r\}$ ')
30     axes[i].legend()
31     axes[i].grid(True, alpha=0.3)
32
```

```
33 plt.tight_layout()
```

Listing 10.2: Dinâmica discreta com Ricker

Diagrama de Linha de Fase.

Plotar $N_t + 1$ vs N_t , Linha de identidade: $N_t + 1 = N_t$, Equilíbrios: intersecções e Estabilidade: inclinação.

$$\left| \frac{df}{dN} \right|_{N^*} < 1$$

Se > 1 : instável (oscilações ou caos)
trajetória iterativa

10.3 Interações entre Espécies

Tipos de Interações.

Notação

(+) = efeito benéfico na taxa de crescimento, (−) = efeito prejudicial na taxa de crescimento e (0) = sem efeito.

Modelo Lotka-Volterra Predador-Presa.

Sistema de Equações

$$\begin{aligned} \frac{dV}{dt} &= \alpha V - \beta VP & (\text{Presas}) \\ \frac{dP}{dt} &= \delta \beta VP - \gamma P & (\text{Predadores}) \end{aligned}$$

V = densidade de presas, P = densidade de predadores,

= *taxa de crescimento das presas*,

= taxa de predação,

= *eficiência de conversão (biomassa presa*

predador) e

= *taxa de morte dos predadores*.

Interpretação Biológica.

$$\frac{dV}{dt} = \underbrace{\alpha V}_{\text{crescimento exponencial}} - \underbrace{\beta VP}_{\text{perda por predação}}$$

Sem predadores: $t = V$, Taxa de encontro: lei de ação das massas (VP) e Predação reduz crescimento.

$$\frac{dP}{dt} = \underbrace{\delta \beta VP}_{\text{crescimento por consumo}} - \underbrace{\gamma P}_{\text{morte natural}}$$

Sem presas: $t = -P$, Crescimento proporcional ao consumo e Morte independente de densidade.

— IMP:

Este sistema exhibe oscilações periódicas permanentes!

Análise de Equilíbrio.

Pontos de Equilíbrio

Resolver $t = 0$ e $t = 0$:

$$(V^*, P^*) = (0, 0).$$

-

$$V^* = \frac{\gamma}{\delta\beta}, \quad P^* = \frac{\alpha}{\beta}$$

$t = 0V = 0$ ou $P = /$ e $t = 0P = 0$ ou $V = /()$.

Implementação: Lotka-Volterra.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def lotka_volterra(y, t, alpha, beta, delta, gamma):
6     """Sistema Lotka-Volterra predador-presa"""
7     V, P = y
8     dVdt = alpha*V - beta*V*P
9     dPdt = delta*beta*V*P - gamma*P
10    return [dVdt, dPdt]
11
12 # Parametros
13 alpha = 1.0      # crescimento presas
14 beta = 0.1       # taxa de predacao
15 delta = 0.75     # eficiencia conversao
16 gamma = 1.5      # morte predadores

```

```

17
18 # Condições iniciais
19 y0 = [40, 9] # [V0, P0]
20 t = np.linspace(0, 50, 1000)
21
22 # Resolver
23 sol = odeint(lotka_volterra, y0, t,
24             args=(alpha, beta, delta, gamma))
25
26 # Plotar series temporais
27 plt.figure(figsize=(12, 5))
28 plt.subplot(1, 2, 1)
29 plt.plot(t, sol[:, 0], 'b-', linewidth=2, label='Presas (V
30          )')
31 plt.plot(t, sol[:, 1], 'r-', linewidth=2, label='
32          Predadores (P)')
33 plt.xlabel('Tempo')
34 plt.ylabel('Densidade')
35 plt.legend()
36 plt.grid(True, alpha=0.3)

```

Listing 10.3: Sistema predador-presa

Retrato de Fase.

```

1 # Retrato de fase
2 plt.subplot(1, 2, 2)
3 plt.plot(sol[:, 0], sol[:, 1],
4          'g-', linewidth=2)
5 plt.plot(sol[0, 0], sol[0, 1],
6          'go', markersize=10,
7          label='Início')
8
9 # Equilíbrio
10 V_eq = gamma/(delta*beta)
11 P_eq = alpha/beta
12 plt.plot(V_eq, P_eq, 'r*',
13          markersize=15,
14          label='Equilíbrio')
15
16 # Isoclinas
17 V_range = np.linspace(0, 60, 100)
18 plt.axhline(y=P_eq, color='r',
19             linestyle='--',
20             alpha=0.5,
21             label='dV/dt=0')
22 plt.axvline(x=V_eq, color='b',
23             linestyle='--',
24             alpha=0.5,
25             label='dP/dt=0')
26
27 plt.xlabel('Presas (V)')
28 plt.ylabel('Predadores (P)')
29 plt.legend(fontsize=8)
30 plt.grid(True, alpha=0.3)
31 plt.tight_layout()

```

Listing 10.4: Plano de fase

Órbitas fechadas (ciclos), Período depende de CI e Centro neutro (não atrator).

Competição: Lotka-Volterra.

Sistema de Competição

Duas espécies competindo pelos mesmos recursos:

$$\begin{aligned}\frac{dN_1}{dt} &= r_1 N_1 \left(1 - \frac{N_1 + \alpha_{12} N_2}{K_1} \right) \\ \frac{dN_2}{dt} &= r_2 N_2 \left(1 - \frac{N_2 + \alpha_{21} N_1}{K_2} \right)\end{aligned}$$

α_{12} = efeito da espécie 2 sobre a espécie 1 e α_{21} = efeito da espécie 1 sobre a espécie 2.

$\alpha_{12} N_2$ converte indivíduos da espécie 2 em "equivalentes" da espécie 1 em termos de uso de recursos.

Exclusão Competitiva.

Quatro Cenários Possíveis

$K_1 > \alpha_{12} K_2$ e $K_2 < \alpha_{21} K_1$, $K_2 > \alpha_{21} K_1$ e $K_1 < \alpha_{12} K_2$, $K_1 > \alpha_{12} K_2$ e $K_2 > \alpha_{21} K_1$ e $K_1 < \alpha_{12} K_2$ e $K_2 < \alpha_{21} K_1$.

— IMP:

Princípio da Exclusão Competitiva (Gause): duas espécies não podem coexistir no mesmo nicho ecológico.

Mutualismo.

Modelo de Mutualismo

Interação benéfica mútua (polinizadores-plantas, micorrizas-raízes):

$$\begin{aligned}\frac{dN_1}{dt} &= r_1 N_1 \left(1 - \frac{N_1}{K_1 + \alpha_{12} N_2} \right) \\ \frac{dN_2}{dt} &= r_2 N_2 \left(1 - \frac{N_2}{K_2 + \alpha_{21} N_1} \right)\end{aligned}$$

Coeficientes $\alpha_{ij} > 0$ aumentam capacidade de suporte, Mutualismo facultativo: espécies podem sobreviver sozinhas e Mutualismo obrigatório: requer ambas espécies.

Abelhas aumentam reprodução das plantas (polinização)

Flores fornecem néctar/pólen para abelhas

Coevolução: dependência mútua crescente

Modelo Nicholson-Bailey.

— Hospedeiro-Parasitóide em Tempo Discreto —

$$\begin{aligned}H_{t+1} &= \lambda H_t e^{-aP_t} \\ P_{t+1} &= cH_t(1 - e^{-aP_t})\end{aligned}$$

H_t = densidade de hospedeiros na geração t , P_t = densidade de parasitóides,

= taxa de reprodução do hospedeiro,

a = taxa de ataque do parasitóide c = número de parasitóides emergindo por hospedeiro.

e^{-aP_t} é a probabilidade de um hospedeiro escapar do parasitismo.

Aplicação

Controle biológico de pragas agrícolas

10.4 Modelos Epidemiológicos

Epidemiologia Matemática.

Definição (Modelo Epidemiológico).

Modelo matemático que descreve a transmissão e propagação de doenças infecciosas em uma população.

Susceptíveis, Expostos (incubação), Infectados (infecciosos), Recuperados (imunes) e Mortos.

Previsão de epidemias, Avaliação de intervenções, Políticas de vacinação e Alocação de recursos.

Exemplos Históricos

COVID-19, Influenza, Ebola, HIV/AIDS, Malária, Dengue

Modelo SIR.

Sistema SIR Clássico

$$\begin{aligned}\frac{dS}{dt} &= -\beta \frac{SI}{N} \\ \frac{dI}{dt} &= \beta \frac{SI}{N} - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

= taxa de transmissão (contatos infectantes por unidade de tempo),

= taxa de recuperação ($1/\gamma$ = período infeccioso) e $N = S + I + R$ = população total (constante).

Número Básico de Reprodução.

— DEF: R_0 (R-naught) —

Número médio de infecções secundárias causadas por um indivíduo infectado introduzido em uma população completamente susceptível.

— DEST:

R_0 ===

Interpretação Epidemiológica

$R_0 < 1$: epidemia se extingue (cada infectado infecta < 1 pessoa), $R_0 = 1$: limiar epidêmico (endemia estável) e $R_0 > 1$: epidemia se propaga.

Valores Típicos de R_0

Sarampo: 12-18 (altamente contagioso), COVID-19 (original): 2-3, Influenza sazonal: 1-2 e Ebola: 1.5-2.5.

Limiar de Vacinação.

Fração Crítica de Vacinação

Para eliminar a doença, é necessário vacinar uma fração f da população:

$$f > 1 - \frac{1}{R_0}$$

Vacinação reduz susceptíveis efetivos. Para controle, precisamos $R_0^e f f = R_0(1 - f) < 1$.

$$f > 1 - 1/15 = 0.93 \text{ (93\%)}$$

Implementação: Modelo SIR.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def sir_model(y, t, beta, gamma, N):
```



```

6      """Modelo SIR"""
7      S, I, R = y
8      dSdt = -beta * S * I / N
9      dIdt = beta * S * I / N - gamma * I
10     dRdt = gamma * I
11     return [dSdt, dIdt, dRdt]
12
13 # Parametros
14 N = 10000          # populacao total
15 beta = 0.5         # taxa de transmissao (1/dia)
16 gamma = 0.1        # taxa de recuperacao (1/dia)
17 R0 = beta/gamma    # R0 = 5
18
19 # Condicoes iniciais
20 I0 = 10
21 S0 = N - I0
22 R0_initial = 0
23 y0 = [S0, I0, R0_initial]
24
25 # Tempo
26 t = np.linspace(0, 200, 1000)
27
28 # Resolver
29 sol = odeint(sir_model, y0, t, args=(beta, gamma, N))
30 S, I, R = sol.T

```

Listing 10.5: Simulação de epidemia SIR

Análise da Epidemia.

```

1 # Plotar
2 plt.figure(figsize=(12, 5))
3
4 plt.subplot(1, 2, 1)
5 plt.plot(t, S, 'b-', linewidth=2, label='Suscetiveis(S)')
6 plt.plot(t, I, 'r-', linewidth=2, label='Infectados(I)')
7 plt.plot(t, R, 'g-', linewidth=2, label='Recuperados(R)')
8 plt.xlabel('Tempo(dias)')
9 plt.ylabel('Numero de individuos')
10 plt.title(f'Dinamica SIR ($R_0$ = {R0:.1f})')
11 plt.legend()
12 plt.grid(True, alpha=0.3)

```

```

13
14 # Encontrar pico da epidemia
15 peak_idx = np.argmax(I)
16 peak_time = t[peak_idx]
17 peak_infected = I[peak_idx]
18
19 print(f"Pico de infectados: {peak_infected:.0f} indivíduos
    ")
20 print(f"Tempo do pico: {peak_time:.1f} dias")
21 print(f"Taxa de ataque final: {R[-1]/N*100:.1f}%")
22
23 # Limiar epidêmico
24 S_threshold = N / R0
25 print(f"Limiar de suscetíveis: {S_threshold:.0f}")
26
27 plt.subplot(1, 2, 2)
28 plt.plot(S, I, 'purple', linewidth=2)
29 plt.axvline(x=S_threshold, color='red', linestyle='--',
30             label=f'S = N/R0 = {S_threshold:.0f}')
31 plt.xlabel('Suscetíveis (S)')
32 plt.ylabel('Infectados (I)')
33 plt.title('Retrato de Fase')
34 plt.legend()
35 plt.grid(True, alpha=0.3)

```

Listing 10.6: Análise de resultados

Modelo SIS.

SIS: Sem Imunidade Permanente

Indivíduos recuperados voltam a ser susceptíveis (ex: resfriado comum, gonorreia):

$$\begin{aligned}\frac{dS}{dt} &= -\beta \frac{SI}{N} + \gamma I \\ \frac{dI}{dt} &= \beta \frac{SI}{N} - \gamma I\end{aligned}$$

0

Se $R_0 > 1$,

$$I^* = N \left(1 - \frac{1}{R_0}\right), \quad S^* = \frac{N}{R_0}$$

— IMP:

Doença persiste permanentemente na população (endemia)!

Modelo SEIR.

SEIR: Com Período de Incubação

Adiciona compartimento de Expostos (infectados mas não infecciosos):

$$\begin{aligned}\frac{dS}{dt} &= -\beta \frac{SI}{N} \\ \frac{dE}{dt} &= \beta \frac{SI}{N} - \sigma E \\ \frac{dI}{dt} &= \sigma E - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

onde

$\beta = \text{taxa de progressão}$

$1/\sigma = \text{período de incubação}$.

1

$$R_0 = \frac{\beta}{\gamma} \quad (\text{permanece o mesmo})$$

mas a dinâmica temporal é alterada (pico mais tardio).

Extensões do Modelo SIR.

Nascimentos: N , Mortes naturais: S, I, R e Mortalidade pela doença: I .

Diferentes

por faixa etária, Matrizes de contato e Vacinação por idade.

Vacinação: S , Quarentena: redução de

, Tratamento : aumento de

e Distanciamento social.

Modelos metapopulacionais, Difusão espacial e Redes de contato.

— Modelagem de COVID-19 —

Combinou SEIR + demografia + estrutura etária + intervenções não-farmacêuticas + vacinação

10.5 Dinâmica Espacial

Metapopulações.

Definição (Metapopulação).

Conjunto de populações locais conectadas por dispersão, onde cada população local pode sofrer extinção e recolonização.

$$\frac{dp}{dt} = cp(1-p) - ep$$

p = fração de patches ocupados, c = taxa de colonização e e = taxa de extinção local.

$$p^* = 1 - \frac{e}{c}$$

2

$$c > e$$

Equações de Reação-Difusão.

Difusão Espacial

Combina crescimento local com dispersão espacial:

$$\frac{\partial N}{\partial t} = \underbrace{f(N)}_{\text{reação}} + \underbrace{D\nabla^2 N}_{\text{difusão}}$$

Em 1D:

$$\frac{\partial N}{\partial t} = f(N) + D \frac{\partial^2 N}{\partial x^2}$$

D = coeficiente de difusão (taxa de dispersão), $f(N)$ = dinâmica local (ex: logística) e ∇^2 = operador Laplaciano.

$$\frac{\partial N}{\partial t} = rN \left(1 - \frac{N}{K}\right) + D \frac{\partial^2 N}{\partial x^2}$$

Modelo de invasão espacial de uma espécie

Ondas Viajantes.

Solução de Onda Viajante

Perturbação que se propaga no espaço mantendo sua forma:

$$N(x, t) = N(z), \quad z = x - ct$$

onde c é a velocidade da onda.

$$c_{min} = 2\sqrt{rD}$$

3

Invasões biológicas, frentes de epidemias, propagação de genes

Padrões de Turing.

Instabilidade de Difusão

Sistema de duas espécies com difusão diferencial pode gerar padrões espaciais espontâneos:

$$\begin{aligned}\frac{\partial u}{\partial t} &= f(u, v) + D_u \nabla^2 u \\ \frac{\partial v}{\partial t} &= g(u, v) + D_v \nabla^2 v\end{aligned}$$

$D_v D_u$ (inibidor difunde mais rápido que ativador)

Ativador de curto alcance, Inibidor de longo alcance e Desestabilização por difusão.

Padrões de pelagem animal, Distribuição de estômatos em folhas e Arranjo de penas em aves.

4

— IMP:

Alan Turing (1952): "The Chemical Basis of Morphogenesis"

Implementação: Difusão 1D.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def fisher_kpp_1d(N, t, r, K, D, dx):
5     """Equacao de Fisher-KPP em 1D"""
6     # Termo de reacao (logistico)
7     reaction = r * N * (1 - N/K)
8
9     # Termo de difusao (diferenca finita central)
10    diffusion = np.zeros_like(N)
11    diffusion[1:-1] = D * (N[2:] - 2*N[1:-1] + N[:-2]) / dx**2
12
13    # Condicoes de contorno (sem fluxo)
14    diffusion[0] = diffusion[-1] = 0
15
16    return reaction + diffusion
17
18 # Parametros

```

```

19 L = 50          # tamanho do dominio
20 nx = 200        # pontos espaciais
21 x = np.linspace(0, L, nx)
22 dx = x[1] - x[0]
23
24 r = 0.5          # taxa de crescimento
25 K = 1.0          # capacidade de suporte
26 D = 0.1          # coeficiente de difusao
27
28 # Condicao inicial (pulso localizado)
29 N0 = np.zeros(nx)
30 N0[nx//4:nx//4+10] = K
31
32 # Simular usando metodo de Euler
33 t_max = 100
34 dt = 0.01
35 steps = int(t_max / dt)
36
37 N = N0.copy()
38 for step in range(steps):
39     N += dt * fisher_kpp_1d(N, step*dt, r, K, D, dx)

```

Listing 10.7: Equação de Fisher-KPP em 1D

Metapopulações e Redes.

$$\frac{dN_i}{dt} = f(N_i) + \sum_{j=1}^n m_{ij}(N_j - N_i)$$

N_i = população no patch i , m_{ij} = taxa de migração de i para j e $f(N_i)$ = dinâmica local.

Alta conectividade

sincronismo, Baixa conectividade

assincronia e Importante para estabilidade.

5

Regular (grade), Aleatória, Escala-livre e Mundo-pequeno.

Aplicação

Conservação: corredores ecológicos conectando fragmentos de habitat

10.6 Exercícios

Exercícios - Parte 1.

Questão 1: Crescimento Populacional

Uma população segue o modelo logístico com $r = 0.6 \text{ dia}^{-1}$ e $K = 5000$. Se $N(0) = 500$, calcule $N(10)$, Em que tempo a população atinge 90 e Qual a taxa máxima de crescimento (dN/dt)? Em que densidade?.

Questão 2: Efeito Allee

Considere o modelo com efeito Allee: $dt = rN(1 - N/K)(N/A - 1)$, com $r = 0.5$, $K = 1000$, $A = 100$.

Encontre os pontos de equilíbrio, Analise a estabilidade de cada equilíbrio, Simule em Python para $N(0) = 50$ e $N(0) = 150$ e Qual a implicação para conservação?.

Exercícios - Parte 2.**Questão 3: Modelo de Ricker**

Implemente o modelo de Ricker: $N_t + 1 = N_t e^r(1 - N_t/K)$

Simule para $r = 1.5, 2.0, 2.5, 3.0$ com $K = 100$, Crie diagramas de bifurcação variando r de 0 a 3, Identifique regiões de estabilidade e caos e Compare com o modelo de Beverton-Holt.

— Questão 4: Lotka-Volterra —

Para o sistema predador-presa com $\alpha = 1.0$, $\beta = 0.1$, $\gamma = 0.75$, $\delta = 1.5$:

Calcule o ponto de equilíbrio não-trivial, Calcule a matriz Jacobiana no equilíbrio, Determine os autovalores. O equilíbrio é estável? e Simule e plote séries temporais e retrato de fase.

Exercícios - Parte 3.**Questão 5: Modelo SIR**

Uma doença tem $\alpha = 0.4 \text{ dia}^{-1}$ e $\beta = 0.1 \text{ dia}^{-1}$ em uma população de $N = 10000$.

Calcule R_0 , Se inicialmente $I(0) = 20$, simule 200 dias, Qual o pico de infectados? Quando ocorre?, Qual a taxa de ataque final (proporção que adoecer)? e Que fração deve ser vacinada para prevenir epidemia?.

Projeto: Competição e Coexistência

Implemente o modelo de competição Lotka-Volterra:

Simule os 4 cenários (sp1 vence, sp2 vence, coexistência, bistabilidade), Crie retratos de fase com isoclinas, Adicione estocasticidade demográfica e Explore efeitos de flutuações ambientais.

10.7 Notas Bibliográficas

Referências Principais.

Murray J.D. (2002). *Mathematical Biology I: An Introduction*, 3rd ed. Springer.

Edelstein-Keshet L. (2005). *Mathematical Models in Biology*. SIAM.

Otto S.P., Day T. (2007). *A Biologist's Guide to Mathematical Modeling in Ecology and Evolution*. Princeton University Press.

Keeling M.J., Rohani P. (2008). *Modeling Infectious Diseases in Humans and Animals*. Princeton University Press.

Anderson R.M., May R.M. (1991). *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press.

Leitura Complementar.

Ecologia Teórica

Case T.J. (2000). *An Illustrated Guide to Theoretical Ecology*, Kot M. (2001). *Elements of Mathematical Ecology* e Tilman D. (1982). *Resource Competition and Community Structure*.

Epidemiologia

Diekmann O., Heesterbeek H. (2000). *Mathematical Epidemiology of Infectious Diseases* e Brauer F., Castillo-Chavez C. (2012). *Mathematical Models in Population Biology and Epidemiology*.

Recursos Computacionais

(R package para epidemiologia), (Python para sistemas dinâmicos) e (simulação de modelos).

Obrigado!

Capítulo 10: Sistemas Populacionais

Tópicos Avançados em Biologia de Sistemas

Dúvidas? Perguntas?

Parte IV

Aplicações Avançadas

Capítulo 11

Análise Integrada Multi-Ômica

11.1 Introdução

Visão Geral do Capítulo.

Objetivos de Aprendizagem

Compreender a abordagem de biologia de sistemas, Dominar estratégias de integração multi-ômica, Analisar caso de estudo: ciclo da trealose em levedura, Aplicar métodos de análise de vias integradas e Construir modelos de sistemas integrados.

Por que Análise Integrada?

Visão holística de sistemas biológicos complexos, Descoberta de mecanismos regulatórios ocultos, Predição de comportamento do sistema e Identificação de alvos terapêuticos.

Abordagem de Biologia de Sistemas.

Definição (Biologia de Sistemas).

Estudo de sistemas biológicos através da integração quantitativa de dados multi-ômicos para compreender propriedades emergentes e comportamento do sistema como um todo.

Estratégia de Integração Multi-Ômica.

sequência, variações, expressão gênica, abundância de proteínas, concentrações de metabólitos e fluxos metabólicos.

Escalas diferentes, Heterogeneidade de dados, Ruído experimental, Dados faltantes, Causalidade vs correlação e Complexidade computacional.

— IMP:

A integração efetiva requer métodos estatísticos robustos e modelagem matemática rigorosa.

Caso de Estudo: Ciclo da Trealose em Levedura.

Por que Trealose?

Saccharomyces cerevisiae utiliza trealose como:

Proteção contra estresse (calor, frio, dessecação), Reserva de carbono, Sinalização celular e Regulação do metabolismo.

Vantagens para Estudo Integrado

Sistema bem caracterizado, Via regulada em múltiplos níveis, Dados multi-ômicos disponíveis, Respostas mensuráveis e Modelo para outros sistemas de estresse.

11.2 Integração Multi-Ômica

Visão Geral das Camadas Ômicas.

— IMP:

Cada camada adiciona informação complementar sobre o estado do sistema.

Tipos e Características de Dados.

Implicações para Integração

Normalização entre escalas, Tratamento de valores faltantes, Métodos estatísticos robustos e Validação experimental.

Estratégias de Integração: Early, Intermediate, Late Fusion.

Métodos Baseados em Correlação.

— Análise de Correlação Multi-Ômica —

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}$$

Correlação de Pearson (linear), Correlação de Spearman (monotônica), Distância de correlação, Informação mútua, Correlação parcial e Correlação canônica (CCA).

mRNA-proteína, Proteína-metabólito, Expressão-fluxo e Módulos co-regulados.

Cuidado!

Correlação

Causalidade

Integração Baseada em Redes.

Construção de Redes

biomoléculas (genes, proteínas, metabólitos), interações, correlações, regulações e força da interação.

Co-expressão, Interação proteína-proteína, Regulação gênica, Metabólica e Multi-camada (multiplex).

Centralidade, Modularidade e Comunidades.

Abordagens de Machine Learning.

Métodos Supervisionados

classificação multi-ômica, identificação de biomarcadores, predição de fenótipo e seleção de features.

— Métodos Não-Supervisionados —

redução de dimensionalidade, visualização, identificação de subgrupos, decomposição de matrizes e aprendizado de representações.

Decomposição fatorial para identificar fontes de variação compartilhadas e específicas entre camadas ômicas.

Desafios: Heterogeneidade de Dados.

valores absolutos vs relativos, normal, log-normal, Poisson, 10^2 a 10^5 features, muitos zeros e técnico e biológico.

Normalização apropriada, Transformações (log, Box-Cox), Batch correction, Imputação de valores faltantes, Regularização e Métodos robustos.

— Normalização Multi-Ômica —

$$X_{norm} = \frac{X - \mu}{\sigma} \quad \text{ou} \quad X_{norm} = \frac{X - \min(X)}{\max(X) - \min(X)}$$

Fluxo de Trabalho Multi-Ômico.

Exemplo Python: Integração de Dados.

```

1 import pandas as pd
2 import numpy as np
3 from scipy import stats
4 import seaborn as sns
5 import matplotlib.pyplot as plt
6
7 # Carregar dados multi-omicos
8 transcriptomics = pd.read_csv('transcriptomics.csv', index_col=0)
9 proteomics = pd.read_csv('proteomics.csv', index_col=0)
10 metabolomics = pd.read_csv('metabolomics.csv', index_col=0)
11
12 # Normalizar cada dataset (z-score)
13 def normalize_zscore(df):
14     return (df - df.mean()) / df.std()
15
16 trans_norm = normalize_zscore(transcriptomics)
17 prot_norm = normalize_zscore(proteomics)
18 metab_norm = normalize_zscore(metabolomics)
19
20 # Correlacao mRNA-proteina
21 correlation_matrix = pd.DataFrame(
22     index=trans_norm.index,
23     columns=prot_norm.index

```

```

24 )
25
26 for gene in trans_norm.index:
27     for protein in prot_norm.index:
28         r, p = stats.pearsonr(trans_norm.loc[gene],
29                               prot_norm.loc[protein])
30         if p < 0.05: # significativo
31             correlation_matrix.loc[gene, protein] = r
32
33 # Visualizar heatmap
34 plt.figure(figsize=(12, 10))
35 sns.heatmap(correlation_matrix.astype(float),
36             cmap='RdBu_r', center=0, vmin=-1, vmax=1)
37 plt.title('Correlação mRNA-Proteína')

```

Listing 11.1: Integração multi-ômica com correlação

11.3 Caso de Estudo: Ciclo da Trealose em Levedura

Background Biológico: Resposta ao Estresse em *S. cerevisiae*.

Trealose como Protetor Celular

Dissacarídeo não-redutor (



-D-glicopiranosídeo), Acumula durante estresse térmico, osmótico, oxidativo, Estabiliza membranas e proteínas, Pode atingir 20 e Mobilizada quando condições melhoram.

Relevância Industrial

Panificação: tolerância ao estresse, Fermentação alcoólica: robustez, Produção de biocombustíveis e Biotecnologia: engenharia de robustez.

Via de Biossíntese e Degradação da Trealose.

— IMP:

Via regulada transcricionalmente, pós-traducionalmente e alostericamente.

Enzimas Chave: TPS1, TPS2, NTH1.

— TPS1 (Trealose-6-P Sintase) —

Catalisa: UDP-Glc + G6P

T6P, Subunidade catalítica, Gene essencial, Regulação: PKA, Snf1.

$$v_{TPS1} = \frac{V_{max}[UDP-Glc][G6P]}{K_m^{UDP} K_m^{G6P}}$$

— TPS2 (Trealose-6-P Fosfatase) —

Catalisa: T6P

Trealose + Pi, RemoveT6P(tóxicoemexcesso)eEssencialparacrescimento.

NTH1 (Trealase Neutra)

Catalisa: Trealose

2Glc, Mobilizareservas, AtivaporPKA(cAMP)eInibidaduranteestresse.

$$v_{NTH1} = \frac{V_{max}[Tre]}{K_m + [Tre]} \cdot f(PKA)$$

Complexo TSC

TPS1, TPS2 e subunidades regulatórias (TSL1, TPS3) formam complexo de 800 kDa

Mecanismos Regulatórios.

— DEST:

Sistema de feedback multi-nível garante homeostase de trealose e G6P.

Dados Experimentais: Multi-Ômicos.

Dataset Integrado (Choque Térmico 37°C)

Observações

mRNA responde rapidamente (15 min), Proteína segue com atraso (30-60 min), Trealose acumula gradualmente e Saturação em 2h.

Construção do Modelo Matemático.

Sistema de EDOs para Ciclo da Trealose

$$\begin{aligned} \frac{d[mRNA_{TPS1}]}{dt} &= k_{trans} \cdot f(stress) - \delta_{mRNA}[mRNA_{TPS1}] \\ \frac{d[TPS1]}{dt} &= k_{transl}[mRNA_{TPS1}] - \delta_{prot}[TPS1] \\ \frac{d[T6P]}{dt} &= v_{TPS1} - v_{TPS2} \\ \frac{d[Tre]}{dt} &= v_{TPS2} - v_{NTH1} \end{aligned}$$

onde:

$f(stress) = K_d^n + stress^n$ (função de Hill), $v_T PS1 =^T PS1[TPS1][G6P][UDPGlc](K_m^G 6P + [G6P])(K_m^U DP + [UDPGlc])$ e $v_N TH1 =^N TH1[NTH1][Tre]K_m^T re + [Tre]f(PKA)$.

Estimação de Parâmetros.

-

$$\min_{\theta} \sum_i (y_i^{obs} - y_i^{model}(\theta))^2$$

-

$$\max_{\theta} \prod_i P(y_i^{obs} | \theta)$$

-

$$P(\theta|y) \propto P(y|\theta)P(\theta)$$

$V_{max} > 0$, $K_m > 0$, > 0 e Limites biológicos.

Ferramentas

scipy.optimize, lmfit, PyMC3, COPASI e MATLAB simbiology.

— IMP:

Usar dados multi-ômicos para restringir espaço de parâmetros.

Validação do Modelo.

R^2 , RMSE, hold-out, k-fold, novas condições, parâmetros críticos e mutantes, perturbações.

$$R^2 = 1 - SS_{tot}, RMSE = n(y_i - \hat{y}_i)^2, AIC = 2k - 2(L) \text{ e } BIC = k(n) - 2(L).$$

Validação Experimental

Testar mutantes:

,

, Perturbações químicas: inibidores de PKA, Diferentes condições de estresse e Comparar predições com medições.

Predições e Verificação Experimental.

Predições do Modelo

Aumento de 10x na expressão de TPS1 sob estresse leva a acúmulo 2x mais rápido de trealose, Deleção de NTH1 resulta em acúmulo de trealose mesmo após estresse, Inibição de PKA aumenta trealose basal em 3x e Pré-adaptação ao estresse acelera resposta em 40

Verificação Experimental

— DEST:

Concordância entre modelo e experimentos valida abordagem integrada!

Implementação Python: Modelo de Trealose.

```

1 import numpy as np
2 from scipy.integrate import odeint
3 import matplotlib.pyplot as plt
4
5 def trehalose_model(y, t, params):
6     """Modelo do ciclo da trealose"""
7     mRNA_TPS1, TPS1, T6P, Tre = y
8
9     k_trans, delta_mRNA, k_transl, delta_prot = params[:4]
10    Vmax_TPS1, Km_G6P, Vmax_TPS2, Km_T6P = params[4:8]
11    Vmax_NTH1, Km_Tre, stress_level = params[8:]
12
13    # Constantes
14    G6P = 5.0 # mM (assumido constante)
15    UDP_Glc = 2.0 # mM
16
17    # Regulacao transcritcional (Hill function)
18    n_hill = 4
19    Kd_stress = 0.5
20    f_stress = stress_level**n_hill / (Kd_stress**n_hill + stress_level**n_hill
21    )
22
23    # Taxas de reacao
24    v_TPS1 = (Vmax_TPS1 * TPS1 * G6P * UDP_Glc) / ((Km_G6P + G6P) * (1.0 +
25    UDP_Glc))
26    v_TPS2 = (Vmax_TPS2 * T6P) / (Km_T6P + T6P)
27    v_NTH1 = (Vmax_NTH1 * Tre) / (Km_Tre + Tre)
28
29    # EDOs
30    d_mRNA = k_trans * f_stress - delta_mRNA * mRNA_TPS1
31    d_TPS1 = k_transl * mRNA_TPS1 - delta_prot * TPS1
32    d_T6P = v_TPS1 - v_TPS2
33    d_Tre = v_TPS2 - v_NTH1
34
35    return [d_mRNA, d_TPS1, d_T6P, d_Tre]

```

Listing 11.2: Simulação do ciclo da trealose

Simulação e Ajuste aos Dados.

```

1 # Parametros (estimados)
2 params = [
3     5.0, # k_trans
4     0.1, # delta_mRNA
5     2.0, # k_transl
6     0.05, # delta_prot
7     10.0, # Vmax_TPS1
8     0.5, # Km_G6P
9     15.0, # Vmax_TPS2
10    0.2, # Km_T6P
11    8.0, # Vmax_NTH1
12    10.0, # Km_Tre
13    1.0 # stress_level (heat shock)
14 ]
15

```

```

16 # Condições iniciais
17 y0 = [1.0, 1.0, 0.1, 5.0] # mRNA, TPS1, T6P, Tre
18
19 # Tempo de simulação
20 t = np.linspace(0, 120, 100) # 0 a 120 min
21
22 # Resolver EDOs
23 sol = odeint(trehalose_model, y0, t, args=(params,))
24
25 # Plotar resultados
26 fig, axes = plt.subplots(2, 2, figsize=(12, 10))
27 axes[0,0].plot(t, sol[:,0], 'b-', label='mRNA_TPS1')
28 axes[0,1].plot(t, sol[:,1], 'g-', label='Proteína_TPS1')
29 axes[1,0].plot(t, sol[:,2], 'r-', label='T6P')
30 axes[1,1].plot(t, sol[:,3], 'orange', label='Trehalose')
31
32 # Adicionar dados experimentais
33 t_exp = [0, 15, 30, 60, 90, 120]
34 tre_exp = [5, 8, 25, 85, 120, 130]
35 axes[1,1].plot(t_exp, tre_exp, 'ko', label='Experimental')

```

Listing 11.3: Ajuste do modelo aos dados experimentais

11.4 Análise de Vias e Redes Integradas

Análise de Enriquecimento de Vias (Pathway Enrichment).

Definição (Análise de Enriquecimento).

Método estatístico para identificar se um conjunto de genes/proteínas está super-representado em vias biológicas ou processos específicos.

Teste de Hipergeométrico

$$p = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}$$

onde:

N = total de genes no genoma, K = genes na via de interesse, n = genes no conjunto (ex: diferencialmente expressos) e k = genes no conjunto que estão na via.

Ferramentas

DAVID, Enrichr, g:Profiler, clusterProfiler, GSEA

Gene Set Enrichment Analysis (GSEA).

Usa ranking completo de genes, Não requer threshold arbitrário, Detecta mudanças coordenadas sutis e Considera magnitude das mudanças.

Ranquear genes (fold-change, t-statistic), Calcular Enrichment Score (ES), Permutar para p-valor e Corrigir para múltiplos testes (FDR).

ES positivo: via ativada, ES negativo: via reprimida e Magnitude: força do enriquecimento.

Reconstrução de Redes a partir de Multi-Ômicos.

Abordagens de Reconstrução

Pearson, Spearman.

Simples, rápido e Não distingue correlação direta vs indireta.

controla co-variáveis.

$$\rho_{XY|Z} = \frac{\rho_{XY} - \rho_{XZ}\rho_{YZ}}{\sqrt{(1 - \rho_{XZ}^2)(1 - \rho_{YZ}^2)}}$$

-

$$I(X; Y) = \sum_x \sum_y p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

inferência de grafos Gaussianos e aprendem causalidade.

Detecção de Módulos e Clustering.

clusters esféricos, dendrograma, baseado em densidade, modularidade em grafos e autovalores da matriz de adjacência.

Modularidade

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

Identificar processos biológicos, Co-regulação e Predição de função.

Genes co-expressos: HSP, TPS, CTT, SSA - todos regulados por Msn2/4

Identificação de Reguladores e Hubs.

Métricas de Centralidade

número de conexões.

$$k_i = \sum_j A_{ij}$$

fração de caminhos mais curtos que passam pelo nó.

$$C_B(i) = \sum_{s \neq i \neq t} \frac{\sigma_{st}(i)}{\sigma_{st}}$$

proximidade média a outros nós.

$$C_C(i) = \frac{n - 1}{\sum_j d(i, j)}$$

conectado a nós importantes e variante de eigenvector centralidade.

Análise Dinâmica de Redes.

Topologia varia com condições, Arestas aparecem/desaparecem, Pesos mudam e Módulos se reorganizam.

Time-lagged correlation, Dynamic Bayesian Networks, Graphical Granger causality e Sliding window analysis.

— IMP:

Redes dinâmicas revelam sequência temporal de eventos regulatórios.

Exemplo Python: Análise de Redes com NetworkX.

```
1 import networkx as nx
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from scipy.stats import pearsonr
5
6 # Construir rede de correlacao
7 def build_correlation_network(data, threshold=0.7):
8     """Cria rede baseada em correlacoes"""
9     G = nx.Graph()
10
11     # Adicionar nos
12     for gene in data.index:
13         G.add_node(gene)
14
15     # Adicionar arestas (correlacoes significativas)
16     for i, gene1 in enumerate(data.index):
17         for gene2 in data.index[i+1:]:
18             r, p = pearsonr(data.loc[gene1], data.loc[gene2])
19             if abs(r) > threshold and p < 0.05:
20                 G.add_edge(gene1, gene2, weight=abs(r))
21
22     return G
23
24 # Analisar centralidade
25 def analyze_network(G):
26     """Calcula metricas de centralidade"""
27     degree_cent = nx.degree_centrality(G)
28     between_cent = nx.betweenness_centrality(G)
29     close_cent = nx.closeness_centrality(G)
30
31     # Identificar hubs (top 10% degree)
32     threshold = sorted(degree_cent.values())[-len(G)//10]
33     hubs = [node for node, cent in degree_cent.items()
34             if cent >= threshold]
35
36     return degree_cent, between_cent, close_cent, hubs
```

Listing 11.4: Construção e análise de rede multi-ômica

Análise de Enriquecimento com Python.

```
1 import gseapy as gp
2 import pandas as pd
3
4 # Dados de expressao diferencial
```

```

5 de_genes = pd.read_csv('diff_expr.csv')
6 de_genes = de_genes.sort_values('log2FC', ascending=False)
7
8 # Criar ranking
9 gene_rank = de_genes.set_index('gene')['log2FC'].to_dict()
10
11 # Executar GSEA pre-ranked
12 gsea_results = gp.prerank(
13     rnk=gene_rank,
14     gene_sets='KEGG_2021_Human', # banco de vias
15     processes=4,
16     permutation_num=1000,
17     outdir='gsea_output',
18     format='png',
19     seed=42
20 )
21
22 # Vias significativamente enriquecidas
23 enriched_pathways = gsea_results.res2d[
24     gsea_results.res2d['FDR_q-val'] < 0.05
25 ]
26
27 print(f"Vias enriquecidas: {len(enriched_pathways)}")
28 print(enriched_pathways[['Term', 'ES', 'NES', 'FDR_q-val']].head(10))
29
30 # Over-representation analysis (ORA)
31 enrich_ora = gp.enrichr(
32     gene_list=de_genes[de_genes['padj'] < 0.05]['gene'].tolist(),
33     gene_sets='GO_Biological_Process_2021',
34     organism='Yeast',
35     outdir='enrichr_output'
36 )

```

Listing 11.5: GSEA com gseapy

11.5 Modelagem de Sistemas Integrados

Modelos de Escala Genômica com Restrições Ômicas.

— DEF: Genome-Scale Model (GEM) —

Reconstrução matemática do metabolismo completo de um organismo, incluindo todas as reações conhecidas, genes e estequiometria.

Integração de Dados de Expressão

usar níveis de expressão como proxy para capacidades enzimáticas.

$$v_{max,i} \propto \text{expression}_i$$

minimizar diferenças entre fluxos e expressão.

$$\min \sum_i |v_i - v_i^{target}| \quad \text{s.t.} \quad Sv = 0$$

maximizar consistência com dados binários (on/off).

Constraint-Based Modeling com Transcriptômica.

$$\begin{aligned} \max_v \quad & c^T v \\ \text{s.t.} \quad & Sv = 0 \\ & lb \leq v \leq ub \end{aligned}$$

Ajustar lb e ub baseado em níveis de mRNA, Genes altamente expressos: ub alto, Genes não-expressos: $v = 0$ e Genes intermediários: ub proporcional.

0

— IMP:

Dados de expressão restringem espaço de soluções, melhorando previsões.

Modelos Cinéticos com Proteômica/Metabolômica.

— Integração Multi-Nível —

concentrações enzimáticas ($[E]$), concentrações de substratos/produtos ($[S]$, $[P]$) e fluxos metabólicos (v).

Modelo Cinético Integrado

Para cada reação i :

$$v_i = [E_i] \cdot k_{cat,i} \cdot \frac{[S_i]}{K_m^S + [S_i]} \cdot \left(1 - \frac{[P_i]}{[S_i]K_{eq,i}}\right)$$

Sistema de EDOs:

$$\frac{d[S_i]}{dt} = \sum_j s_{ij} v_j$$

onde s_{ij} é o coeficiente estequiométrico.

— DEST:

Dados proteômicos e metabolômicos fornecem valores absolutos para parametrização!

Abordagens de Modelagem Híbrida.

-

FBA para metabolismo central e ODEs para regulação.

-

Gillespie para expressão gênica e ODEs para metabolismo.

-

Molecular (ms), Celular (min-h) e População (h-dias).

1

Vantagens

Captura múltiplas escalas, Usa melhor formalismo para cada nível e Mais realista.

Validação com Dados Independentes.

Estratégias de Validação

treinar em 70 k-fold ($k=5$ ou 10), para datasets pequenos, treinar em tempo t , validar em $t + t$ e perturbações independentes.

Mutantes deletados, Superexpressões, Diferentes condições nutricionais, Resposta a drogas e Dados de literatura.

Acurácia de predição, Correlação observado vs predito, Erro médio absoluto, Sensibilidade/especificidade e AUC-ROC.

Exemplo Python: FBA Integrado com Transcriptômica.

```

1 import cobra
2 import pandas as pd
3
4 # Carregar modelo genome-scale
5 model = cobra.io.read_sbml_model('iMM904.xml') # S. cerevisiae
6
7 # Carregar dados de expressao (RNA-seq)
8 expression_data = pd.read_csv('expression_stress.csv', index_col=0)
9 expression_data = expression_data / expression_data.max() # normalizar 0-1
10
11 # Integrar dados de expressao (metodo E-Flux)
12 def integrate_expression(model, expression, percentile=25):
13     """Ajusta bounds baseado em expressao"""
14     for reaction in model.reactions:
15         # Encontrar genes associados
16         genes = reaction.genes
17
18         if len(genes) > 0:
19             # Media de expressao dos genes
20             gene_expr = [expression.get(g.id, 0.5) for g in genes]
21             avg_expr = sum(gene_expr) / len(gene_expr)
22
23             # Ajustar upper bound
24             if avg_expr < percentile/100:
25                 # Gene pouco expresso: bloquear reacao
26                 reaction.upper_bound = 0
27                 reaction.lower_bound = 0
28             else:
29                 # Escalar bound pela expressao
30                 original_ub = reaction.upper_bound
31                 if original_ub > 0:
32                     reaction.upper_bound = original_ub * avg_expr
33
34 # Aplicar restricoes de expressao
35 integrate_expression(model, expression_data)
36

```

```

37 # Executar FBA
38 solution = model.optimize()
39 print(f"Taxa de crescimento predita: {solution.objective_value:.3f}")

```

Listing 11.6: FBA com restrições de expressão usando COBRApy

Modelo Híbrido: Regulação + Metabolismo.

```

1  import numpy as np
2  from scipy.integrate import odeint
3  import cobra
4
5  def hybrid_model(y, t, model, params):
6      """Modelo híbrido: ODE para regulacao + FBA para metabolismo"""
7      mRNA, protein = y
8      k_trans, k_transl, delta_mRNA, delta_prot = params
9
10     # Parte 1: Regulacao genica (ODE)
11     stress = 1.0 if t < 60 else 0.0 # estresse nos primeiros 60 min
12
13     dmRNA_dt = k_trans * stress - delta_mRNA * mRNA
14     dprotein_dt = k_transl * mRNA - delta_prot * protein
15
16     # Parte 2: Ajustar metabolismo baseado em nivel proteico
17     # Exemplo: protein = TPS1, afeta fluxo de trealose
18     for rxn in model.reactions:
19         if 'TPS1' in rxn.id:
20             # Escalar capacidade pela abundancia proteica
21             rxn.upper_bound = 10.0 * protein
22
23     # Resolver FBA
24     try:
25         solution = model.optimize()
26         growth_rate = solution.objective_value
27         trehalose_flux = solution.fluxes.get('TPS1_rxn', 0.0)
28     except:
29         growth_rate = 0.0
30         trehalose_flux = 0.0
31
32     # Feedback: trealose afeta transcricao
33     feedback = trehalose_flux / 10.0
34     dmRNA_dt += feedback
35
36     return [dmRNA_dt, dprotein_dt]
37
38 # Condições iniciais e parametros
39 y0 = [1.0, 1.0]
40 params = [5.0, 2.0, 0.1, 0.05]
41 t = np.linspace(0, 120, 100)
42
43 # Simular
44 # sol = odeint(hybrid_model, y0, t, args=(model, params))

```

Listing 11.7: Modelo híbrido simples

11.6 Exercícios

Exercícios - Parte 1: Correlação Multi-Ômica.

Questão 1: Análise de Correlação

Dados de transcriptômica e proteômica para 5 genes sob condição de estresse:

Calcule a correlação de Pearson entre mRNA e proteína, É estatisticamente significativa? e Interprete biologicamente o resultado.

Exercícios - Parte 2: Enriquecimento de Vias.

Questão 2: Teste de Hipergeométrico

Em um experimento de estresse oxidativo, 150 genes foram diferencialmente expressos de um total de 6000 genes no genoma. A via de "Resposta ao Estresse Oxidativo" contém 80 genes, e 25 deles estão no conjunto de genes diferencialmente expressos.

Calcule o p-valor usando o teste hipergeométrico, A via está significativamente enriquecida ($= 0.05$)? e Se aplicarmos correção de Bonferroni para 300 vias testadas, ainda é significativa?.

Questão 3: GSEA

Explique a diferença entre ORA e GSEA, Por que GSEA pode detectar mudanças sutis que ORA perde? e Quando você usaria cada método?.

Exercícios - Parte 3: Construção de Modelos.

— Questão 4: Projeto - Modelagem Integrada —

Construa um modelo integrado para a via de glicólise em *S. cerevisiae*:

Baixe dados de RNA-seq, proteômica e metabolômica do GEO, Identifique genes/proteínas/metabólitos da glicólise, Construa modelo cinético simplificado (3-5 reações chave), Estime parâmetros usando dados experimentais, Valide o modelo com dados de mutantes e Simule resposta a mudança de glicose extracelular.

Código Python documentado, Gráficos de ajuste e validação, Análise de sensibilidade de parâmetros e Relatório interpretando resultados biologicamente.

Exercícios - Parte 4: Análise de Redes.

— Questão 5: Construção de Rede de Co-expressão —

Usando dados de RNA-seq de séries temporais (0, 15, 30, 60, 120 min após estresse):

Calcule matriz de correlação entre todos os pares de genes, Construa rede mantendo correlações $|r| > 0.8$ e $p < 0.01$, Identifique módulos usando algoritmo de Louvain, Calcule centralidade de grau, betweenness e close-

ness, Identifique top 10 hubs e Faça análise de enriquecimento GO para cada módulo.

Questão 6: FBA com Restrições

Carregue modelo genome-scale de *S. cerevisiae* (iMM904), Integre dados de expressão usando método E-Flux, Compare taxa de crescimento predita com/sem restrições e Identifique reações essenciais sob estresse.

11.7 Notas Bibliográficas

Referências Principais.

99

Klipp E. et al. (2016). *Systems Biology: A Textbook*, 2nd ed. Wiley-VCH.

Palsson B.O. (2015). *Systems Biology: Constraint-based Reconstruction and Analysis*. Cambridge University Press.

Ideker T., Galitski T., Hood L. (2001). A new approach to decoding life: systems biology. *Ann Rev Genomics Hum Genet*, 2:343-372.

Hasin Y. et al. (2017). Multi-omics approaches to disease. *Genome Biology*, 18:83.

Subramanian A. et al. (2005). Gene set enrichment analysis: a knowledge-based approach. *PNAS*, 102(43):15545-15550.

Leitura Complementar.

— Reviews Multi-Ômicos —

Ritchie M.D. et al. (2015). Methods of integrating data to uncover genotype-phenotype interactions. *Nat Rev Genet*, Huang S. et al. (2017). More is better: recent progress in multi-omics data integration methods. *Front Genet* e Cavill R. et al. (2016). Transcriptomic and metabolomic data integration. *Brief Bioinform.*

Bancos de Dados e Ferramentas

-
-
-
-
-
-
-

Recursos Online.

Tutoriais e Cursos

-
-
-

Pacotes Python/R

cobra, networkx, gseapy, scanpy, scipy, scikit-learn e clusterProfiler, WGCNA, mixOmics, limma, DESeq2.

Livros Especializados

Alon U. (2019). An Introduction to Systems Biology, 2nd ed, Voit E.O. (2017). A First Course in Systems Biology, 2nd ed e Covert M.W. (2014). Fundamentals of Systems Biology.

Obrigado!

Capítulo 11: Integrated Analysis

Integração multi-ômica, Caso de estudo: ciclo da trealose, Análise de vias e redes e Modelagem de sistemas integrados.

Dúvidas? Perguntas?

Capítulo 12

Fisiologia Cardíaca como Sistema de Controle

12.1 Introdução

O Coração como Sistema Complexo.

Definição (Sistema Cardíaco).

O coração é um sistema fisiológico complexo que integra eletrofisiologia, mecânica e regulação para bombear sangue continuamente através do organismo.

Bomba muscular rítmica, Automaticidade intrínseca, Acoplamento elétrico-mecânico, Regulação autônoma e Adaptação à demanda metabólica.

60-100 bpm

5 L/min (repouso)

Níveis de Organização Cardíaca.

Visão Geral do Capítulo.

Objetivos de Aprendizagem

Compreender a anatomia e fisiologia cardíaca, Dominar a eletrofisiologia do cardiomiócito, Estudar modelos matemáticos de células cardíacas, Entender o acoplamento excitação-contração, Explorar mecanismos de arritmias e Aplicar modelagem computacional ao coração.

— IMP:

O coração é um dos sistemas fisiológicos mais bem modelados matematicamente, com implicações clínicas diretas para diagnóstico e terapia de doenças cardiovasculares.

12.2 Anatomia e Fisiologia Cardíaca

Estrutura Anatômica do Coração.

direito e esquerdo

.

(recepção de sangue)

direito e esquerdo

.

(bombeamento)

.

Tricúspide (AD-VD)

Mitral (AE-VE)

.

Pulmonar (VD)

Aórtica (VE)

Circulação

sangue venoso

pulmões(oxigenação)

sangue arterial

corpo(per fusãosistêmica)

Ciclo Cardíaco.

Fases do Ciclo

relaxamento e enchimento.

Enchimento ventricular passivo e Contração atrial (sístole atrial).

contração e ejeção.

Contração isovolumétrica, Ejeção ventricular e Relaxamento isovolumétrico.

— IMP:

Duração típica:

0.8sa75bpm(diástole

0.5 s, sístole

0.3s)

Sistema de Condução Elétrica.

.

Marca-passo natural

60-100 bpm

.

Atraso de condução

0.1s

.

Condução rápida

.

Distribuição ventricular

.

Ativação endocárdio

— DEST:

Velocidade de condução: Purkinje (4 m/s) > His (1-1.5 m/s) > AV (0.05 m/s)

Eletrocardiograma (ECG).

despolarização atrial, despolarização ventricular, repolarização ventricular, 0.12-0.20 s, 0.35-0.44 s e platô ventricular.

Significado Clínico

Arritmias, Isquemia/infarto, Distúrbios de condução e Efeitos de drogas.

Circulação Coronariana.

Artérias Coronárias

VD, parede inferior VE, nó SA/AV, septo, parede anterior VE e parede lateral e posterior VE.

Fluxo principalmente diastólico, Autorregulação metabólica, Alta extração de O₂ (

75 e Reservava vasodilatadora limitada.

Fluxo Coronariano

$$Q = \frac{\Delta P}{R_{cor}}$$

onde:

Q = fluxo (

250 mL/min),

$P = \text{gradiente de pressão} \times R_{cor} = \text{resistência coronariana.}$

— IMP:

Isquemia ocorre quando demanda de O_2 excede oferta (estenose, espasmo, aumento de demanda)

12.3 Eletrofisiologia do Cardiomiócito

Potencial de Ação Cardíaco.

□ Despolarização rápida

Entrada de Na^+ (I_{Na})

□ Repolarização inicial

Saída de K^+ (I_{to})

□ Platô

Entrada Ca^{2+} ($I_{Ca, L}$)

Saída K^+ (I_{Kr} , I_{Ks})

□ Repolarização final

Saída de K^+ (I_{K1})

□ Repouso

Potencial de membrana estável

200-400 ms

120mV

— IMP:

O platô prolongado (Fase 2) previne tetania cardíaca e garante enchimento ventricular adequado

Canais Iônicos Cardíacos.

Principais Correntes Iônicas

Gating

Canais possuem gates (comportas):

ativação, inativação rápida e inativação lenta.

Canalopatias

Mutações em canais causam:

Síndrome do QT longo, Síndrome de Brugada, Taquicardia ventricular e Morte súbita.

Potencial de Membrana em Repouso.

— Equação de Goldman-Hodgkin-Katz —

Para múltiplos íons:

$$V_m = \frac{RT}{F} \ln \left(\frac{P_K[K^+]_o + P_{Na}[Na^+]_o + P_{Cl}[Cl^-]_i}{P_K[K^+]_i + P_{Na}[Na^+]_i + P_{Cl}[Cl^-]_o} \right)$$

$$E_K \approx -90 \text{ mV}$$

$$E_{Na} \approx +70 \text{ mV}$$

$$E_{Ca} \approx +130 \text{ mV}$$

$$E_{Cl} \approx -70 \text{ mV}$$

Em repouso (Fase 4):

$$V_m = -85 \text{ a } -90 \text{ mV}$$

(próximo de E_K)

— DEST:

V_m em repouso dominado por permeabilidade ao K^+ via I_K

Mecanismo do Platô (Fase 2).

$I_{Ca,L}$: corrente de Ca^{2+} tipo-L e Pequena corrente Na^+ persistente.

I_{Kr} : corrente retificadora rápida, I_{Ks} : corrente retificadora lenta e I_{to} : corrente transitória externa.

— IMP:

Equilíbrio delicado:

$$I_{Ca,L} + I_{Kr} + I_{Ks}$$

Importância Clínica

Bloqueio de I_{Kr} (drogas, mutações)

prolongamento QT

torsades de pointes

Período Refratário Efetivo.

Definição (Período Refratário).

Intervalo após um potencial de ação durante o qual a célula não pode ser reexcitada (período refratário absoluto) ou requer estímulo maior (período refratário relativo).

Inativação de canais Na^+ , Recuperação dependente de voltagem, Recuperação lenta (

200ms) e Prolonga com frequência.

Previne reentrada, Limita frequência máxima e Protege contra tetania.

— IMP:

ERP cardíaco (

$200 - 400\text{ms}$) é muito maior que neuronal (

1-2 ms)

Formalismo de Hodgkin-Huxley.

Modelo Matemático de Canais Iônicos

Hodgkin e Huxley (1952) descreveram canais como gates probabilísticos:

$$I_{ion} = g_{ion} \cdot (V_m - E_{ion})$$

onde a condutância é modulada por variáveis de gating:

$$g_{ion} = \bar{g}_{ion} \cdot m^p \cdot h^q$$

m : ativação (abertura), h : inativação (fechamento) e p, q : expoentes de cooperatividade.

Dinâmica de Gating

$$\frac{dm}{dt} = \alpha_m(V)(1 - m) - \beta_m(V)m = \frac{m_\infty(V) - m}{\tau_m(V)}$$

Corrente de Sódio: I_{Na} .

Modelo de I_{Na}

$$I_{Na} = g_{Na} m^3 h j (V_m - E_{Na})$$

m : ativação rápida (0.1 ms), h : inativação rápida (1 – 5 ms) e j :

inativação lenta (20 – 200 ms).

$$m_{\infty} = \frac{1}{1 + e^{-(V+40)/5}}$$

$$h_{\infty} = \frac{1}{1 + e^{(V+65)/7}}$$

$$\tau_m = 0.12 \text{ ms}$$

$$\tau_h = \frac{1}{0.13(1 + e^{-(V+10)/10})}$$

— IMP:

Bloqueio de I_{Na} (classe I antiarrítmicos) reduz velocidade de condução

Correntes de Potássio e Cálcio.

(Corrente – $LdeCa^{2+}$):

$$I_{Ca,L} = g_{Ca}df f_{Ca}(V - E_{Ca})$$

d : ativação, f : inativação voltagem-dependente, f_{Ca} : inativação Ca^{2+} -dependente, Ativa em

$-40mVe$ crucial para contração.

Dihidropiridinas (nifedipina) e Verapamil, diltiazem.

(*Rapiddelayedrectifier*) :

$$I_{Kr} = g_{Kr}x_{Kr1}x_{Kr2}(V - E_K)$$

hERG channel (Kv11.1), Ativação lenta, Retificação interna e Repolarização Fase 3.

(*Slowdelayedrectifier*) :

$$I_{Ks} = g_{Ks}x_s^2(V - E_K)$$

KCNQ1/KCNE1, Ativação muito lenta e Importante em frequências altas.

Simulação: Potencial de Ação Simples.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

```

3
4 def simple_cardiac_ap(t, V, gates):
5     """Modelo simplificado de PA cardíaco"""
6     m, h, d, f, x = gates
7
8     # Parametros
9     gNa, gCa, gK = 23, 0.09, 0.282
10    ENa, ECa, EK = 50, 60, -85
11
12    # Correntes iônicas
13    INa = gNa * m**3 * h * (V - ENa)
14    ICa = gCa * d * f * (V - ECa)
15    IK = gK * x * (V - EK)
16
17    # Capacitancia
18    Cm = 1.0 # uF/cm^2
19
20    # dV/dt
21    dVdt = -(INa + ICa + IK) / Cm
22
23    # Gating variables (simplified)
24    m_inf = 1/(1 + np.exp(-(V+40)/5))
25    h_inf = 1/(1 + np.exp((V+65)/7))
26    d_inf = 1/(1 + np.exp(-(V+10)/6.24))
27    f_inf = 1/(1 + np.exp((V+32)/8))
28    x_inf = 1/(1 + np.exp(-(V+20)/15))
29
30    tau_m, tau_h = 0.1, 2.0
31    tau_d, tau_f, tau_x = 3.0, 20.0, 100.0
32
33    dmdt = (m_inf - m) / tau_m
34    dhdt = (h_inf - h) / tau_h
35    dddt = (d_inf - d) / tau_d
36    dfdt = (f_inf - f) / tau_f
37    dxdt = (x_inf - x) / tau_x
38
39    return dVdt, [dmdt, dhdt, dddt, dfdt, dxdt]

```

Listing 12.1: Modelo simplificado de PA cardíaco

12.4 Modelos de Cardiomiócitos

História dos Modelos Cardíacos.

Evolução Temporal

— IMP:

Modelos mais recentes: >15 correntes, >40 variáveis de estado

Modelo de Noble (1962).

Denis Noble adaptou o modelo de Hodgkin-Huxley para fibras de Purkinje cardíacas.

I_{Na} : corrente de sódio, I_K : corrente de potássio, I_{leak} : corrente de vazamento e I_{an} : corrente aniônica.

$$C_m \frac{dV}{dt} = -I_{Na} - I_K - I_{leak} - I_{an}$$

Reproduziu PA de Purkinje, Automaticidade espontânea, Resposta a estímulos e Base para modelos futuros.

Limitações

Sem Ca^{2+} intracelular, Sem compartimentos e Cinética simplificada.

Modelo de Luo-Rudy (LR1, 1991).

Avanços do LR1

Primeiro modelo ventricular com dinâmica de Ca^{2+} detalhada, Retículo sarcoplasmático (SR), Bomba Na^+/K^+ -ATPase, Trocador Na^+/Ca^{2+} (NCX) e Acoplamento excitação-contração.

I_{Na} , I_{Ca} , L , I_K , I_{K1} , I_{Kp} e I_b (background).

Na^+/K^+ pump, Na^+/Ca^{2+} exchanger e Ca^{2+} pump (sarcolemma).

Citosol e SR (uptake/release).

Modelo ten Tusscher (2004).

Modelo Ventricular Humano Detalhado

Desenvolvido a partir de dados experimentais humanos

PA mais curto, PA mais longo (mid-myocardium) e I_{to} proeminente.

I_{Na} , I_{to} , I_{Kr} , I_{Ks} , I_{K1} , I_{Ca} , L , I_{NaCa} , I_{NaK} , I_pCa , I_pK e I_bNa , I_bCa .

Dispersão transmural da repolarização, Efeitos pró-arrítmicos de drogas, Síndrome do QT longo e Fibrilação ventricular.

Heterogeneidade

A diferença entre tipos celulares cria gradiente de repolarização que gera a onda T do ECG

— IMP:

Modelo amplamente usado: >500 citações, base para estudos de arritmias

Comparação: Modelos Atriais vs Ventriculares.

Courtemanche et al. (1998), Nygren et al. (1998) e Grandi et al. (2011).

Diferenças Importantes

Modelos atriais essenciais para estudar fibrilação atrial, a arritmia cardíaca mais comum

Complexidade Computacional.

— Trade-off: Realismo vs Custo —

0

— IMP:

Para simulações de tecido 3D: modelos simplificados (FitzHugh-Nagumo, Aliev-Panfilov)

Implementação: Modelo Simplificado.

```

1 import numpy as np
2 from scipy.integrate import odeint
3
4 def cardiac_model(y, t, I_stim):
5     """Modelo simplificado baseado em LR1"""
6     V, m, h, j, d, f, X = y
7
8     # Parametros
9     gNa, gCa, gK, gK1 = 23, 0.09, 0.282, 0.6047
10    ENa, ECa, EK = 54.4, 65, -77
11
12    # Gating steady-states e time constants
13    m_inf = 1/(1 + np.exp(-(V+47.13)/10.3))
14    h_inf = 1/(1 + np.exp((V+78.5)/6.22))
15    # ... (outros gates)
16
17    # Correntes
18    INa = gNa * m**3 * h * j * (V - ENa)
19    ICa = gCa * d * f * (V - ECa)
20    IK = gK * X * (V - EK)
21    IK1 = gK1 * (V - EK) / (1 + np.exp(0.07*(V+80)))
22
23    # dV/dt
24    Cm = 1.0
25    dVdt = -(INa + ICa + IK + IK1 - I_stim) / Cm
26
27    # Gating kinetics
28    dmdt = (m_inf - m) / tau_m
29    dhdt = (h_inf - h) / tau_h
30    # ... (outros gates)
31
32    return [dVdt, dmdt, dhdt, djdt, dddt, dfdt, dXdt]
33

```

```

34 # Simulacao
35 t = np.linspace(0, 500, 5000)
36 y0 = [-85, 0.01, 0.99, 0.99, 0.001, 0.99, 0.01]
37 I_stim = lambda t: 50 if 5 < t < 6 else 0
38
39 sol = odeint(cardiac_model, y0, t, args=(I_stim,))

```

Listing 12.2: Modelo cardíaco simplificado

12.5 Acoplamento Excitação-Contração

Acoplamento Excitação-Contração (EC).

Definição (Acoplamento EC).

Processo pelo qual o potencial de ação elétrico desencadeia a contração mecânica do cardiomiócito através da regulação do Ca^{2+} intracelular.

Sequência de Eventos

PA despolariza membrana, Abertura de canais Ca^{2+} tipo-L (DHPR), Entrada de Ca^{2+} no citosol ($I_{Ca,L}$), *–induced* Ca^{2+} release (CICR), Receptores de rianodina (RyR) no SR liberam Ca^{2+} , $[\text{Ca}^{2+}]_i$ aumenta de

0.1

M para

1 – 10

M, Ca^{2+} liga-se à troponina C, Contração dos filamentos de actina-miosina, Recaptação de Ca^{2+} pelo SR (SERCA) e Relaxamento.

CICR: Liberação de Ca^{2+} .

Induzida por Ca^{2+}

Pequena entrada Ca^{2+} via DHPR, Ativa receptores RyR2 no SR, Liberação massiva de Ca^{2+} , Amplificação: 1:10 ou maior e "Calcium sparks".

$[\text{Ca}^{2+}]_{SR}$: carga do SR, Fosforilação de RyR, Calmodulina e ATP, Mg^{2+} .

1

— IMP:

Falha do CICR: causa principal de insuficiência cardíaca sistólica

Dinâmica de Ca^{2+} .

no Retículo Sarcoplasmático

Compartimentos de Ca^{2+}

$[\text{Ca}^{2+}]_i$

0.1

M (diástole)

1 – 10

M (sístole), $[\text{Ca}^{2+}]_{SR}$

1mM(reservatório)e[Ca

$^{2+}]_{ss}$ (microdomínio).

•

$$J_{up} = V_{max, up} [Ca^{2+}]_i^2 K_m^2 + [Ca^{2+}]_i^2$$

•

$$J_{rel} = k_{rel} [Ca^{2+}]_{SR} O_{RyR}$$

•

$$J_{leak} = k_{leak} ([Ca^{2+}]_{SR} - [Ca^{2+}]_i)$$

$$\frac{d[Ca^{2+}]_i}{dt} = J_{rel} - J_{up} + J_{leak} + \frac{I_{Ca,L}}{2FV_{cell}}$$

$$\frac{d[Ca^{2+}]_{SR}}{dt} = J_{up} - J_{rel} - J_{leak}$$

Buffers

Ca^{2+} é tamponado por troponina C, calmodulina, calsequestrin (SR)

Geração de Força: Cross-Bridge Cycling.

Ca^{2+} liga troponina C, Conformação da tropomiosina, Exposição de sítios de ligação, Miosina liga actina, Power stroke (ADP liberado), ATP liga miosina, Desligamento e Hidrólise ATP (recarga).

$$F = F_{max} \cdot f([Ca^{2+}]_i) \cdot g(L)$$

2

:

$$f([Ca^{2+}]) = \frac{[Ca^{2+}]_i^n}{K_d^n + [Ca^{2+}]_i^n}$$

Hill coefficient $n = 4 - 6$

Relação Comprimento-Tensão (Frank-Starling).

— DEF: Lei de Frank-Starling —

A força de contração do coração é proporcional ao comprimento inicial das fibras musculares (pré-carga), dentro de limites fisiológicos.

Sobreposição actina-miosina ótima, Sensibilidade ao Ca^{2+} aumenta com comprimento, Alinhamento de filamentos e Recrutamento de pontes cruzadas.

Ajuste do débito cardíaco, Compensação de alterações de pré-carga e Equilíbrio entre ventrículos.

3

— IMP:

Comprimento ótimo do sarcômero: 2.0-2.2

$m(\text{máximasobreposição})$

Modelo de Contração.

```

1 def contraction_model(Ca_i, SL):
2     """
3     Modelo de geracao de forca
4     Ca_i: concentracao de calcio intracelular (uM)
5     SL: sarcomere length (um)
6     """
7     # Parametros
8     Ca_50 = 1.0 # uM, [Ca] para 50% ativacao
9     n_Hill = 4 # Coeficiente de Hill
10    F_max = 100 # kPa, forca maxima
11    L_0 = 2.0 # um, comprimento otimo
12
13    # Dependencia de Ca2+
14    Ca_activation = Ca_i**n_Hill / (Ca_50**n_Hill + Ca_i**
15                                   n_Hill)
16
17    # Dependencia de comprimento (Frank-Starling)
18    if SL < 1.6:
19        length_dep = 0
20    elif SL < L_0:
21        length_dep = (SL - 1.6) / (L_0 - 1.6)
22    elif SL < 2.4:
23        length_dep = 1.0
24    else:
25        length_dep = 1.0 - 0.5*(SL - 2.4)
26
27    # Forca total
28    Force = F_max * Ca_activation * length_dep

```

```

28
29     return Force
30
31 # Exemplo
32 import numpy as np
33 import matplotlib.pyplot as plt
34
35 Ca_range = np.linspace(0.1, 10, 100)
36 SL_values = [1.8, 2.0, 2.2]
37
38 plt.figure(figsize=(10, 6))
39 for SL in SL_values:
40     F = [contraction_model(ca, SL) for ca in Ca_range]
41     plt.plot(Ca_range, F, linewidth=2, label=f'SL={SL}um')
42
43 plt.xlabel('[Ca2+]i (uM)')
44 plt.ylabel('Força (kPa)')
45 plt.legend()
46 plt.grid(True)

```

Listing 12.3: Modelo simplificado de contração

12.6 Propagação e Arritmias

Propagação da Onda Elétrica.

Teoria do Cabo (Cable Theory)

Propagação do potencial em tecido cardíaco:

$$\lambda^2 \frac{\partial^2 V}{\partial x^2} = V - V_{rest} + \tau \frac{\partial V}{\partial t}$$

onde:

λ : constante de comprimento espacial,

τ : constante de tempo e V : potencial transmembrana.

$$\frac{\partial V}{\partial t} = D \nabla^2 V - \frac{I_{ion}}{C_m}$$

D : coeficiente de difusão, ∇^2 : operador laplaciano e I_{ion} : correntes iônicas.

$$\theta = \lambda \sqrt{\frac{2}{\tau}}$$

Densidade de Na^+ channels, Resistência gap junctions, Geometria do tecido, Refratariedade e Isquemia, fibrose.

Mecanismos de Arritmias.

Classificação

-

Automaticidade anormal e Atividade deflagrada (DADs, EADs).

-

Bloqueios (AV, ramo) e Condução lenta.

-

Circuito anatômico e Circuito funcional (spiral waves).

— IMP:

Reentrada é o mecanismo mais comum de taquiarritmias sustentadas (FA, FV, TV)

Reentrada e Ondas Espirais.

Circuito de condução, Bloqueio unidirecional, Condução lenta e ERP < comprimento do circuito.

$$\lambda = \theta \times ERP$$

Condição de Reentrada

$$\text{Perímetro do circuito} > \lambda$$

4

Rotor: centro de rotação, Frequência alta, Sustentadas e Fibrilação. Fibrilação Atrial (FA).

Epidemiologia e Impacto

Arritmia sustentada mais comum, Prevalência: 1-2 Aumenta com idade (>10 Risco de AVC aumentado 5x e Insuficiência cardíaca.

focos ectópicos (veias pulmonares), remodelamento atrial, Multiple wavelets, Rotores e Fibrose.

I

$Ca, L,$

I

K^1 , APD encurtado, ERP reduzido e "AF begets AF".

.

Beta-bloqueadores, diltiazem

.

Antiarrítmicos (classe I, III)

Cardioversão

.

Isolamento de veias pulmonares

.

Prevenção de AVC

Modelagem

Modelos computacionais de FA ajudam a:

Identificar alvos de ablação, Prever resposta a drogas e Personalizar tratamento.

Fibrilação Ventricular (FV).

Emergência Médica

Causa mais comum de morte súbita cardíaca, Ritmo caótico, sem débito cardíaco efetivo, Fatal em minutos sem tratamento e Desfibrilação: único tratamento efetivo.

Múltiplas ondas espirais, Break-up de spiral waves, Dispersão de reentrância, Isquemia aguda e Cicatriz pós-infarto.

Infarto agudo do miocárdio, Cardiomiopatia, Canalopatias (Brugada, QTL), Drogas pró-arrítmicas e Distúrbios eletrolíticos.

5

Choque elétrico 200-360 J, Despolarização global, Reset do sistema e Permite retomada do ritmo sinusal.

Síndrome do QT Longo (LQTS).

Definição

Prolongamento do intervalo QT no ECG (>440 ms em homens, >460 ms em mulheres), associado a risco aumentado de torsades de pointes e morte súbita.

Drogas (antiarrítmicos, antibióticos), Distúrbios eletrolíticos (hipocalcemia) e Bradicardia.

APD prolongado, Dispersão de repolarização, Early afterdepolarizations (EADs) e Torsades de pointes.

6

Beta-bloqueadores, Evitar drogas que prolongam QT e CDI (cardio-desfibrilador implantável).

Efeitos de Drogas nos Canais Iônicos.

Classes de Antiarrítmicos (Classificação de Vaughan Williams)

Efeitos Desejados

Terminação de arritmias, Prevenção de recorrência, Controle de frequência e Melhora sintomática.

Efeitos Adversos

Pró-arritmia (paradoxal), Bradicardia, Bloqueio AV, Prolongamento QT e Insuficiência cardíaca.

— IMP:

Modelos computacionais são usados para prever efeitos cardiotóxicos de novas drogas (teste in silico)

12.7 Exercícios

Exercícios - Parte 1.

Questão 1: Análise de Potencial de Ação

Um PA ventricular tem as seguintes características:

$$V_{rest} = -85 \text{ mV}, V_{peak} = +30 \text{ mV} \text{ e } APD_{90} = 300 \text{ ms.}$$

Calcule a amplitude do PA, Se a frequência cardíaca é 75 bpm, qual a porcentagem do ciclo em que a célula está refratária? e Como uma droga que bloqueia I_{Kr} afetaria o APD?.

Questão 2: Concentrações Iônicas

Calcule o potencial de equilíbrio para Ca^{2+} a 37°C , dados:

$$[\text{Ca}^{2+}]_o = 2 \text{ mM e } [\text{Ca}^{2+}]_i = 0.1$$

$M.$

Use a equação de Nernst: $E = zF[X]_i = z \log_{10}[X]_i \text{ mV a } 37^\circ\text{C}$

Exercícios - Parte 2.

Questão 3: Condutância de Canais

Uma célula cardíaca tem $g_{Na} = 23 \text{ mS/cm}^2$. Durante o upstroke do PA:

$$V_m = 0 \text{ mV}, E_{Na} = +50 \text{ mV} \text{ e Ativação: } m^3h = 0.9$$

$$0.6 = 0.54.$$

Calcule a corrente I_{Na} (em

$$\text{A/cm}$$

²).

Questão 4: Acoplamento EC

Desenhe o fluxograma completo do acoplamento excitação-contração, Explique como uma redução de 50 e Por que bloqueadores de canal de Ca^{2+} são efetivos no tratamento de hipertensão?.

Questão 5: Comprimento de Onda de Reentrada

Se a velocidade de condução é

$$= 0.5\text{m}/\text{seg} \text{ ERP} = 250\text{ms} :$$

Calcule o comprimento de onda

eQual o perímetro mínimo de circuito para sustentar reentrada?.

Exercícios - Parte 3.

Questão 6: Projeto Computacional

Implemente um modelo simplificado de cardiomiócito ventricular:

Use o modelo fornecido nos slides (Noble ou LR simplificado), Simule um potencial de ação com estímulo em $t = 5 \text{ ms}$, Plote $V_m(t)$, $[\text{Ca}^{2+}]_i(t)$ e as principais correntes iônicas e Investigue o efeito de:.

Redução de 50 Aumento de 20 e Redução de 30

Para cada manipulação, calcule APD_{90} e compare com controle.

Questão 7: Análise Clínica

Um paciente apresenta:

FC: 180 bpm (taquicardia), Ritmo irregular, Ausência de ondas P e Complexo QRS estreito.

Qual o diagnóstico mais provável?, Explique o mecanismo fisiopatológico e Quais opções terapêuticas você consideraria?.

12.8 Notas Bibliográficas

Referências Principais.

99

Keener J., Sneyd J. (2009). Mathematical Physiology, 2nd ed. Springer.

Katz A.M. (2010). Physiology of the Heart, 5th ed. Lippincott Williams & Wilkins.

Noble D. (1962). A modification of the Hodgkin-Huxley equations applicable to Purkinje fibre action and pacemaker potentials. J Physiol.

Luo C.H., Rudy Y. (1991). A model of the ventricular cardiac action potential. *Circ Res*.

ten Tusscher K.H., Panfilov A.V. (2006). Alternans and spiral breakup in a human ventricular tissue model. *Am J Physiol Heart Circ Physiol*.

O'Hara T., Virág L., Varró A., Rudy Y. (2011). Simulation of the undiseased human cardiac ventricular action potential. *PLoS Comput Biol*.

Leitura Complementar.

Livros

Clayton R.H. et al. (2011). Models of cardiac tissue electrophysiology.

Prog Biophys Mol Biol, Trayanova N.A. (2011). Whole-heart modeling.

Circ Res, Zipes D.P., Jalife J. (2013). *Cardiac Electrophysiology: From*

Cell to Bedside, 6th ed e Klabunde R. (2011). *Cardiovascular Physiology*

Concepts, 2nd ed.

Recursos Online

-

-

-

(dados de ECG).

-

Obrigado!

Capítulo 12: Physiological Systems - Heart

Physiological Systems - Neuronal Dynamics

Dúvidas? Perguntas?

Capítulo 13

Medicina de Sistemas

13.1 Introdução

Visão Geral do Capítulo.

Objetivos de Aprendizagem

Compreender medicina de sistemas e medicina personalizada, Aplicar biologia de sistemas à descoberta de fármacos, Modelar doenças complexas como sistemas integrados, Explorar farmacogenômica e estratificação de pacientes e Analisar estudos de caso clínicos com abordagens sistêmicas.

Por que Systems Medicine?

Doenças complexas requerem abordagens integrativas, Medicina personalizada baseada em perfis moleculares, Descoberta de fármacos mais eficiente e segura e Previsão de respostas terapêuticas individualizadas.

Da Bancada ao Leito: Medicina de Sistemas.

Definição (Medicina de Sistemas).

Aplicação dos princípios da biologia de sistemas à medicina clínica, integrando dados multi-ômicos, redes moleculares e modelagem computacional para compreender, diagnosticar e tratar doenças.

!

Medicina Personalizada e de Precisão.

Medicina Tradicional

Abordagem "one-size-fits-all", Tratamento baseado em sintomas, Decisões empíricas e Resposta variável.

Medicina Personalizada

Tratamento individualizado, Baseada em perfil molecular, Decisões orientadas por dados e Resposta otimizada.

— IMP:

Medicina personalizada = tratamento certo + paciente certo + momento certo

Pilares da Medicina de Precisão

Genômica individual, Biomarcadores moleculares, Estratificação de pacientes e Farmacogenômica.

Abordagens Sistêmicas para Doenças.

Mudança de Paradigma

Aplicações Clínicas

Identificação de alvos terapêuticos, Reposicionamento de fármacos, Biomarcadores diagnósticos e prognósticos, Estratificação de pacientes em ensaios clínicos e Previsão de efeitos adversos.

13.2 Descoberta e Desenvolvimento de Fármacos

Descoberta de Fármacos: Tradicional vs Sistêmica.

Identificar alvo único, Triagem de compostos, Otimização de lead, Testes pré-clínicos e Ensaios clínicos.

Alta taxa de falha (>90 Custo elevado (\$2.6 bilhões), Tempo longo (10-15 anos) e Efeitos adversos inesperados.

Análise de redes, Identificação de módulos, Alvos múltiplos/sinérgicos, Modelagem in silico e Estratificação de pacientes.

Melhor compreensão de mecanismos, Previsão de efeitos adversos, Reposicionamento de fármacos e Medicina personalizada.

Identificação e Validação de Alvos.

Pipeline Sistêmico

-

Identificar genes/proteínas associados à doença, Análise de centralidade (degree, betweenness, closeness) e Identificar hubs e bottlenecks.

-

Deteção de comunidades, Módulos funcionalmente relacionados e Proximidade de rede.

-

Druggability (capacidade de ser alvo de fármaco), Especificidade tecidual, Papel funcional validado e Sem efeitos adversos graves.

Farmacologia de Redes (Network Pharmacology).

Definição (Network Pharmacology).

Estudo de interações fármaco-alvo no contexto de redes biológicas, reconhecendo que fármacos afetam múltiplos alvos e vias.

— IMP:

Polifarmacologia: um fármaco, múltiplos alvos

efeitos sinérgicos

Fármacos Multi-Alvo.

Conceito

Fármacos projetados para modular múltiplos alvos simultaneamente, explorando sinergias e redundâncias em redes biológicas.

Maior eficácia, Menor resistência, Efeitos sinérgicos e Adequado para doenças complexas.

Inibidores multi-quinase (câncer), Aspirina (COX-1/COX-2) e Estatinas (HMG-CoA + pleiotrópicos).

Design complexo, Otimização de múltiplos alvos, Previsão de efeitos adversos e Propriedades farmacocinéticas.

Índice de Seletividade

$$S = \frac{IC_{50}(\text{off-target})}{IC_{50}(\text{on-target})}$$

$S > 100$: seletivo

$S < 10$: multi-alvo

Redes de Interação Fármaco-Alvo.

Construção de Redes DTI

Estrutura química de fármacos, Estrutura 3D de proteínas alvo, Dados de afinidade (IC_{50} , K_d), Screening em larga escala e Bases de dados: DrugBank, ChEMBL, BindingDB.

Grau de promiscuidade de fármacos, Polifarmacologia, Reposicionamento e Previsão de efeitos adversos.

Métrica de Promiscuidade

$$P_d = \frac{\# \text{ alvos do fármaco}}{\# \text{ total de fármacos}}$$

Fármacos com P_d alto:

Maior chance de efeitos adversos e Potencial para reposicionamento.

Farmacocinética: Modelo ADME.

ADME

absorção: entrada do fármaco no organismo, distribuição: dispersão pelos tecidos, metabolismo: biotransformação (fígado) e excreção: eliminação (rins,

bile).

Propriedades físico-químicas, Solubilidade, permeabilidade, Ligação a proteínas plasmáticas, Metabolismo de primeira passagem e Polimorfismos genéticos (CYP450).

Regra de Lipinski (Regra dos 5)

Fármaco oralmente biodisponível:

Peso molecular < 500 Da, LogP < 5, Doadores H-bond

$$5eAceptoresH - bond$$

10.

Modelagem PK/PD.

Definição (Farmacocinética/Farmacodinâmica).

"O que o corpo faz com o fármaco— concentração vs tempo

"O que o fármaco faz com o corpo— efeito vs concentração

Modelos Compartmentais

Representam o corpo como compartimentos onde o fármaco se distribui.

Modelo de 1 compartimento (mais simples), Modelo de 2 compartimentos (central + periférico) e Modelo de 3+ compartimentos (tecidos específicos).

Parâmetros PK Importantes

C_{max} : concentração máxima, T_{max} : tempo de concentração máxima, AUC: área sob a curva (exposição total), $t_{1/2}$: meia-vida de eliminação, V_d : volume de distribuição e CL: clearance (depuração).

Modelo de Um Compartimento.

— DEST:

$$\frac{dC}{dt} = -k_e C$$

$$C(t) = C_0 e^{-k_e t}$$

$$t_{1/2} = \frac{\ln 2}{k_e} = \frac{0.693}{k_e}$$

$$CL = k_e \cdot V_d$$

$$AUC = \frac{C_0}{k_e} = \frac{\text{Dose}}{CL}$$

Modelo de Dois Compartimentos.

— DEST:

$$\frac{dC_1}{dt} = -(k_e + k_{12})C_1 + k_{21}C_2$$

$$\frac{dC_2}{dt} = k_{12}C_1 - k_{21}C_2$$

Simulação PK: Modelo de Um Compartimento.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def one_compartment_pk(C, t, ke):
6     """Modelo PK de 1 compartimento"""
7     dCdt = -ke * C
8     return dCdt
9
10 # Parametros
11 ke = 0.2          # constante de eliminacao (1/h)
12 C0 = 100          # concentracao inicial (mg/L)
13 t_half = 0.693 / ke # meia-vida
14
15 # Tempo
16 t = np.linspace(0, 24, 200)
17
18 # Resolver
19 C = odeint(one_compartment_pk, C0, t, args=(ke,))
20
21 # Plotar
22 plt.figure(figsize=(10, 6))
23 plt.plot(t, C, 'b-', linewidth=2, label='[Farmaco]')
24 plt.axhline(y=C0/2, color='r', linestyle='--',
25             label=f't1/2= {t_half:.2f}h')
26 plt.xlabel('Tempo (horas)')
27 plt.ylabel('Concentracao (mg/L)')
28 plt.title('Farmacocinetica: Modelo 1-Compartimento')
29 plt.legend()
30 plt.grid(True)

```

Listing 13.1: Farmacocinética monocompartimental

Ensaio Clínicos e Biologia de Sistemas.**Fases de Ensaio Clínicos**

Segurança, farmacocinética (20-100 voluntários), Eficácia preliminar, dose (100-500 pacientes), Eficácia, efeitos adversos (1000-5000 pacientes) e Monitoramento pós-comercialização.

Aplicações de Biologia de Sistemas

Estratificação de pacientes por perfil molecular, Identificação de marcadores de resposta/toxicidade, "Digital twins" para prever resultados, Ajustes baseados em dados intermediários e Identificar respondedores vs não-respondedores.

13.3 Doenças Complexas

Câncer como Doença Sistêmica.

Definição (Visão Sistêmica do Câncer).

Câncer não é apenas uma doença de um gene mutado, mas resulta de alterações em redes regulatórias, vias de sinalização e interações com o microambiente tumoral.

Heterogeneidade genômica, Plasticidade celular, Evolução clonal, Interações com estroma, Resposta imune e Metabolismo reprogramado.

Genético: mutações driver/passenger, Epigenético: metilação, histonas, Transcriptômico: expressão gênica, Proteômico: sinalização, Metabólico: Warburg, glutaminólise e Microambiente: células imunes, CAFs.

— IMP:

Abordagens de alvo único falham devido à redundância e plasticidade das redes tumorais

Hallmarks of Cancer.

10 Características Essenciais do Câncer (Hanahan & Weinberg)

Sinais de crescimento sustentado, Evasão de supressores de crescimento, Resistência à morte celular, Imortalidade replicativa, Angiogênese induzida e Invasão e metástase.

6

Reprogramação metabólica e Evasão imune.

8

Instabilidade genômica e Inflamação promotora de tumor.

Implicações Terapêuticas

Cada hallmark representa um alvo terapêutico potencial

Redes Oncogênicas e Supressoras de Tumor.

— IMP:

Câncer emerge do desequilíbrio entre oncogenes ativados e supressores inativados

Metabolismo do Câncer: Efeito Warburg.

Definição (Efeito Warburg).

Células cancerosas preferem glicólise aeróbica (menos eficiente) em vez de fosforilação oxidativa, mesmo na presença de oxigênio.

Geração rápida de ATP, Biomassa (nucleotídeos, lipídios), Acidificação do microambiente e Evasão de apoptose mitocondrial.

Reprogramação Metabólica

Captação de glicose (GLUT1),

Glicólise (HK2, PKM2),

Glutaminólise,

Síntese de lipídios (FASN) e

OXPHOS.

Alvos Metabólicos

Inibidores de hexoquinase, Inibidores de LDH-A e Inibidores de glutaminase.

Heterogeneidade Tumoral.

Tipos de Heterogeneidade

diferentes clones no mesmo tumor, variação entre pacientes, evolução durante tratamento e gradientes no microambiente.

— IMP:

Heterogeneidade

resistência terapêutica

necessidade de terapias combinadas

Doenças Cardiovasculares como Sistemas.

Redes em Doenças Cardiovasculares

Doenças cardiovasculares envolvem interações complexas entre:

Fatores genéticos (polimorfismos, mutações), Metabolismo lipídico (colesterol, triglicerídeos), Inflamação (citocinas, células imunes), Função endotelial (NO, vasodilatação), Coagulação (plaquetas, fatores de coagulação) e Pressão arterial (RAAS, SNS).

Acúmulo de LDL oxidado, Recrutamento de macrófagos, Formação de células foam, Placa aterosclerótica e Inflamação crônica.

Identificação de módulos de risco, Biomarcadores multi-ômicos, Scores de risco poligênico, Terapias multi-alvo e Medicina preventiva personalizada.

Diabetes e Síndrome Metabólica.

Diabetes Tipo 2: Doença Sistêmica

Resistência à insulina (músculo, fígado, tecido adiposo), Disfunção de células

pancreáticas, Metabolismo de glicose alterado, Lipotoxicidade, Inflamação crônica de baixo grau e Estresse

Obesidade abdominal, Hiperglicemia, Hipertensão, Dislipidemia e Estado pró-inflamatório.

Modelagem Sistêmica

Modelos de homeostase glicose-insulina, Redes de sinalização insulínica, Integração multi-órgão, Predição de progressão e Resposta a intervenções.

Doenças Neurodegenerativas.

Alzheimer

Placas amiloides (

– amiloide), Emaranhados neurofibrilares (tau), Neuroinflamação, Perda sináptica, Disfunção mito

APOE4 (genético), Idade e Fatores cardiovasculares.

Parkinson

Perda de neurônios dopaminérgicos, Corps de Lewy (

– sinucleína), Disfunção mitocondrial, Estresse do RE e Neuroinflamação.

SNCA, LRRK2, PARK7 e GBA, PINK1, PARKIN.

— IMP:

Doenças neurodegenerativas: falha em múltiplos processos celulares

cascata patológica

Módulos de Doença (Disease Modules).

Definição (Módulo de Doença).

Conjunto de genes/proteínas topologicamente próximos no interactoma humano cujas perturbações estão associadas a um fenótipo patológico comum.

Aplicações

Identificação de novos genes de doença, Compreensão de comorbidades (sobreposição de módulos), Reposicionamento de fármacos e Classificação molecular de doenças.

Classificação de Doenças Baseada em Redes.

Abordagem Tradicional vs Sistêmica

Exemplos

Classificação por perfil de expressão e mutações (The Cancer Genome Atlas - TCGA), Subtipos baseados em resistência à insulina vs disfunção de células

eEndotiposbaseadosemperfisdecitocinas.

— IMP:

Doenças com apresentação clínica similar podem ter mecanismos moleculares distintos

requeremtratamentosdiferentes

13.4 Medicina Personalizada

Farmacogenômica.

Definição (Farmacogenômica).

Estudo de como variações genéticas individuais influenciam a resposta a fármacos, incluindo eficácia e toxicidade.

Fontes de Variabilidade na Resposta a Fármacos

variantes em genes ADME (CYP450, transportadores), variantes em alvos terapêuticos (receptores, enzimas), heterogeneidade molecular da doença e dieta, comorbidades, interações medicamentosas.

Maximizar eficácia, Minimizar toxicidade, Otimizar dose e Selecionar fármaco adequado.

Benefícios

Medicina personalizada, Redução de eventos adversos, Economia de custos e Melhor adesão.

Polimorfismos do Citocromo P450.

Família CYP450

Enzimas hepáticas responsáveis pelo metabolismo de 75

25 warfarina, NSAIDs, hipoglicemiantes, clopidogrel, IBPs, antidepressivos e 50

Fenótipos Metabólicos

Exemplo: Clopidogrel e CYP2C19.

Pró-fármaco antiagregante, Previne trombose (pós-stent) e Requer ativação por CYP2C19.

metabolizador normal, perda de função e ganho de função.

— IMP:

PM têm

3xriscodeeventoscardiovasculares

PM: usar prasugrel ou ticagrelor (alternativas), EM: clopidogrel dose padrão e UM: monitorar cuidadosamente.

Descoberta de Biomarcadores.

Tipos de Biomarcadores

detectar presença de doença, prever evolução da doença, prever resposta a terapia, medir efeito do tratamento e detectar toxicidade.

Pipeline de Descoberta

análise ômica em larga escala (casos vs controles), validação em coorte independente, ensaios prospectivos e testes clínicos validados.

Assinaturas multi-gene, Perfis de proteínas/metabólitos e Integração multi-ômica.

Reprodutibilidade, Validação independente, Padronização de ensaios e Custo-benefício.

Estratificação de Pacientes.

Conceito

Classificar pacientes em subgrupos com base em características moleculares, clínicas ou demográficas para tratamento personalizado.

Clustering (K-means, hierárquico), Análise de componentes principais (PCA), Non-negative matrix factorization (NMF), Machine learning supervisionado e Network-based stratification.

Genômicos (mutações, CNVs), Transcriptômicos (expressão gênica), Proteômicos, Metabólicos e Clínicos.

Aplicações

enriquecer para respondedores, classificar subtipos, identificar alto vs baixo risco e selecionar tratamento ótimo.

— IMP:

Objetivo: maximizar benefício clínico, minimizar exposição desnecessária a toxicidade

Diagnósticos Baseados em Ômicas.

— Testes Multi-Ômicos Clínicos —

Sequenciamento de painel de genes (câncer), Whole-exome sequencing

(WES), Whole-genome sequencing (WGS) e Liquid biopsy (ctDNA).

Oncotype DX (câncer de mama), MammaPrint (prognóstico de mama) e Prosigna (PAM50).

Painéis de citocinas e Perfis metabólicos (diabetes, erros inatos).

Vantagens

Informação molecular detalhada

decisões terapêuticas precisas

Subtipos de Câncer de Mama.

Classificação Molecular (PAM50)

Baseada em perfil de expressão de 50 genes:

— IMP:

Mesmo órgão, histologia similar

diferentes doenças moleculares

tratamentos distintos

Implicação Clínica

Teste de expressão gênica determina:

Necessidade de quimioterapia adjuvante, Escolha de terapia-alvo (trastuzumab para HER2+) e Prognóstico e risco de recorrência.

Seleção de Tratamento por Perfil Molecular.

Exemplos Clínicos

erlotinib/gefitinib (câncer de pulmão), vemurafenib (melanoma), crizotinib (câncer de pulmão) e imatinib (leucemia mieloide crônica).

Digital Twins e Ensaios Clínicos In Silico.

Definição (Digital Twin).

Réplica computacional personalizada de um paciente individual, baseada em dados multi-ômicos, clínicos e fisiológicos, que permite simular respostas a intervenções terapêuticas.

Dados do paciente (genoma, transcriptoma, metaboloma, clínicos), Modelos mecanísticos (redes, EDOs), Parâmetros personalizados e Simulação computacional.

Predição de resposta a fármacos, Otimização de dose, Identificação de efeitos adversos e Testes de terapias combinadas.

Ensaios In Silico

Testar milhares de cenários rapidamente, Reduzir necessidade de ensaios animais, Acelerar desenvolvimento de fármacos e Stratificação pré-

clínica.

Desafios

Validação clínica, Complexidade dos modelos, Integração de dados e Incerteza paramétrica.

Desafios em Medicina Personalizada.

Desafios Técnicos

multi-ômicos heterogêneos, variantes de significado incerto (VUS), necessidade de estudos prospectivos, protocolos laboratoriais e análise e sequenciamento e análise ainda caros.

Desafios Éticos e Sociais

dados genômicos sensíveis, uso de dados, achados incidentais, acesso desigual a tecnologias, genética por seguradoras/empregadores e médicos e pacientes precisam compreender resultados.

— IMP:

Medicina personalizada requer framework regulatório, ético e de treinamento profissional adequado

13.5 Aplicações Clínicas

Estudo de Caso: Câncer de Mama HER2+ e Trastuzumab.

Contexto Clínico

15-20 HER2 (ERBB2): receptor tirosina-quinase, Superexpressão

proliferação descontrolada e Prognóstico pior (antes de terapias – alvo).

Anticorpo monoclonal anti-HER2, Bloqueia sinalização HER2, Induz resposta imune (ADCC) e Primeiro sucesso de medicina personalizada em oncologia.

50 Sobrevida global melhorada e Uso em adjuvância e metastático.

— IMP:

Teste HER2 é mandatório em câncer de mama

decisão terapêutica

Doenças Infecciosas: Abordagem Sistêmica.

— Interações Hospedeiro-Patógeno —

Visão sistêmica considera:

Redes moleculares do patógeno, Resposta imune do hospedeiro, Interações proteína-proteína (patógeno-hospedeiro), Modulação de vias do hospedeiro por patógeno e Evolução e adaptação do patógeno.

Identificação de alvos terapêuticos (essenciais para patógeno), Reposicionamento de fármacos, Previsão de virulência, Assinaturas de resposta imune e Estratégias de vacinação.

Exemplo: Tuberculose

Rede metabólica de *M. tuberculosis*, Identificar enzimas essenciais, Testar in silico inibidores, Prever resistência a fármacos e Biomarcadores de progressão latente

ativa.

Resistência a Antibióticos como Problema Sistêmico.

Mecanismos de Resistência

Mutações em alvos (RNA polimerase, ribossomo), Bombas de efluxo (superexpressão), Enzimas inativadoras (

—lactamases), Modificação de alvos (metilação de RNA ribossomal) e Biofilmes (tolerância).

Análise de redes de resistência, Identificar vulnerabilidades colaterais, Terapias combinadas racionais, Predição de evolução de resistência e Desenvolvimento de novos antibióticos.

Estratégias

resistência a A causa sensibilidade a B, reduzir pressão seletiva, virulência sem morte bacteriana e fortalecer resposta do hospedeiro.

— IMP:

Resistência antimicrobiana é uma das maiores ameaças à saúde global
Imunoterapia e Imunologia de Sistemas.

Imunoterapia do Câncer

Estratégias que ativam o sistema imune para eliminar células tumorais: anti-PD-1, anti-CTLA-4 (bloqueiam sinais inibitórios), células T geneticamente modificadas, antígenos tumorais e vírus que infectam e lisam tumor.

Apenas 20-40 Efeitos adversos (autoimunidade), Microambiente imunossupressor e Heterogeneidade tumoral.

Perfil de infiltrado imune (hot vs cold tumors), Assinaturas de resposta, Carga mutacional tumoral (TMB), Expressão de PD-L1 e Modelagem de resposta imune.

— IMP:

Biomarcadores preditivos são essenciais para selecionar pacientes que se beneficiarão

Terapia com Células CAR-T.

Conceito

Células T do paciente são geneticamente modificadas para expressar receptor de antígeno quimérico (CAR) que reconhece antígeno tumoral específico.

!

0

Leucemias/linfomas B (CD19), Taxa de remissão completa >80 e FDA aprovado (Kymriah, Yescarta).

Dinâmica CAR-T vs tumor, Predição de síndrome de liberação de citocinas (CRS), Otimização de dose e Design de CARs de próxima geração.

Design de Vacinas: Abordagem Sistêmica.

Vacinologia de Sistemas

Uso de abordagens ômicas e modelagem para:

Identificar antígenos imunogênicos e protetores, Prever resposta imune a candidatos vacinais, Otimizar formulações e adjuvantes, Estratificar respondedores vs não-respondedores e Compreender correlatos de proteção.

Genômica de patógeno, Predição de epítomos (MHC binding), Análise de conservação, Teste in silico de imunogenicidade, Validação experimental e Ensaios clínicos.

— Exemplo: Vacinas mRNA (COVID-19) —

Análise genômica rápida do SARS-CoV-2, Identificação da proteína Spike (S), Design de mRNA otimizado, Predição de resposta imune e Desenvolvimento em tempo recorde (<1 ano).

— IMP:

Biologia de sistemas acelerou desenvolvimento de vacinas COVID-19 COVID-19 e Resposta de Biologia de Sistemas.

Contribuições de Systems Biology na Pandemia

sequenciamento rápido, rastreamento de variantes, análise de redes identificou candidatos (remdesivir, dexametasona), perfis de citocinas, assinaturas de gravidade (tempestade de citocinas), mapa de interações SARS-CoV-2-humano, previsão de disseminação, impacto de intervenções e design racional (mRNA, vetores virais).

Biomarcadores de Gravidade

IL-6, D-dímero, ferritina

em casos graves, Linfopenia, neutrofilia e Perfi transcriptômicos distinguem progressão.

Exemplo: Estratificação de Pacientes com Clustering.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.cluster import KMeans
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.decomposition import PCA
6
7 # Dados simulados: pacientes x features (ômicas + clínicas)
8 np.random.seed(42)
9 n_patients = 100
10 data = np.random.randn(n_patients, 10) # 10 features
11
12 # Normalizar
13 scaler = StandardScaler()
14 data_scaled = scaler.fit_transform(data)
15
16 # Clustering K-means
17 kmeans = KMeans(n_clusters=3, random_state=42)
18 clusters = kmeans.fit_predict(data_scaled)
19
20 # PCA para visualizacao
21 pca = PCA(n_components=2)
22 data_pca = pca.transform(data_scaled)
23
24 # Plotar
25 plt.figure(figsize=(8, 6))
26 scatter = plt.scatter(data_pca[:, 0], data_pca[:, 1],
27                       c=clusters, cmap='viridis', s=50)
28 plt.xlabel('PC1')
29 plt.ylabel('PC2')
30 plt.title('Estratificacao de Pacientes (K-means)')
31 plt.colorbar(scatter, label='Cluster')
32 plt.legend(['Subgrupo 1', 'Subgrupo 2', 'Subgrupo 3'])

```

Listing 13.2: Análise de clusters em dados clínicos

13.6 Exercícios

Exercícios - Parte 1: Farmacocinética.

Questão 1: Modelo PK

Um paciente recebe dose IV bolus de 500 mg de um fármaco. O volume de distribuição é $V_d = 50$ L e a meia-vida é $t_{1/2} = 4$ h.

Calcule a constante de eliminação k_e , Calcule o clearance (CL), Qual será a concentração após 8 horas? e Calcule a AUC (área sob a curva).

Questão 2: Modelo de Dois Compartimentos

Implemente um modelo farmacocinético de dois compartimentos em Python e simule a concentração ao longo de 24 horas. Use os seguintes parâmetros:

$k_e = 0.15 \text{ h}^{-1}$, $k_{12} = 0.3 \text{ h}^{-1}$, $k_{21} = 0.2 \text{ h}^{-1}$ e Dose = 1000 mg IV, $C_1(0) = 20 \text{ mg/L}$, $C_2(0) = 0$.

Exercícios - Parte 2: Redes de Doenças.**Questão 3: Análise de Rede DTI**

Dado o seguinte grafo bipartido de interações fármaco-alvo:

Fármaco A: alvos T1, T2, T3, Fármaco B: alvos T2, T4 e Fármaco C: alvos T3, T4, T5.

Construa a matriz de adjacência, Calcule o grau de promiscuidade de cada fármaco, Identifique alvos compartilhados (potencial para efeitos adversos similares) e Sugira reposicionamento de fármacos baseado em sobreposição de alvos.

Questão 4: Módulos de Doença

Use algoritmo de Louvain ou Girvan-Newman para detectar módulos em uma rede de interações proteína-proteína associada a doença cardiovascular. Interprete os módulos funcionalmente.

Exercícios - Parte 3: Medicina Personalizada.**Questão 5: Farmacogenômica**

Um paciente é metabolizador pobre (PM) de CYP2D6 e recebe codeína (pró-fármaco convertido em morfina por CYP2D6).

Preveja o efeito no manejo da dor, Qual ajuste de dose ou medicação alternativa você recomendaria? e Como você explicaria ao paciente a importância do teste farmacogenômico?.

Questão 6: Estratificação de Pacientes

Implemente clustering K-means em um dataset de expressão gênica de câncer de mama (use dataset público como TCGA ou simulado). Identifique subgrupos moleculares e compare com subtipos clínicos conhecidos (Luminal A/B, HER2+, Basal).

Projeto Prático: Análise de Dados Clínicos Ômicos.**Projeto Integrador**

Analisar dados multi-ômicos de pacientes com câncer para identificar biomarcadores e estratificar pacientes.

Baixar dados de expressão gênica e clínicos (TCGA, GEO, cBioPortal), Normalização, controle de qualidade, filtro de variância, PCA, heatmaps, clustering hierárquico, K-means ou NMF para identificar subgrupos, Análise diferencial (DESeq2, limma) entre subgrupos, Curvas de Kaplan-Meier, teste log-rank, Dataset independente e Enriquecimento funcional (GO, KEGG).

Relatório com código Python/R, figuras e interpretação clínica

13.7 Notas Bibliográficas

Referências Principais.

99

Barabási A.L., Gulbahce N., Loscalzo J. (2011). Network medicine: a network-based approach to human disease. *Nature Reviews Genetics* 12:56-68.

Auffray C., Chen Z., Hood L. (2009). Systems medicine: the future of medical genomics and healthcare. *Genome Medicine* 1:2.

Schadt E.E. (2009). Molecular networks as sensors and drivers of common human diseases. *Nature* 461:218-223.

Zhou X., Menche J., Barabási A.L., Sharma A. (2014). Human symptoms-disease network. *Nature Communications* 5:4212.

Hanahan D., Weinberg R.A. (2011). Hallmarks of cancer: the next generation. *Cell* 144:646-674.

Referências: Network Medicine e Farmacologia.

99

Hopkins A.L. (2008). Network pharmacology: the next paradigm in drug discovery. *Nature Chemical Biology* 4:682-690.

Zhao S., Iyengar R. (2012). Systems pharmacology: network analysis to identify multiscale mechanisms of drug action. *Annual Review of Pharmacology and Toxicology* 52:505-521.

Csermely P., Korcsmáros T., Kiss H.J., et al. (2013). Structure and dynamics of molecular networks: a novel paradigm of drug discovery. *Pharmacology & Therapeutics* 138:333-408.

Menche J., Sharma A., Kitsak M., et al. (2015). Disease networks: uncovering disease-disease relationships through the incomplete interactome. *Science* 347:1257601.

Referências: Medicina Personalizada e Ômicas.

99

Ashley E.A. (2016). Towards precision medicine. *Nature Reviews Genetics* 17:507-522.

Roden D.M., McLeod H.L., Relling M.V., et al. (2019). Pharmacogenomics. *The Lancet* 394:521-532.

Chen R., Snyder M. (2013). Promise of personalized omics to precision medicine. *Wiley Interdisciplinary Reviews: Systems Biology and Medicine* 5:73-82.

Brunet J.P., et al. (2004). Metagenes and molecular pattern discovery using matrix factorization. *PNAS* 101:4164-4169.

Bases de Dados e Recursos.

Bases de Dados Clínicas e Farmacogenômicas

— fármacos, alvos, PK/PD, — dados multi-ômicos de câncer, — dados de saúde populacional, — variantes clínicas e — farmacogenômica.

Ferramentas de Análise

— análise de dados de câncer, — interações proteína-proteína, — visualização de redes e — enriquecimento funcional.

Leitura Complementar.

Livros e Reviews

Loscalzo J., Barabási A.L., Silverman E.K. (2017). *Network Medicine: Complex Systems in Human Disease and Therapeutics*. Harvard University Press, Zhu H., Xia M. (2016). *Advances in Systems Biology*. Springer e Wolkenhauer O. (2013). *Systems Medicine*. Springer.

Artigos de Revisão Importantes

Hood L., Friend S.H. (2011). Predictive, personalized, preventive, participatory (P4) cancer medicine. *Nature Reviews Clinical Oncology* 8:184-187, Ideker T., Krogan N.J. (2012). Differential network biology. *Molecular Systems Biology* 8:565 e Califano A., Alvarez M.J. (2017). The recurrent architecture of tumour initiation, progression and drug sensitivity. *Nature Reviews Cancer* 17:116-130.

Obrigado!

Capítulo 13: Systems Biology in Medicine

Descoberta de fármacos

Doenças complexas

Medicina personalizada

Aplicações clínicas

Dúvidas? Perguntas?

A medicina do futuro é sistêmica, preditiva, personalizada e participativa

Capítulo 14

Design de Sistemas Biológicos

14.1 Introdução

Visão Geral do Capítulo.

Objetivos de Aprendizagem

Compreender princípios de engenharia aplicados à biologia, Projetar e analisar circuitos genéticos sintéticos, Aplicar engenharia metabólica para produção de compostos, Utilizar ferramentas computacionais de design e Entender desafios e perspectivas da biologia sintética.

Biologia Sintética

Aplicação de princípios de engenharia para projetar e construir novos sistemas biológicos ou redesenhar sistemas naturais existentes.

O que é Biologia Sintética?.

Definição (Biologia Sintética).

Disciplina que combina biologia, engenharia e ciência da computação para projetar e construir novas funções e sistemas biológicos que não existem na natureza.

Modularidade, Padronização, Abstração, Design racional e Otimização.

Biocombustíveis, Fármacos, Biosensores, Biomateriais, Terapia celular e Bioremediação.

Ciclo Design-Build-Test-Learn.

— IMP:

Processo iterativo: cada ciclo refina o design com base nos dados experimentais

Ética e Biossegurança.

Considerações Éticas

Organismos geneticamente modificados (OGMs), Impacto ambiental,

Uso dual (militar/civil), Bioterrorismo e Propriedade intelectual.

Níveis de Biossegurança

risco mínimo, risco moderado, alto risco e risco extremo.

Regulação

Protocolos de contenção, monitoramento e aprovação ética

14.2 Circuitos Genéticos Sintéticos

Circuitos Genéticos como Dispositivos Computacionais.

Analogia com Circuitos Eletrônicos

Circuitos genéticos processam sinais bioquímicos assim como circuitos eletrônicos processam sinais elétricos

— IMP:

Princípios de design modular: componentes intercambiáveis e reutilizáveis

BioBricks: Partes Biológicas Padronizadas.

Componentes Básicos

controla transcrição, Ribosome Binding Site, Coding Sequence (gene) e fim da transcrição.

BioBrick Assembly, Gibson Assembly e Golden Gate.

Registry of Standard Biological Parts

Toggle Switch: Interruptor Genético.

Princípio

Dois repressores que se inibem mutuamente

Gene A reprime gene B, Gene B reprime gene A e Sistema bistável.

Estado 1: A alto, B baixo e Estado 2: A baixo, B alto.

— IMP:

Primeiro circuito genético sintético com memória bistável

Modelo Matemático do Toggle Switch.

Sistema de EDOs

$$\begin{aligned}\frac{du}{dt} &= \frac{\alpha_1}{1 + v^\beta} - u \\ \frac{dv}{dt} &= \frac{\alpha_2}{1 + u^\gamma} - v\end{aligned}$$

onde:

u, v = concentrações dos repressores, α_1, α_2 = taxas de produção máximas e γ, β = coeficientes de Hill (cooperatividade).

— DEST:

$$\alpha_1 \alpha_2 > (+1)(+1)$$

Análise de Bistabilidade.

$$u = \frac{\alpha_1}{1 + v^\beta}, \quad v = \frac{\alpha_2}{1 + u^\gamma}$$

Jacobiano, Autovalores e Diagrama de bifurcação.

Resultado

Sistema pode ter 1 ou 3 pontos de equilíbrio dependendo dos parâmetros

$$t = 0 \text{ e } t = 0$$

Repressilator: Oscilador Genético.

Princípio

Três genes que se reprimem em cascata:

Gene reprime, Gene reprime e Gene reprime.

Oscilações periódicas, Período

$$2 - 3 \text{ horase Visualização com GFP.}$$

— IMP:

Primeiro oscilador genético sintético

Modelo Matemático do Repressilator.

Sistema de EDOs

$$\begin{aligned} \frac{dm_i}{dt} &= -m_i + \frac{\alpha}{1 + p_j^n} + \alpha_0 \\ \frac{dp_i}{dt} &= -\beta(p_i - m_i) \end{aligned}$$

onde $i = 1, 2, 3$ (lacI, tetR, cI) e $j = i - 1$

m_i = concentração de mRNA do gene i , p_i = concentração de proteína do gene i ,

$$= \text{taxa de transcrição máxima,}$$

n = coeficiente de Hill = razão de degradação de proteína / mRNA.

Repressão forte (n grande) e Tempo de resposta adequado (

apropriado).

Feed-Forward Loops (FFLs).

Motivo de Rede

Padrão comum em redes regulatórias:

Gene X regula Y, Gene X regula Z e Gene Y regula Z.

Coherent FFL (C-FFL) e Incoherent FFL (I-FFL).

Filtro de ruído, Detector de pulso e Acelerador de resposta.

Portas Lógicas Genéticas.

Implementação Biológica

dois promotores necessários, promotores alternativos, repressor e dois repressores.

Biosensores e Repórteres.

Biosensor

Sistema que detecta sinais químicos/físicos e produz resposta mensurável

Domínio sensor (receptor), Domínio de sinal e Gene repórter.

GFP (fluorescência verde), RFP (fluorescência vermelha), Luciferase (bioluminescência) e LacZ (

–galactosidase).

Biosensor para detecção de arsênico em água usando promotor ars + GFP

Simulação: Toggle Switch em Python.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import odeint
4
5 def toggle_switch(y, t, alpha1, alpha2, beta, gamma):
6     """Modelo do toggle switch"""
7     u, v = y
8     dudt = alpha1 / (1 + v**beta) - u
9     dvdt = alpha2 / (1 + u**gamma) - v
10    return [dudt, dvdt]
11
12 # Parametros para sistema bistavel
13 alpha1 = 156.25
14 alpha2 = 15.6
15 beta = 2.5
16 gamma = 1.0
17
18 # Tempo
19 t = np.linspace(0, 50, 1000)
20
21 # Duas condicoes iniciais diferentes
22 y0_1 = [0.1, 5.0] # Estado 1
23 y0_2 = [5.0, 0.1] # Estado 2
24
25 # Resolver
26 sol1 = odeint(toggle_switch, y0_1, t, args=(alpha1, alpha2, beta, gamma))
27 sol2 = odeint(toggle_switch, y0_2, t, args=(alpha1, alpha2, beta, gamma))
28
29 # Plotar
30 plt.figure(figsize=(10, 6))
31 plt.plot(t, sol1[:, 0], 'b-', label='u(Estado 1)', linewidth=2)
```



```

32 plt.plot(t, sol1[:, 1], 'r-', label='v(Estado1)', linewidth=2)
33 plt.plot(t, sol2[:, 0], 'b--', label='u(Estado2)', linewidth=2)
34 plt.plot(t, sol2[:, 1], 'r--', label='v(Estado2)', linewidth=2)
35 plt.xlabel('Tempo')
36 plt.ylabel('Concentracao')
37 plt.legend()
38 plt.title('ToggleSwitch: Bistabilidade')

```

Listing 14.1: Modelo do toggle switch

Simulação: Repressilator em Python.

```

1 def repressilator(y, t, alpha, alpha0, beta, n):
2     """Modelo do repressilator (3 genes)"""
3     m1, p1, m2, p2, m3, p3 = y
4
5     # mRNA equations
6     dm1dt = -m1 + alpha / (1 + p3**n) + alpha0
7     dm2dt = -m2 + alpha / (1 + p1**n) + alpha0
8     dm3dt = -m3 + alpha / (1 + p2**n) + alpha0
9
10    # Protein equations
11    dp1dt = -beta * (p1 - m1)
12    dp2dt = -beta * (p2 - m2)
13    dp3dt = -beta * (p3 - m3)
14
15    return [dm1dt, dp1dt, dm2dt, dp2dt, dm3dt, dp3dt]
16
17 # Parametros
18 alpha = 216.0
19 alpha0 = 0.0
20 beta = 0.2
21 n = 2.0
22
23 # Condiçoes iniciais
24 y0 = [0.5, 1.0, 1.5, 0.5, 2.0, 1.0]
25 t = np.linspace(0, 500, 5000)
26
27 # Resolver
28 sol = odeint(repressilator, y0, t, args=(alpha, alpha0, beta, n))
29
30 # Plotar
31 plt.figure(figsize=(12, 6))
32 plt.subplot(1, 2, 1)
33 plt.plot(t, sol[:, 1], 'g-', label='Proteína1', linewidth=2)
34 plt.plot(t, sol[:, 3], 'orange', label='Proteína2', linewidth=2)
35 plt.plot(t, sol[:, 5], 'purple', label='Proteína3', linewidth=2)
36 plt.xlabel('Tempo')
37 plt.ylabel('Concentracao')
38 plt.legend()
39 plt.title('Repressilator: Series Temporais')
40
41 plt.subplot(1, 2, 2)
42 plt.plot(sol[:, 1], sol[:, 3], 'b-', linewidth=1)
43 plt.xlabel('Proteína1')
44 plt.ylabel('Proteína2')
45 plt.title('Retrato de Fase')

```

Listing 14.2: Modelo do repressilator

14.3 Engenharia Metabólica

Engenharia de Vias Metabólicas.

Definição (Engenharia Metabólica).

Modificação direcionada de vias metabólicas usando tecnologia de DNA recombinante para melhorar a produção de compostos de interesse.

Aumentar rendimento, Produzir novos compostos, Remover subprodutos, Melhorar eficiência e Reduzir custos.

Biocombustíveis (etanol, biodiesel), Fármacos (artemisinina, taxol), Aminoácidos, Vitaminas e Polímeros biodegradáveis.

— IMP:

Combina biologia molecular, bioquímica e modelagem computacional
Estratégias de Engenharia Metabólica.

1. Overexpression (Superexpressão)

Aumentar expressão de enzimas limitantes da via

Promotores fortes, Múltiplas cópias do gene e Otimização de códons.

2. Gene Knockout (Deleção)

Eliminar vias competidoras ou indesejáveis

CRISPR-Cas9, Recombinação homóloga e Direcionar fluxo para via de interesse.

3. Heterologous Expression (Expressão Heteróloga)

Introduzir genes de outros organismos

Criar novas vias e Produzir compostos não-nativos.

Estratégia Push-Pull.

Estratégia

aumentar fluxo entrando na via, aumentar conversão para produto final e eliminar vias competidoras.

Balanceamento de Cofatores.

Problema

Muitas reações requerem cofatores (NADH, NADPH, ATP)

Disponibilidade limitada, Necessário balanço entre vias e Afeta rendimento máximo teórico.

0

— IMP:

Engenharia de cofatores: modificar especificidade de enzimas (NADH

NADPH)

Exemplo: Produção de Artemisinina.

Contexto

Antimalárico potente, Extraído de *Artemisia annua*, Rendimento baixo
(

1 e *Altocusto*.

Via heteróloga em *E. coli*, Via estendida em levedura, Expressão de genes de planta e Otimização metabólica.

Produção 25 g/L e Redução de custo 90

!

1

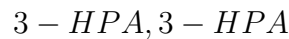
Exemplo: Produção de 1,3-Propanodiol.

Aplicação

Monômero para polímeros (PTT)

Carpets, plásticos, Produção tradicional: petroquímica e Alternativa: fermentação.

Glicerol



1,3-PDO, Genes de *Klebsiella* e Expressão em *E. coli*.

Produção industrial, Rendimento 135 g/L e Processo econômico.

2

— IMP:

Exemplo de sucesso comercial da engenharia metabólica

Análise de Balanço de Fluxo (FBA).

Flux Balance Analysis

Método computacional para prever fluxos metabólicos

Baseado em estequiometria, Estado estacionário e Otimização linear.

— DEST:

$$c^T v$$

$$Sv = 0$$

$$v_{min} \leq v \leq v_{max}$$

onde:

S = matriz estequiométrica, v = vetor de fluxos e c = coeficientes da função objetivo (ex: biomassa).

Identificação de Alvos Genéticos.

Métodos

prever fluxos, análise de variabilidade, minimização de ajuste metabólico e identificar knockouts ótimos.

Problema bi-nível, Acoplar crescimento e produção e Identificar deleções sinérgicas.

3

Engenharia de Cofatores.

Estratégias

Modificar especificidade NADH/NADPH, Superexpressar transidrogenases, Redirecionar fluxo de cofatores e Usar vias alternativas.

Necessário para biossíntese, Via das pentoses (PPP), Enzima málica e Transidrogenase PntAB.

4

— IMP:

Balanco de cofatores é crítico para rendimento máximo

Regulação Dinâmica de Vias.

Controle Temporal

Expressão gênica responsiva a sinais metabólicos

Promotores induzíveis, Biosensores de metabólitos, Controle de feed-back e Evitar toxicidade de intermediários.

5

Fase 1: crescimento celular (promotor constitutivo)

Fase 2: produção (promotor induzível por IPTG ou temperatura)

Engenharia Metabólica com COBRApy.

```
1 import cobra
2 from cobra import Model, Reaction, Metabolite
3
4 # Carregar modelo de E. coli
5 model = cobra.io.load_json_model("e_coli_core.json")
6
7 # Realizar FBA
8 solution = model.optimize()
9 print(f"Taxa de crescimento: {solution.objective_value:.3f} h-1")
10
11 # Analisar fluxos
12 print("\nFluxos principais:")
13 for rxn_id in ['BIOMASS_Ecoli_core_w_GAM', 'EX_glc_D_e', 'EX_o2_e']:
14     rxn = model.reactions.get_by_id(rxn_id)
15     print(f"{rxn.name}: {solution.fluxes[rxn_id]:.3f}")
16
17 # Simular knockout
18 model_ko = model.copy()
19 model_ko.reactions.PGI.knock_out() # Knockout phosphoglucose isomerase
20
21 solution_ko = model_ko.optimize()
22 print(f"\nCrescimento pos-knockout: {solution_ko.objective_value:.3f} h-1")
23
24 # Analise de variabilidade de fluxo (FVA)
25 from cobra.flux_analysis import flux_variability_analysis
26
27 fva_result = flux_variability_analysis(model, reaction_list=['PGI', 'PFK'])
28 print("\nFVA:")
29 print(fva_result)
30
31 # Identificar genes essenciais
32 from cobra.flux_analysis import single_gene_deletion
```

```

33
34 deletion_results = single_gene_deletion(model)
35 essential_genes = deletion_results[deletion_results['growth'] < 0.01]
36 print(f"\nNúmero de genes essenciais: {len(essential_genes)}")

```

Listing 14.3: Análise FBA com COBRApy

FSEOF: Flux Scanning com Objetivo Forçado.

```

1 def fseof_analysis(model, target_reaction, biomass_fraction=0.1):
2     """
3     Flux Scanning based on Enforced Objective Flux
4     Identifica reacoes para superexpressao/delecao
5     """
6     import pandas as pd
7
8     # Forcar producao minima de biomassa
9     biomass_rxn = model.reactions.BIOMASS_Ecoli_core_w_GAM
10    max_biomass = model.optimize().objective_value
11    biomass_rxn.bounds = (biomass_fraction * max_biomass, 1000)
12
13    # Maximizar produto alvo
14    model.objective = target_reaction
15
16    results = []
17
18    # Escanear cada reacao
19    for rxn in model.reactions:
20        if rxn.id == target_reaction.id:
21            continue
22
23        # Estado original
24        original_bounds = rxn.bounds
25
26        # Testar knockout
27        rxn.bounds = (0, 0)
28        sol_ko = model.optimize()
29        flux_ko = sol_ko.objective_value if sol_ko.status == 'optimal' else 0
30
31        # Testar superexpressao (aumentar limite superior)
32        rxn.bounds = (original_bounds[0], original_bounds[1] * 10)
33        sol_oe = model.optimize()
34        flux_oe = sol_oe.objective_value if sol_oe.status == 'optimal' else 0
35
36        # Restaurar
37        rxn.bounds = original_bounds
38
39        results.append({
40            'reaction': rxn.id,
41            'knockout_effect': flux_ko,
42            'overexpression_effect': flux_oe
43        })
44
45    return pd.DataFrame(results)
46
47    # Executar FSEOF
48    target = model.reactions.EX_succ_e # Succinato como produto alvo
49    fseof_results = fseof_analysis(model, target)
50
51    # Identificar alvos promissores
52    print("Melhores alvos para superexpressao:")
53    print(fseof_results.nlargest(5, 'overexpression_effect'))

```

Listing 14.4: FSEOF para engenharia metabólica

14.4 Ferramentas de Design e Modelagem

Ferramentas CAD para Biologia Sintética.

— Computer-Aided Design (CAD) —

Software para projetar circuitos e vias biológicas

design automático de circuitos, montagem de DNA, linguagem de design, editor visual e modelagem e simulação.

Design de construtos, Predição de comportamento, Otimização de parâmetros, Seleção de partes e Planejamento de montagem.

Cello

Entrada: especificação lógica (Verilog)

Saída: sequência de DNA otimizada

Caracterização e Padronização de Partes.

Caracterização

Medição quantitativa de propriedades de partes biológicas

Força de promotor (PoPS - Polymerase per Second), Eficiência de RBS (taxa de tradução), Taxa de degradação e Resposta a indutores.

20,000 partes, Caracterização padronizada, Compartilhamento comunitário e iGEM competition.

6

SBOL: Synthetic Biology Open Language.

Definição (SBOL).

Formato de dados padronizado para representar designs de biologia sintética

Interoperabilidade entre ferramentas, Compartilhamento de designs, Documentação estruturada e Reprodutibilidade.

ComponentDefinition, ModuleDefinition, Sequence e Interaction.

!

7

— IMP:

Padrão aceito pela comunidade para intercâmbio de dados

Frameworks de Modelagem.

Modelos Determinísticos

sistemas contínuos, FBA e Ferramentas: COPASI, Tellurium.

Modelos Estocásticos

simulação exata, aproximado e Para baixo número de moléculas.

Modelos Espaciais

difusão e reação, células individuais e Virtual Cell.

— Modelos Multi-escala —

Integrar múltiplos níveis, Molecular

Celular

População e CompuCell3D.

Escolha do Modelo

Depende da escala, número de moléculas e questão biológica

Algoritmos de Otimização para Design.

OptKnock

Identifica knockouts genéticos ótimos

Problema bi-nível, Otimização mista inteira (MILP) e Acopla crescimento e produção.

OptGene

Extensão usando algoritmos genéticos

Busca heurística, Múltiplos objetivos e Mais flexível que OptKnock.

OptForce

Identifica intervenções mínimas

Superexpressão e knockout e Garante produção mínima.

Machine Learning para Predição de Partes.

Aplicações de ML

Predição de força de promotor, Otimização de RBS, Predição de estrutura de RNA, Design de proteínas e Predição de interações.

Regressão, Redes neurais, Random forests e Deep learning.

8

— IMP:

ML acelera ciclo de design-build-test-learn

Simulação Estocástica: Algoritmo de Gillespie.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def gillespie_ssa(reactions, initial_state, tmax):
5     """
6     Gillespie Stochastic Simulation Algorithm
7     reactions: lista de (propensity_func, stoichiometry_change)
8     """
9     state = np.array(initial_state, dtype=float)
10    time = 0
11    times = [time]
12    states = [state.copy()]
13
14    while time < tmax:
15        # Calcular propensidades
16        propensities = np.array([prop(state) for prop, _ in reactions])
17        a0 = propensities.sum()
18
19        if a0 == 0:
20            break
21
22        # Tempo ate proxima reacao
23        tau = np.random.exponential(1/a0)

```

```

24
25     # Escolher reacao
26     r = np.random.random() * a0
27     cumsum = 0
28     for i, a in enumerate(propensities):
29         cumsum += a
30         if cumsum >= r:
31             _, change = reactions[i]
32             state += change
33             break
34
35     time += tau
36     times.append(time)
37     states.append(state.copy())
38
39     return np.array(times), np.array(states)
40
41 # Exemplo: producao e degradacao de proteina
42 k_prod = 10.0 # taxa de producao
43 k_deg = 0.5   # taxa de degradacao
44
45 reactions = [
46     (lambda s: k_prod, np.array([1])),      # Producao
47     (lambda s: k_deg * s[0], np.array([-1])) # Degradacao
48 ]
49
50 times, states = gillespie_ssa(reactions, [0], tmax=20)
51 plt.plot(times, states[:, 0])
52 plt.xlabel('Tempo')
53 plt.ylabel('Numero de moléculas')

```

Listing 14.5: Gillespie SSA para circuitos genéticos

Otimização de Vias com Algoritmos Genéticos.

```

1 import numpy as np
2 from scipy.integrate import odeint
3
4 def pathway_model(y, t, gene_expression_levels):
5     """Modelo de via metabolica simples"""
6     S, I, P = y
7     k1, k2 = gene_expression_levels # Niveis de expressao
8
9     dSdt = -k1 * S
10    dIdt = k1 * S - k2 * I
11    dPdt = k2 * I
12
13    return [dSdt, dIdt, dPdt]
14
15 def fitness_function(gene_levels):
16     """Objetivo: maximizar produto final"""
17     y0 = [10, 0, 0] # [S, I, P]
18     t = np.linspace(0, 10, 100)
19     sol = odeint(pathway_model, y0, t, args=(gene_levels,))
20     return sol[-1, 2] # Concentracao final de P
21
22 def genetic_algorithm(pop_size=50, generations=100):
23     """Algoritmo genetico simples"""
24     # Populacao inicial
25     population = np.random.uniform(0, 10, (pop_size, 2))
26
27     for gen in range(generations):
28         # Avaliar fitness
29         fitness = np.array([fitness_function(ind) for ind in population])
30
31         # Selecao
32         sorted_idx = np.argsort(fitness)[::-1]
33         population = population[sorted_idx]
34
35         # Elitismo: manter melhores 20%
36         n_elite = pop_size // 5
37         new_pop = population[:n_elite].copy()
38
39         # Crossover e mutacao
40         while len(new_pop) < pop_size:
41             parents = population[np.random.choice(n_elite, 2, replace=False)]

```



```

42     child = (parents[0] + parents[1]) / 2 # Crossover
43     child += np.random.normal(0, 0.5, 2) # Mutacao
44     child = np.clip(child, 0, 10)
45     new_pop = np.vstack([new_pop, child])
46
47     population = new_pop[:pop_size]
48
49     return population[0]
50
51 # Executar
52 optimal_levels = genetic_algorithm()
53 print(f"Niveis otimos de expressao: {optimal_levels}")

```

Listing 14.6: AG para otimização de expressão gênica

14.5 Biologia Sintética de Células Inteiras

Genomas Mínimos.

Conceito

Conjunto mínimo de genes necessários para vida

Redução sistemática do genoma, Identificar genes essenciais e Criar chassis simplificado.

Menor carga metabólica, Mais previsível, Reduz interações indesejadas e Plataforma para engenharia.

Mycoplasma mycoides, 531 kb (vs 1079 kb original), 473 genes, Primeiro genoma mínimo sintético e Venter Institute, 2016.

Desafios

Crescimento lento, Genes de função desconhecida e Dependência de contexto.

Xenobiologia e Código Genético Expandido.

Definição (Xenobiologia).

Criação de sistemas biológicos com química alternativa à base da vida natural

Bases não-naturais (X, Y), Aminoácidos sintéticos, XNA (xeno nucleic acids) e Ribossomos ortogonais.

Contenção biológica, Propriedades novas e Bioortogonalidade.

9

— IMP:

Romesberg lab: organismos com 6 letras genéticas (2017)

Sistemas Cell-Free.

— Síntese Cell-Free —

Produção de proteínas e metabolismo sem células vivas

Extratos celulares, Componentes purificados (PURE system) e Adicionar DNA/RNA template.

Sem viabilidade celular, Controle preciso, Sem membranas, Prototipagem rápida e Proteínas tóxicas.

Produção de proteínas, Biossensores, Terapêutica, Educação e Biologia sintética.

Produtos Comerciais

PURExpress (NEB) e myTXTL (Arbor Biosciences).

Protocélulas e Células Artificiais.

Protocélulas

Estruturas sintéticas que mimetizam propriedades celulares

Vesículas lipídicas, Contêm maquinaria molecular, Auto-replicação primitiva e Origem da vida.

Membrana (lipossomas), Genoma (DNA/RNA), Metabolismo (enzimas) e Informação hereditária.

0

— IMP:

Objetivo de longo prazo: criar vida artificial de novo

Sistemas Biológicos Ortogonais.

Definição (Ortogonalidade).

Sistemas que operam independentemente dos sistemas naturais da célula hospedeira

Ribossomos ortogonais, tRNAs ortogonais, Aminoacil-tRNA sintetases, Códon não-naturais e Fatores de transcrição ortogonais.

Sem interferência com host, Controle independente e Segurança biológica.

1

Incorporar aminoácidos não-naturais sem afetar proteoma endógeno

Aplicações: Células Terapêuticas.

— Imunoterapia CAR-T —

Células T modificadas para atacar câncer

Receptor quimérico (CAR), Reconhece antígeno tumoral, Resposta imune direcionada e FDA aprovado (2017).

Switches suicidas, Controle temporal, Multi-targeting e Resistência à exaustão.

2

— IMP:

Biologia sintética na medicina personalizada

Aplicações: Living Sensors.

Biossensores Vivos

Células modificadas para detectar e responder a sinais ambientais

Deteção de poluentes, Diagnóstico de doenças, Monitoramento intestinal, Deteção de explosivos e Controle de qualidade.

Sensor (promotor responsivo), Processamento (circuito), Repórter (GFP, luz, etc) e Memória (opcional).

3

E. coli engineered para detectar inflamação intestinal e liberar terapêuticos

14.6 Desafios e Perspectivas

Context Dependence e Genetic Burden.

Context Dependence

Comportamento de partes biológicas depende do contexto

Efeitos de posição, Disponibilidade de recursos, Carga de expressão e Crosstalk regulatório.

Comportamento imprevisível, Não-composicionalidade e Necessidade de re-caracterização.

Genetic Burden

Custo metabólico da expressão heteróloga

Ribossomos, RNA polimerases, Precusores (aminoácidos, NTPs) e Energia (ATP, GTP).

Crescimento lento, Baixa produtividade, Instabilidade e Seleção contra circuito.

— IMP:

Design deve considerar carga metabólica e recursos compartilhados

Estabilidade Evolutiva.

Problema

Circuitos sintéticos podem ser perdidos ou modificados por evolução

Pressão seletiva contra carga metabólica, Mutações inativadoras, Perda de plasmídeos e Rearranjos cromossomais.

Integração cromossomal, Acoplamento essencial, Contenção genética, Seleção positiva e Toxina-antitoxina.

4

— IMP:

Trade-off entre função e fitness evolutivo

Escalonamento: Do Laboratório à Indústria.

— Desafios de Scale-up —

Condições de fermentação diferentes, Gradientes de nutrientes e oxigênio, Heterogeneidade populacional, Controle de processo, Custo de produção e Regulação e aprovação.

Shake flask (mL), Biorreator bancada (L), Piloto (10-100 L) e Industrial (10,000+ L).

Mistura e aeração, Controle pH e T, Feeding strategies, Downstream processing, Purificação e Custo de meios.

Produção de artemisinina e 1,3-PDO em escala industrial

Questões Regulatórias e Éticas.

Regulação

Organismos Geneticamente Modificados (OGM), Protocolos de biossegurança, Aprovação de produtos (FDA, EMA), Liberação ambiental e Rastreabilidade.

Física (laboratório), Biológica (auxotrofia) e Genética (switches suicidas).

Questões Éticas

"Brincar de Deus", Impacto ecológico, Bioterrorismo, Acesso e equidade, Propriedade intelectual e Consentimento informado.

Diretrizes

Protocolo de Cartagena, Convenção de Armas Biológicas e iGEM Safety Committee.

Biossegurança e Dual-Use.

— Dual-Use Research —

Pesquisa com potencial para uso benéfico e maléfico

Síntese de patógenos, Engenharia de virulência, Resistência a antibióticos e Toxinas.

Screening de síntese de DNA, Controle de acesso, Revisão ética, Educação em biossegurança e Códigos de conduta.

Síntese de poliovírus (2002), Influenza 1918 (2005) e H5N1 transmissível (2011).

— IMP:

Balanco entre progresso científico e segurança pública

Direções Futuras.

— AI-Driven Design —

Machine learning para predição, Design generativo, Otimização automática, Análise de big data e Modelagem multi-escala.

Foundries Automatizadas

Robótica, High-throughput, Design-build-test automático, Cloud labs e Redução de custos e tempo.

Novas Aplicações

Medicina regenerativa, Agricultura sustentável, Materiais vivos, Computação biológica, Terraformação e Exploração espacial.

Visão

Biologia sintética como engenharia madura com design preditivo e confiável

— IMP:

Convergência de biologia, engenharia, computação e IA

14.7 Exercícios

Exercícios - Parte 1.

Questão 1: Análise de Bistabilidade

Considere o toggle switch com parâmetros $\alpha_1 = 100$, $\alpha_2 = 20$, $\beta_1 = 2$, $\beta_2 = 2$.

Encontre os pontos de equilíbrio numericamente, Calcule o Jacobiano em cada ponto, Determine a estabilidade, Verifique a condição de bistabilidade: $\alpha_1\alpha_2 > (\beta_1)(\beta_2)$ e Simule o sistema para diferentes condições iniciais.

Questão 2: Design de Biosensor

Projete um biosensor para detectar glicose:

Identifique promotor responsivo a glicose, Escolha gene repórter apropriado, Desenhe o construto genético (BioBrick format) e Estime a sensibilidade e faixa dinâmica.

Exercícios - Parte 2.

Questão 3: Otimização Metabólica

Para produção de succinato em *E. coli*:

Use COBRApy para carregar modelo *E. coli* core, Maximize produção de succinato via FBA, Identifique genes para knockout usando FSEOF, Simule o efeito de knockouts e Calcule rendimento teórico máximo (g succinato / g glicose).

Dica

Succinato: EX_succ_e

Glicose uptake: EX_glc__D_e

Exercícios - Parte 3.

Questão 4: Simulação Estocástica

Implemente simulação de Gillespie para repressilator:

Defina reações de transcrição e tradução, Inclua degradação de mRNA e proteína, Simule para 500 unidades de tempo, Compare com solução determinística (ODEs) e Analise variabilidade célula-a-célula.

Questão 5: Design de Circuito Lógico

Projete um circuito genético que implementa porta NAND:

Desenhe diagrama do circuito, Especifique componentes (promotores, genes, repressores), Construa tabela verdade, Estime parâmetros necessários e Discuta possíveis problemas de implementação.

Projeto Integrador.

Projeto: Engenharia de Via para Produção de Bioplástico

Objetivo: Produzir poli-3-hidroxibutirato (PHB) em *E. coli*

-

Identificar genes necessários (*phaA*, *phaB*, *phaC*), Projetar construtos com promotores apropriados e Planejar estratégia de integração/plasmídeo.

-

Construir modelo ODE da via, Usar FBA para prever rendimento e Identificar gargalos metabólicos.

-

Propor knockouts para aumentar fluxo, Balancear cofatores (NADPH) e Simular estratégias de regulação dinâmica.

-

Calcular rendimento teórico máximo, Estimar custos de produção e Discutir viabilidade comercial.

14.8 Notas Bibliográficas

Referências Principais.

99

Gardner, T.S., Cantor, C.R., & Collins, J.J. (2000). Construction of a genetic toggle switch in *Escherichia coli*. *Nature*, 403, 339-342.

Elowitz, M.B. & Leibler, S. (2000). A synthetic oscillatory network of transcriptional regulators. *Nature*, 403, 335-338.

Keasling, J.D. (2010). Manufacturing molecules through metabolic engineering. *Science*, 330, 1355-1358.

Nielsen, J. & Keasling, J.D. (2016). Engineering cellular metabolism. *Cell*, 164, 1185-1197.

Cameron, D.E., Bashor, C.J., & Collins, J.J. (2014). A brief history of synthetic biology. *Nature Reviews Microbiology*, 12, 381-390.

Leitura Complementar.

Livros

Channon, K., Bromley, E.H., & Woolfson, D.N. (2008). *Synthetic Biology*. Oxford University Press, Smolke, C.D. (2018). *The Metabolic Pathway Engineering Handbook*. CRC Press e Keasling, J.D. & Venter, J.C. (2012). *Synthetic Biology*. Academic Press.

Reviews

Khalil, A.S. & Collins, J.J. (2010). Synthetic biology: applications come of age. *Nature Reviews Genetics*, 11, 367-379, Church, G.M. et al. (2014). Realizing the potential of synthetic biology. *Nature Reviews Molecular Cell Biology*, 15, 289-294 e Lim, W.A. (2010). Designing customized cell signalling circuits. *Nature Reviews Molecular Cell Biology*, 11, 393-403.

Recursos e Ferramentas.

Software e Ferramentas

-
-
-
-
-

Repositórios de Partes

-
-
-

Cursos Online

MIT OpenCourseWare: Synthetic Biology e Coursera: Bioinformatics Specialization.

Obrigado!

Capítulo 14: Design of Biological Systems

Circuitos genéticos sintéticos, engenharia metabólica,

ferramentas de design, células inteiras, desafios e perspectivas

Dúvidas? Perguntas?

Capítulo 15

Tópicos Emergentes em Biologia de Sistemas

15.1 Introdução

Visão Geral do Capítulo.

Objetivos de Aprendizagem

Explorar tópicos emergentes em biologia de sistemas, Compreender modelagem multi-escala, Conhecer análise de célula única e transcriptômica espacial, Aprender sobre inteligência artificial aplicada à biologia, Discutir doenças complexas, envelhecimento e medicina de precisão e Vislumbrar direções futuras da área.

— IMP:

Este é o capítulo final do curso, onde integramos conhecimentos anteriores e olhamos para o futuro da biologia de sistemas!

Evolução Rápida da Biologia de Sistemas.

Tendências Atuais

Integração de múltiplas camadas ômicas, Resolução de célula única e espacial, Modelagem multi-escala (molécula

organismo), IA para descoberta e predição e Medicina personalizada e digital twins.

15.2 Modelagem Multi-Escala

15.1 Modelagem Multi-Escala.

— DEF: Modelagem Multi-Escala —

Integração de modelos computacionais em diferentes níveis de organização biológica: molecular, celular, tecidual, órgão e organismo completo.

— Desafios da Integração Multi-Escala —
ms (cinética enzimática)

anos(envelhecimento), nm(proteínas)

m (organismo), 10^{12} moléculas

trilhões de células, variabilidade celular e individual e custos proibitivos de simulações completas.

— IMP:

Nenhum modelo único pode capturar todos os níveis simultaneamente!

Hierarquia Multi-Escala.

Exemplo: Metabolismo da Glicose

cinética de hexoquinase, glicólise completa, hepatócito produzindo ATP, fígado regulando glicemia e homeostase sistêmica.

Modelos Baseados em Agentes (ABM).

— DEF: Agent-Based Model —

Modelo computacional onde entidades autônomas (agentes) seguem regras simples, e comportamentos complexos emergem das interações.

Agentes autônomos, Regras locais simples, Interações entre agentes, Ambiente compartilhado e Emergência de padrões.

Crescimento tumoral, Resposta imune, Morfogênese, Dinâmica de populações e Epidemiologia.

Exemplo: Tumor

Cada célula é um agente que pode: proliferar, migrar, morrer, ou entrar em quiescência baseado em sinais locais (nutrientes, fatores de crescimento).

Exemplo Simples de ABM em Python.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class Cell:
5     """Agente representando uma célula"""
6     def __init__(self, x, y, cell_type='normal'):
7         self.x = x
8         self.y = y
9         self.type = cell_type
10        self.alive = True
```

```

11
12     def proliferate(self, environment):
13         """Celula pode proliferar se houver espaco"""
14         if environment.nutrient_level[self.x, self.y] > 0.5:
15             # Criar celula filha adjacente
16             new_x = self.x + np.random.randint(-1, 2)
17             new_y = self.y + np.random.randint(-1, 2)
18             if environment.is_empty(new_x, new_y):
19                 return Cell(new_x, new_y, self.type)
20         return None
21
22     def move(self, environment):
23         """Celula se move aleatoriamente"""
24         dx, dy = np.random.randint(-1, 2, 2)
25         new_x, new_y = self.x + dx, self.y + dy
26         if environment.is_valid(new_x, new_y):
27             self.x, self.y = new_x, new_y

```

Listing 15.1: Agent-Based Model básico

Modelos Híbridos: Discreto + Contínuo.

Abordagem Híbrida

Combina modelos discretos (células individuais) com modelos contínuos (difusão de nutrientes, fatores de crescimento).

$t = D^2C - \sum_i i(C)$ e crescimento celular baseado em $C(x_i, y_i)$.

Órgãos Virtuais e Fisiologia de Corpo Inteiro.

Coração virtual (eletrof.), Fígado virtual (metabolismo), Pulmão virtual (ventilação) e Rim virtual (filtração).

-
-
-

Desafio

Integrar dinâmicas de múltiplos órgãos acoplados!

Exemplo: Metabolismo Hepático Multi-Escala.

Níveis Integrados

cinética de enzimas (CYP450, glicoquinase), hepatócitos individuais (zonas periportal/pericentral), gradiente de O_2 e nutrientes, fluxo sanguíneo, metabolismo sistêmico e regulação hormonal (insulina, glucagon).

Modelagem

ODEs para metabolismo celular, PDEs para difusão espacial, ABM para heterogeneidade hepatócito e Controle sistêmico (feedback hormonal).

— IMP:

Permite simular resposta a drogas, doenças hepáticas, cirrose!

15.3 Biologia de Sistemas de Célula Única

15.2 Biologia de Sistemas de Célula Única.

Revolução da Célula Única

Tecnologias que permitem medir milhares de moléculas em células individuais, revelando heterogeneidade antes invisível.

scRNA-seq, scATAC-seq (cromatina), scProteomics (CyTOF), scMetabolomics e Multi-omic single-cell.

Descoberta de tipos celulares, Trajetórias de diferenciação, Heterogeneidade tumoral, Desenvolvimento embrionário e Resposta imune.

— IMP:

A célula única é a unidade fundamental da vida. Tecidos aparentemente homogêneos são, na verdade, mosaicos heterogêneos!

Single-Cell RNA-seq (scRNA-seq).

Identificação de Tipos Celulares e Clustering.

— Pipeline de Análise scRNA-seq —

remover células de baixa qualidade, corrigir profundidade de sequenciamento, genes altamente variáveis, PCA, UMAP, t-SNE, Louvain, Leiden e marcadores conhecidos.

Ferramentas

(Python), (R), (trajectories) e (10x).

Análise scRNA-seq com Scanpy.

```
1 import scanpy as sc
2 import numpy as np
3
4 # Carregar dados
5 adata = sc.read_10x_mtx('filtered_feature_bc_matrix/')
6
7 # QC: filtrar células e genes
8 sc.pp.filter_cells(adata, min_genes=200)
9 sc.pp.filter_genes(adata, min_cells=3)
10
11 # Calcular % mitocondriais
12 adata.var['mt'] = adata.var_names.str.startswith('MT-')
13 sc.pp.calculate_qc_metrics(
14     adata, qc_vars=['mt'], inplace=True
15 )
16
```

```

17 # Filtrar células ruins
18 adata = adata[adata.obs.pct_counts_mt < 5, :]
19
20 # Normalizar e log-transformar
21 sc.pp.normalize_total(adata, target_sum=1e4)
22 sc.pp.log1p(adata)
23
24 # Identificar genes altamente variáveis
25 sc.pp.highly_variable_genes(adata, n_top_genes=2000)
26
27 # PCA
28 sc.tl.pca(adata, svd_solver='arpack')

```

Listing 15.2: Análise básica com scanpy

Clustering e Visualização.

```

1 # Construir grafo de vizinhos
2 sc.pp.neighbors(adata, n_neighbors=10, n_pcs=40)
3
4 # Clustering com algoritmo Leiden
5 sc.tl.leiden(adata, resolution=0.5)
6
7 # UMAP para visualizacao
8 sc.tl.umap(adata)
9
10 # Plotar
11 sc.pl.umap(adata, color=['leiden', 'n_genes'])
12
13 # Encontrar genes marcadores
14 sc.tl.rank_genes_groups(adata, 'leiden', method='wilcoxon')
15 sc.pl.rank_genes_groups(adata, n_genes=20, sharey=False)
16
17 # Anotar clusters manualmente
18 cluster_names = {
19     '0': 'T_cells',
20     '1': 'B_cells',
21     '2': 'Monocytes',
22     '3': 'NK_cells',
23     '4': 'Dendritic_cells'
24 }
25 adata.obs['cell_type'] = adata.obs['leiden'].map(cluster_names)
26 sc.pl.umap(adata, color='cell_type')

```

Listing 15.3: Clustering e UMAP

Pseudotempo e Inferência de Trajetórias.

Definição (Pseudotempo).

Ordem inferida de células ao longo de um processo dinâmico (diferenciação, ativação) baseada em similaridade transcricional.

Monocle, Slingshot, PAGA, RNA velocity

Transcriptômica Espacial.

Resolvendo a Localização

scRNA-seq perde informação espacial. Novas tecnologias preservam contexto espacial:

(10x): arrays espaciais, : imaging baseado em FISH, : multiplexado, : beads espaciais e : ultra-alta resolução.

Arquitetura tecidual, Microambiente tumoral, Organização cerebral, Nicho de células-tronco e Desenvolvimento embrionário.

— IMP:

Combina melhor dos dois mundos: expressão gênica detalhada + localização espacial!

Comunicação Célula-Célula.

Inferindo Interações

Análise de pares ligante-receptor para identificar comunicação entre tipos celulares.

Ferramentas

-
-
-
-

microambiente tumoral, resposta imune, nicho de células-tronco

15.4 Inteligência Artificial em Biologia de Sistemas

15.3 IA em Biologia de Sistemas.

A Revolução da IA

Machine learning e deep learning estão transformando a biologia de sistemas, permitindo análise de dados massivos e descoberta de padrões complexos.

Supervisionado (classificação), Não-supervisionado (clustering), Reforço (otimização) e Ensemble methods.

Redes neurais profundas, CNNs (imagens), RNNs/Transformers (sequências), GNNs (grafos/redes) e GANs (geração).

— IMP:

IA não substitui modelagem mecânica, mas complementa: híbrido é o futuro!

Aprendizado Profundo para Dados Biológicos.

Predição de expressão gênica, Classificação de tipos celulares, Predição de estrutura proteica (AlphaFold), Design de drogas e Diagnóstico de imagens médicas.

Rede Neural para Classificação.

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class GeneExpressionClassifier(nn.Module):
6     """Rede neural para classificar amostras"""
7     def __init__(self, input_dim, hidden_dim, output_dim):
8         super().__init__()
9         self.fc1 = nn.Linear(input_dim, hidden_dim)
10        self.relu = nn.ReLU()
11        self.dropout = nn.Dropout(0.3)
12        self.fc2 = nn.Linear(hidden_dim, hidden_dim // 2)
13        self.fc3 = nn.Linear(hidden_dim // 2, output_dim)
14        self.softmax = nn.Softmax(dim=1)
15
16    def forward(self, x):
17        x = self.fc1(x)
18        x = self.relu(x)
19        x = self.dropout(x)
20        x = self.fc2(x)
21        x = self.relu(x)
22        x = self.fc3(x)
23        return self.softmax(x)
24
25 # Inicializar modelo
26 model = GeneExpressionClassifier(
27     input_dim=20000, # genes
28     hidden_dim=256,
29     output_dim=5     # classes
30 )

```

Listing 15.4: RNA simples com PyTorch

Graph Neural Networks (GNNs).

Definição (GNN).

Redes neurais que operam diretamente em grafos, aprendendo representações de nós e arestas. Ideais para redes biológicas!

Predição de função de proteínas, Descoberta de drogas, Propriedades moleculares, Redes regulatórias e Interações proteína-proteína.

GCN, GraphSAGE, GAT, Message Passing NN

GNN Básico para Rede Molecular.

```
1 import torch
2 from torch_geometric.nn import GCNConv, global_mean_pool
3
4 class MolecularGNN(torch.nn.Module):
5     """GNN para predicao de propriedades moleculares"""
6     def __init__(self, num_features, hidden_dim, num_classes):
7         super().__init__()
8         self.conv1 = GCNConv(num_features, hidden_dim)
9         self.conv2 = GCNConv(hidden_dim, hidden_dim)
10        self.conv3 = GCNConv(hidden_dim, hidden_dim)
11        self.fc = torch.nn.Linear(hidden_dim, num_classes)
12
13    def forward(self, x, edge_index, batch):
14        # Node embeddings
15        x = self.conv1(x, edge_index)
16        x = torch.relu(x)
17        x = self.conv2(x, edge_index)
18        x = torch.relu(x)
19        x = self.conv3(x, edge_index)
20
21        # Graph-level pooling
22        x = global_mean_pool(x, batch)
23
24        # Prediction
25        return self.fc(x)
```

Listing 15.5: GNN com PyTorch Geometric

Transformers e AlphaFold.

Mecanismo de Atenção

Transformers usam "attention" para capturar dependências de longo alcance em sequências.

AlphaFold 2 (DeepMind, 2020)

Revolucionou predição de estrutura proteica, Precisão próxima a estruturas experimentais, Baseia-se em: transformers + MSA + embeddings estruturais e Predisse estrutura de >200M proteínas (AlphaFold DB).

— IMP:

AlphaFold é um dos maiores sucessos de IA em biologia!

Modelos Generativos: Design de Moléculas.

Variational Autoencoder, Generative Adversarial Network, Diffusion models e Reinforcement learning.

Design de proteínas, Design de drogas (SMILES) e Otimização de anticorpos.

Exemplo

MegaMolBART (transformers) gera moléculas drug-like otimizadas
IA Interpretável e Explicável.

— IMP:

Modelos de caixa-preta são problemáticos em biologia e medicina!

Abordagens para Interpretabilidade

quais genes são mais importantes?, contribuição de cada feature e o que o modelo está "olhando".

-

integrar conhecimento biológico.

Inferência Causal vs Correlação

Correlação: A e B variam juntos, Causação: AB (mudança em A causa mudança em B) e Ferramentas: causal graphs, do-calculus, experimentos perturbacionais.

15.5 Doenças Complexas e Envelhecimento

15.4 Doenças Complexas como Perturbações de Rede.

Abordagem de Network Medicine

Doenças complexas (câncer, diabetes, Alzheimer) não resultam de defeito em gene único, mas de perturbações em redes moleculares.

0

genes mutados/desregulados

perturbação se espalha pela rede

Multi-Omics para Envelhecimento.

Instabilidade genômica, Atrito de telômeros, Alterações epigenéticas, Perda de proteostase, Senescência celular, Disfunção mitocondrial, Desregulação nutricional, Exaustão de células-tronco e Comunicação intercelular alterada.

Genômica (mutações), Epigenômica (metilação), Transcriptômica (expressão), Proteômica (proteostase) e Metabolômica (NAD⁺, etc.).

Relógios Epigenéticos

DNA methylation age: preditor de idade biológica (Horvath clock)

Vias de Longevidade.

Pathways Conservados

Vias moleculares que modulam longevidade em múltiplas espécies:

1

Restrição calórica, Rapamicina (inibe mTOR), Metformina (ativa AMPK), NAD⁺ boosters (ativam sirtuínas) e Senolytics (eliminam células senescentes).

Redes de Comorbidade.

Comorbidity Networks

Grafos representando co-ocorrência de doenças em populações, revelando mecanismos compartilhados.

2

diabetes mellitus tipo 2 (DM2) é hub central

priorizar intervenções, predizer risco

Sistemas para Preparação Pandêmica.

— IMP:

COVID-19 demonstrou necessidade de biologia de sistemas para resposta rápida!

Abordagens Sistêmicas

sequenciamento em tempo real de variantes, previsão de disseminação (SIR, SEIR), identificar fatores de risco, IA para identificar drogas existentes, epitopos computacionais e entender síndrome pós-COVID.

integração de dados heterogêneos + modelos + infraestrutura computacional

Medicina de Precisão: Futuro.

Genoma pessoal (WGS), Multi-omics longitudinal, Microbioma, Wearables (monitoramento) e Histórico médico completo.

Custo, Interpretação clínica, Privacidade e Equidade de acesso.

3

15.6 Direções Futuras

15.5 Direções Futuras.

O Futuro da Biologia de Sistemas

A área está em rápida evolução. Várias tecnologias e abordagens prometem revolucionar nossa compreensão da vida.

Laboratórios automatizados, Plataformas robóticas, Microfluídica, Organoides e organs-on-chip, Edição genômica (CRISPR) e Biologia sintética.

Computação quântica, Edge computing, Federated learning, Blockchain para dados e Cloud-based pipelines.

Laboratórios Automatizados e Robótica.

4

Experimentos 24/7, Alta reprodutibilidade, Exploração massiva do espaço de parâmetros e Loop fechado: IA propõe experimentos

robôexecuta

análise

novahipótese.

Emerald Cloud Lab, Strateos, Ada (Carnegie Mellon)

Geração de Hipóteses por IA.

IA Criativa

Sistemas de IA que não apenas analisam dados, mas propõem novas hipóteses biológicas testáveis.

Abordagens

extrair relações de milhões de papers, raciocínio sobre entidades e relações, explorar espaço de experimentos, propor novas moléculas, vias e identificar relações causais.

— IMP:

Humanos continuam essenciais: interpretar, validar, contextualizar!

Digital Twins para Medicina Personalizada.

Definição (Digital Twin).

Réplica computacional dinâmica de um paciente individual, integrando dados multi-omics, fisiológicos e clínicos em tempo real.

5

Testar tratamentos in silico, Predizer resposta a drogas, Otimizar dosagem e Antecipar eventos adversos.

Computação Quântica para Simulação Biológica.

Computação Quântica

Computadores quânticos exploram superposição e entrelaçamento para resolver problemas intratáveis classicamente.

dinâmica eletrônica precisa, explorar espaço conformacional exponencial, otimização combinatorial, redes biológicas massivas e aceleração de algoritmos.

Realidade Atual

Ainda estamos na era NISQ (Noisy Intermediate-Scale Quantum). Aplicações práticas em biologia ainda distantes, mas pesquisa ativa!

Integração de Microbioma e Fatores Ambientais.

— IMP:

O hospedeiro não pode ser estudado isoladamente: microbioma e ambiente são partes integrantes do sistema!

10

maiscélulasquehumanas, Metabolismocomplementar, Modulaçãoimune, Eixointestino—cérebroeRe

16S rRNA-seq (taxonomia), Shotgun metagenomics (função) e Metatranscriptomics.

Dieta, Poluentes, Estresse, Atividade física, Sono e Medicamentos.

Holobionte

Organismo + microbioma = unidade evolutiva e funcional

Ciência Aberta e Compartilhamento de Dados.

Movimento Open Science

Transparência, reprodutibilidade e acesso aberto são cruciais para acelerar descobertas.

indable: dados encontráveis, ccessible: acessíveis, nteroperable: interoperáveis e eusable: reutilizáveis.

Repositórios públicos (GEO, SRA, PDB, etc.), Pré-prints (bioRxiv, medRxiv), Código aberto (GitHub), Notebooks reprodutíveis (Jupyter) e Protocolos detalhados (protocols.io).

— IMP:

Colaboração global é essencial para enfrentar desafios complexos!

Considerações Éticas.

Questões Éticas Emergentes

genomas pessoais, re-identificação, uso secundário de dados, acesso a tecnologias avançadas (precision medicine gap), datasets não-representativos, CRISPR em embriões, biossegurança, dual use e patentes de genes, dados.

— IMP:

Cientistas têm responsabilidade de considerar implicações sociais de seu trabalho!

15.7 Exercícios

Exercícios - Parte 1.

— Questão 1: Modelagem Multi-Escala —

Descreva como você modelaria o metabolismo da glicose integrando os níveis:

Molecular (cinética de hexoquinase), Celular (hepatócito) e Órgão (fígado).

Quais são os principais desafios de acoplamento entre escalas?

— Questão 2: Single-Cell RNA-seq —

Dado um dataset scRNA-seq de um tumor:

Como você identificaria diferentes tipos celulares?, Como inferiria comunicação entre células tumorais e células imunes? e Qual a importância da informação espacial?.

Exercícios - Parte 2.

Questão 3: Machine Learning

Quando usar modelos de ML vs. modelos mecanísticos?, Explique o problema de interpretabilidade em redes neurais profundas e Como você validaria um modelo de ML para predição clínica?.

Projeto Prático

Escolha um tópico emergente (célula única, IA, envelhecimento) e:

Revise a literatura recente (últimos 2 anos), Identifique um dataset público relevante, Aplique pelo menos uma técnica computacional, Gere visualizações e interprete resultados e Discuta limitações e direções futuras.

15.8 Notas Bibliográficas

Referências Principais.

99

Jumper J. et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*.

Wolf F.A. et al. (2018). SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology*.

López-Otín C. et al. (2023). Hallmarks of aging: An expanding universe. *Cell*.

Barabási A.L. et al. (2011). Network medicine: a network-based approach to human disease. *Nature Reviews Genetics*.

Moses C. et al. (2021). Museum of spatial transcriptomics. Nature Methods.

Leitura Complementar.

Artigos de Revisão Recentes

Integrative multi-omics approaches (Nature Reviews, 2023), Single-cell multi-omics technologies (Nature Biotechnology, 2023), AI for drug discovery (Nature Machine Intelligence, 2024), Digital twins in healthcare (Nature Medicine, 2023) e Longevity and healthspan interventions (Cell, 2023).

Recursos Online

-
-
-
-

Obrigado!

Capítulo 15: Emerging Topics

Conclusão do Curso

Este curso cobriu desde fundamentos de sistemas biológicos até os tópicos mais avançados e emergentes da biologia de sistemas. Vocês agora têm as ferramentas conceituais e computacionais para:

Modelar sistemas biológicos em múltiplos níveis, Analisar dados omics em larga escala, Integrar conhecimento multi-escala e multi-omic, Aplicar IA e aprendizado de máquina e Contribuir para o futuro da medicina de precisão.

O futuro da biologia de sistemas é brilhante. sorte em suas jornadas científicas!